

APS Global Summit GDS Tutorial: **AI-Ready Time Series Datasets in Physics**

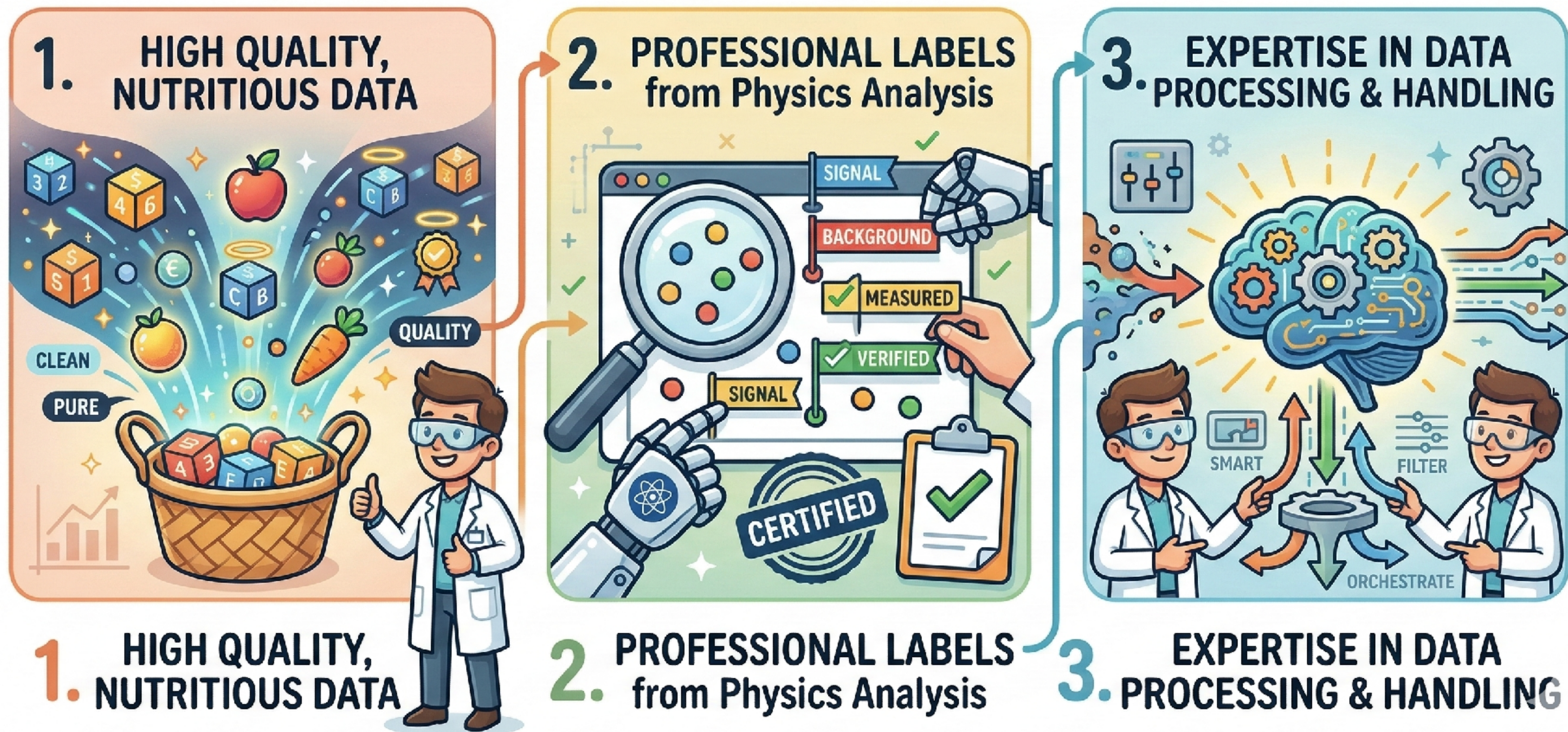
Aobo Li
Halicioğlu Data Science Institute
Department of Physics
UC San Diego



“Data is the fuel for AI, and modern AI systems need not only calories but also high-quality nutrition”

— Andrew Ng

As Physicists, We have access to...



Majorana Dataset

<https://doi.org/10.5281/zenodo.8257027>

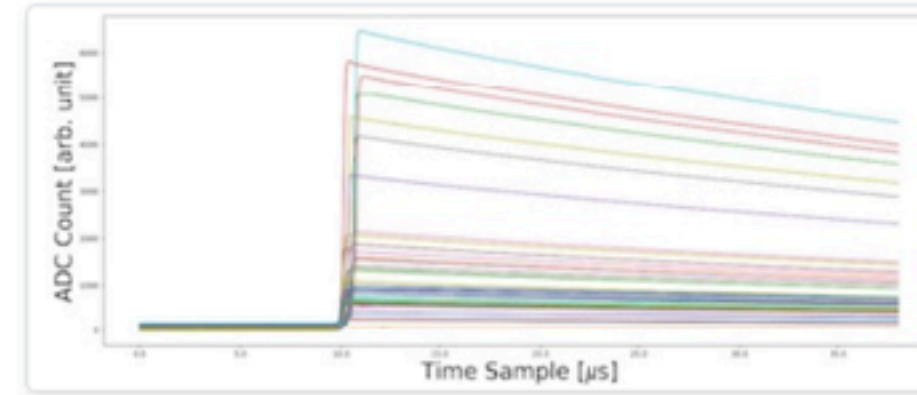


Fig 1. Sample Waveforms

MAJORANA DEMONSTRATOR DATA RELEASE

BASIC DATASET HIGH PURITY GERMANIUM DETECTOR SIGNALS

This dataset contains over **3 million data points** derived from the **Majorana Demonstrator experiment**, which utilizes High Purity Germanium Detectors to search for Neutrinoless Double-Beta Decay. Each data point represents a real time-series waveform generated by the detector.

Input Data: 1D Numpy vector with 4000 samples (Time Series).

🔗 **Labels:** 4 Binary Classification labels + 1 Energy Regression label.

🎯 **Goal:** Train ML models to use the waveform as input to predict the analysis labels.

Format: HDF5 (See Tutorial 1 for reading instructions).

📄 Download Dataset (Zenodo)

📖 Read Description (arXiv)

TIDMAD Dataset

NeurIPS 2025 D&B Track Spotlight Paper

<https://openreview.net/forum?id=GgKHKZgg0Y>

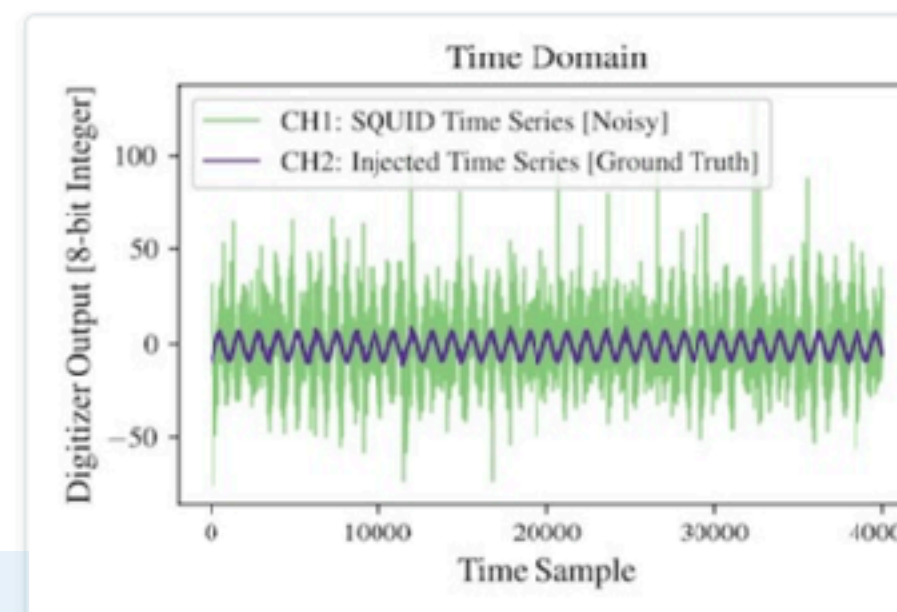


Fig 2. Noisy Input vs Ground Truth

TIDMAD: DARK MATTER DISCOVERY WITH AI DENOISING

ADVANCED DATASET QUANTUM-ENABLED AXION DETECTOR DATA

Generated by a cutting-edge **quantum-enabled lumped-element axion detector**, this dataset targets the search for dark matter (85% of the universe's matter). It is an ultra-long time series dataset (>1 TB) containing 8-bit integer samples.

The Logic: We inject a clean dark-matter-like signal (Ground Truth) and read it out via a quantum sensor (Noisy Input). **The Task:** Build a Neural Network to denoise the readout and recover the injected signal.

📁 **Structure:** HDF5 format. Split into Training (20 files), Validation (20 files), and Science (208 files).

Benchmark 1: Denoising Score (calculated on Validation Set).

🌐 **Benchmark 2:** Dark Matter Limit (Physics result calculated on Science Set).

⚠️ **High Difficulty:** This dataset is extremely large (>1TB). Beginners are strongly suggested to complete other tutorials first. If starting here, please begin with a single training file.

📄 Download & Scripts

📖 Benchmark Paper

TIDMAD: A Sandbox for Physics-Informed LLM Agents in Dark Matter Searches



Mon. March 16, 8:36 — 9:12 a.m.



Convention Center, Meeting Room 507

Check out AI-Tutorials on my Website!

<https://aobol.github.io/AoboLi/>



ABOUT

RESEARCH

GROUP MEMBERS

AI TUTORIALS

AWARDS

TEACHING

APPOINTMENT

EDUCATION

AI TUTORIALS

INTRODUCTION

Welcome to this AI/ML tutorial repository, developed during my faculty career. This tutorial series aims to provide **interdisciplinary training**, specifically designed to help physicists cross disciplinary borders into the world of Artificial Intelligence.

No prior AI/ML expertise is needed. We start from zero. During these tutorials, you will have the opportunity to move beyond theory and play with **real data derived from cutting-edge physics detectors**. You will learn how to build neural network models from scratch using this data.

Note: This is a growing repository. We plan to release more physics detector datasets and create additional tutorials in the future.

CONTENT OVERVIEW

- **Datasets:** Ranges from the BASIC (Majorana Demonstrator Data Release) to the ADVANCED (TIDMAD dataset for dark matter discovery).
- **Tutorials:** Covers the essentials ("AI In a Nutshell"), coding cookbooks, using ChatGPT for ML, and two additional Machine Learning Course Series.

SUGGESTED LEARNING PATHS

- **The Essential Path (Quick Start):**
BASIC DATASET → 1. AI In a Nutshell (run Jupyter Notebook) → 3. Using ChatGPT (update Jupyter Notebook)
- **The Full Exploration (Deep Dive):**
BASIC DATASET → 1. Nutshell → 2. Cookbook → 3. ChatGPT → 4. Practical Series → 5. Bootcamp → ADVANCED DATASET.

Time-series definition

A time series is a set of time-tagged measurements: $\{X_i(t_i)\}$ possibly with observation errors σ_i

Concept

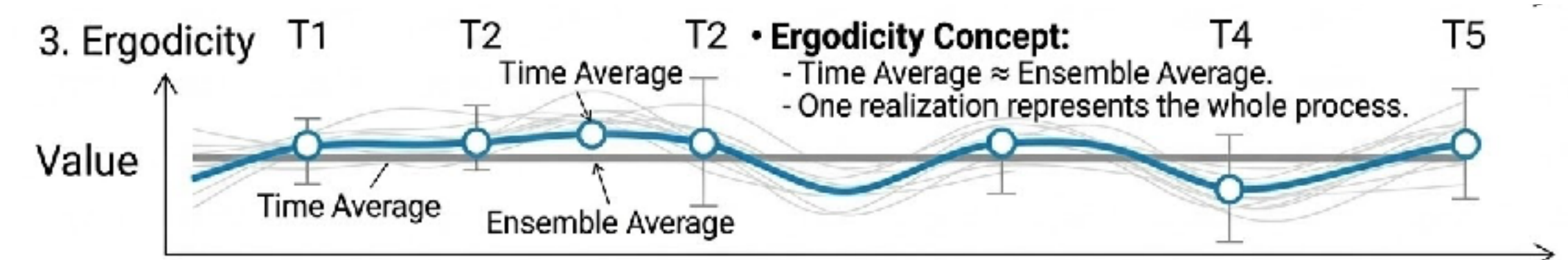
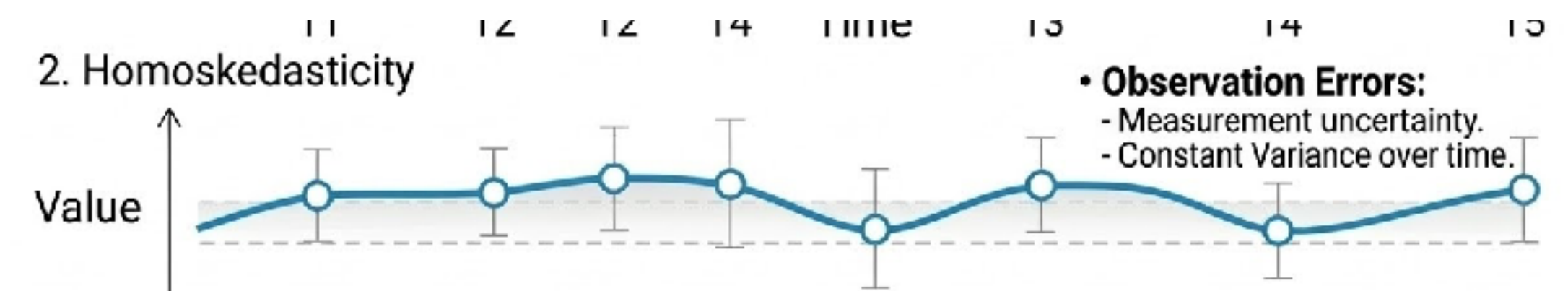
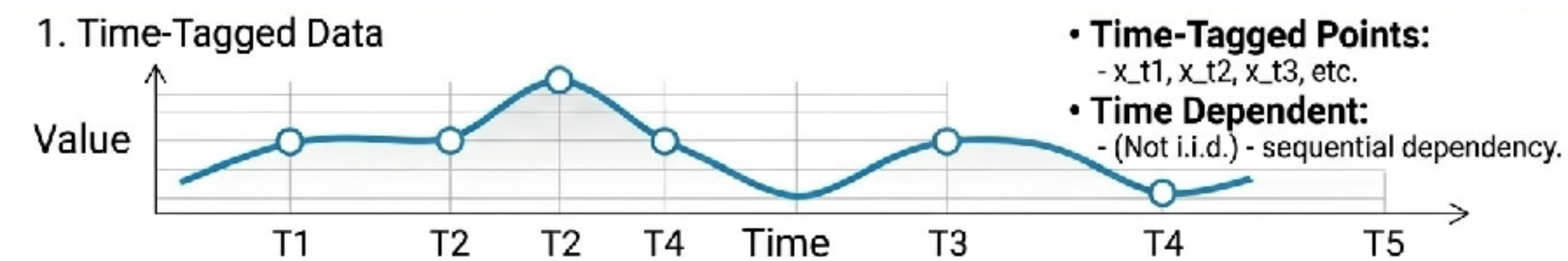
Not i.i.d.: Data is sequential (i.e. next point depends on previous point)

Concept

Homoskedasticity: All errors drawn from same process

Concept

Ergodicity: The time average for one sequence is the same as the ensemble average



Stationarity of Time Series

The time-series generation process is time independent:

Concept

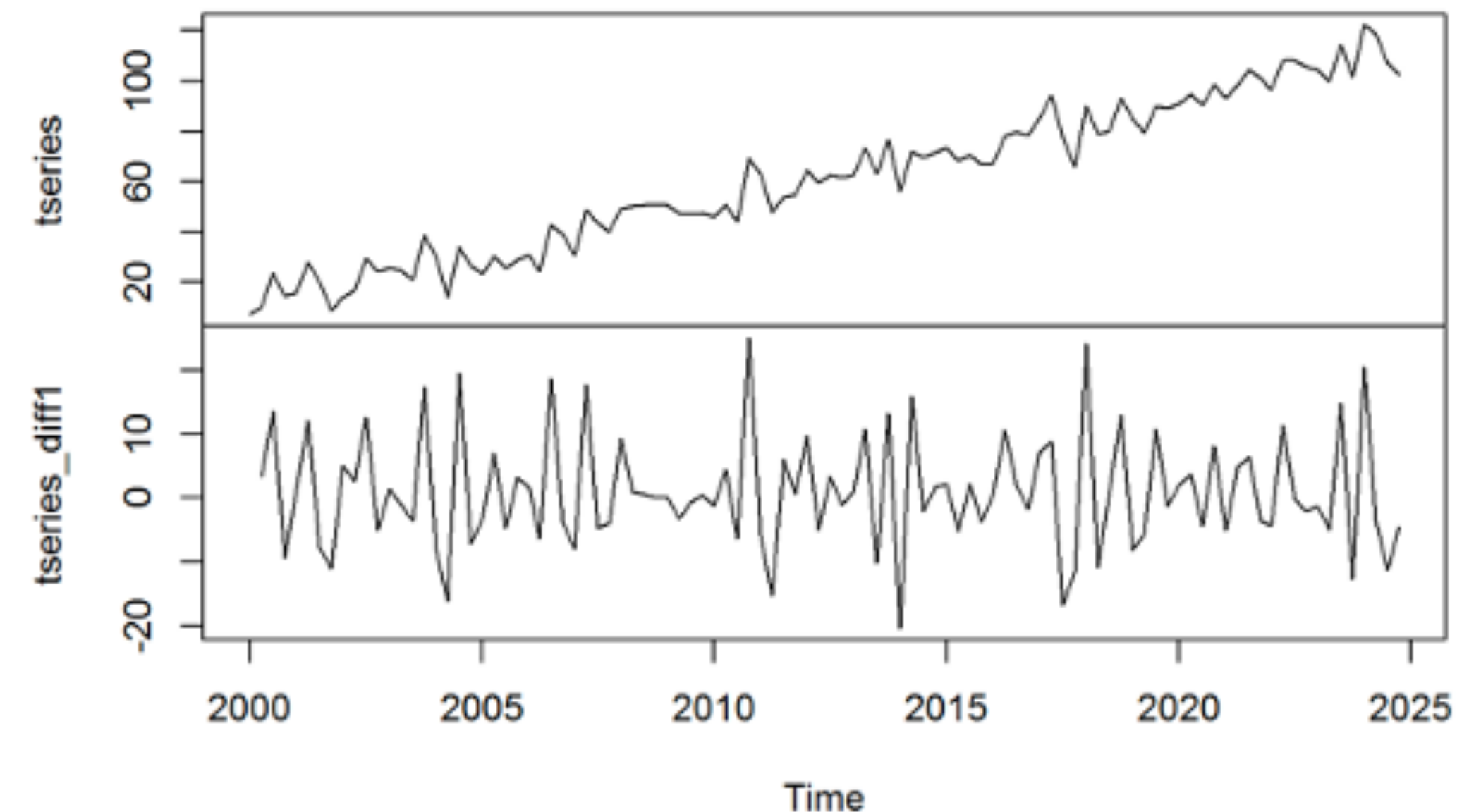
Strong: the joint probability distribution is translationally invariant

Weak: Mean, variance, autocorrelation are constant

- Transformations to achieve stationarity (constant location and scale)

- Difference the data: $Z_i = X_i - X_{i-1}$
- Detrend the data: $Z(t) = X(t) - f(t)$
- Stabilize the variance:

$$Z(t) = \sqrt{X(t) + A} \text{ or } \log(X(t) + A)$$



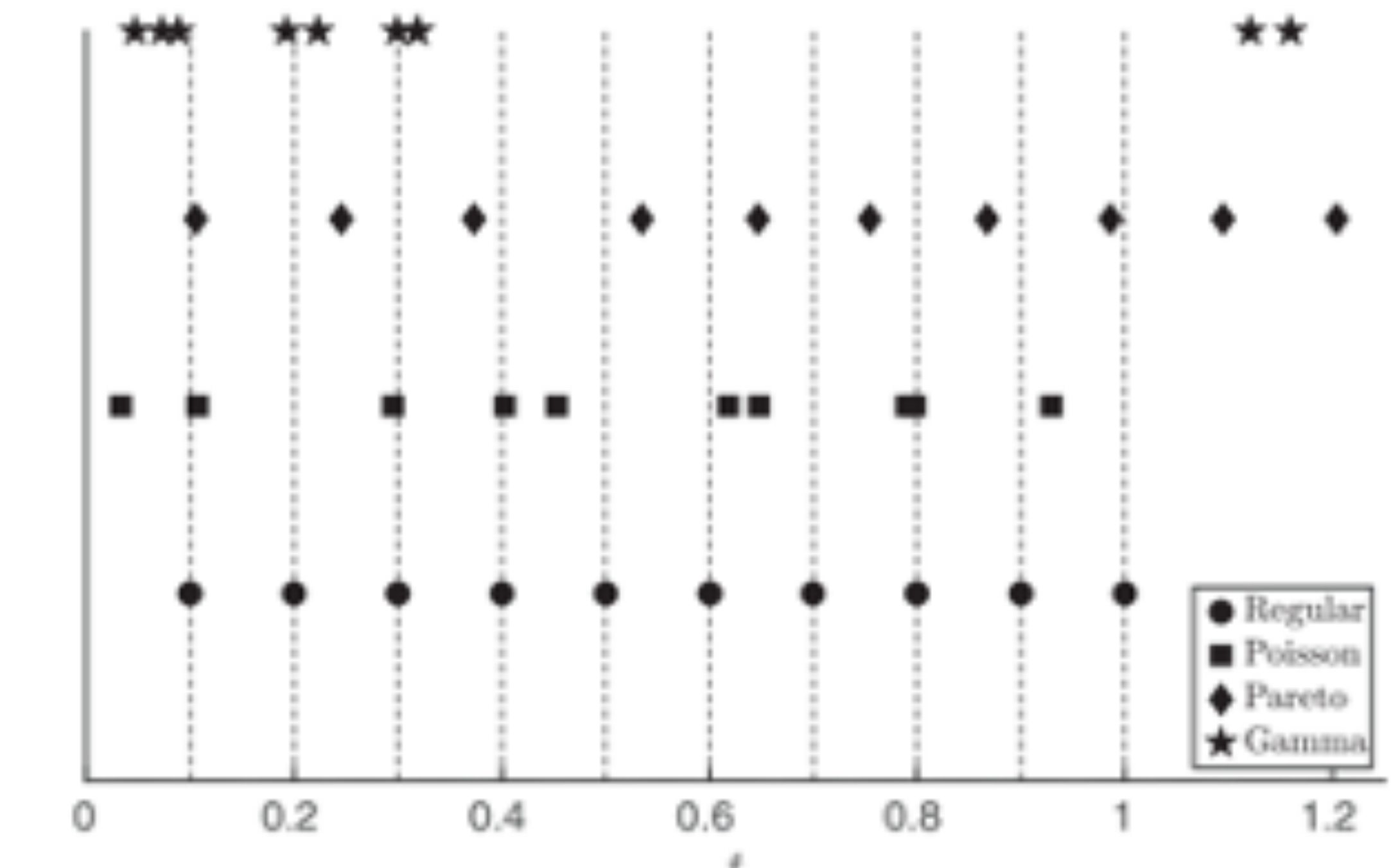
Sampling

- Even or regular sampling: $y(t) = x(t_0 + n\Delta t)$ where $n = 0, 1, \dots, m$
- Uneven or irregular sampling: $y(t) = x(t_0), \dots, x(t_m)$
- Regularization/resampling

- Bin data onto regular grid:

$$y(t) = \frac{\sum_i w_i x_i}{\sum_i w_i} \text{ for } t_i \in [t_a, t_b]$$

- Interpolate: linear, spline, Gaussian process
- Continuous time process:
 - Observations are a random sample drawn from a continuous process described by some differential equation



Majorana Dataset

<https://doi.org/10.5281/zenodo.8257027>

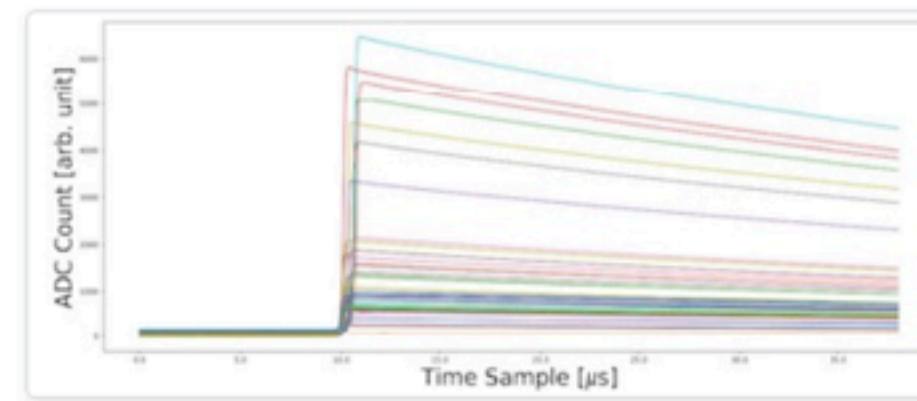


Fig 1. Sample Waveforms

MAJORANA DEMONSTRATOR DATA RELEASE

BASIC DATASET HIGH PURITY GERMANIUM DETECTOR SIGNALS

This dataset contains over **3 million data points** derived from the **Majorana Demonstrator experiment**, which utilizes High Purity Germanium Detectors to search for Neutrinoless Double-Beta Decay. Each data point represents a real time-series waveform generated by the detector.

Input Data: 1D Numpy vector with 4000 samples (Time Series).

🔗 **Labels:** 4 Binary Classification labels + 1 Energy Regression label.

🎯 **Goal:** Train ML models to use the waveform as input to predict the analysis labels.

Format: HDF5 (See Tutorial 1 for reading instructions).

📄 Download Dataset (Zenodo)

📖 Read Description (arXiv)

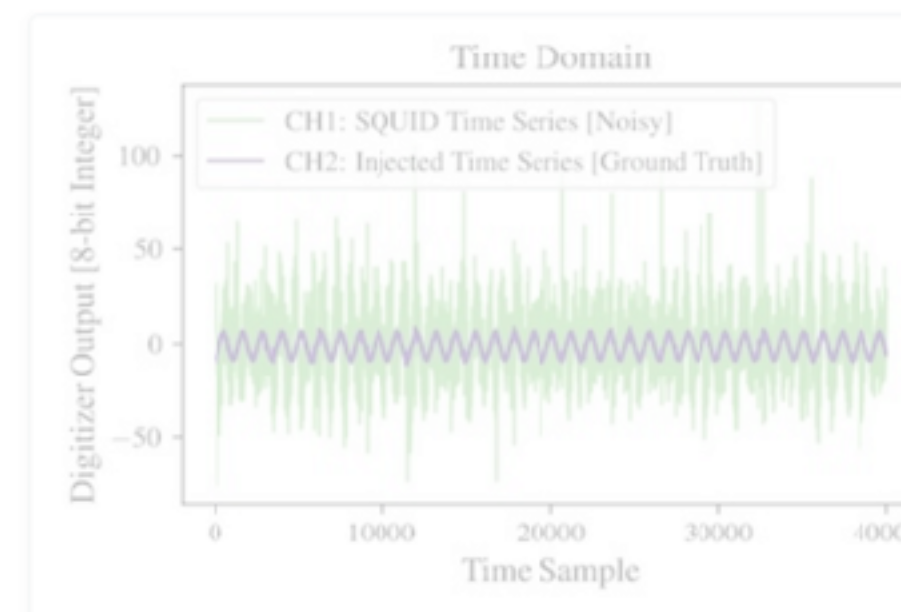


Fig 2. Noisy Input vs Ground Truth

TIDMAD: DARK MATTER DISCOVERY WITH AI DENOISING

ADVANCED DATASET QUANTUM-ENABLED AXION DETECTOR DATA

Generated by a cutting-edge **quantum-enabled lumped-element axion detector**, this dataset targets the search for dark matter (85% of the universe's matter). It is an ultra-long time series dataset (>1 TB) containing 8-bit integer samples.

The Logic: We inject a clean dark-matter-like signal (Ground Truth) and read it out via a quantum sensor (Noisy Input). **The Task:** Build a Neural Network to denoise the readout and recover the injected signal.

📁 **Structure:** HDF5 format. Split into Training (20 files), Validation (20 files), and Science (208 files).

Benchmark 1: Denoising Score (calculated on Validation Set).

🌐 **Benchmark 2:** Dark Matter Limit (Physics result calculated on Science Set).

⚠️ **High Difficulty:** This dataset is extremely large (>1TB). Beginners are strongly suggested to complete other tutorials first. If starting here, please begin with a single training file.

📄 Download & Scripts

📖 Benchmark Paper

The Physics of MAJORANA DATASET

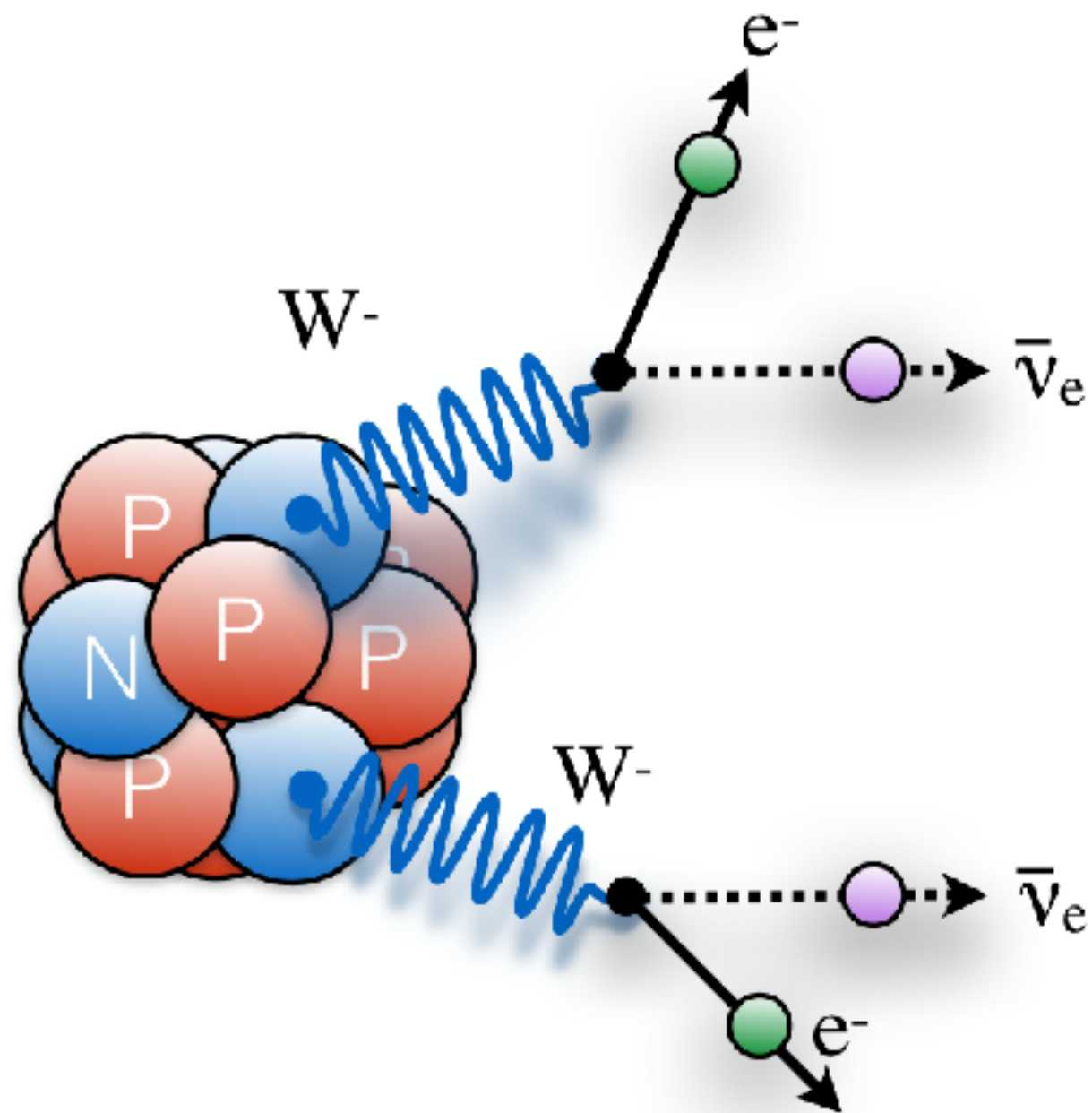
Double Beta Decay ($2\nu\beta\beta$)

First proposed by Maria Goeppert Mayer in 1935

First detection by Elliott, Hahn, Moe, in 1987

Decay half-life $T_{\frac{1}{2}} \sim 10^{14} - 10^{24} \text{ yrs}$

Much longer than the age of universe!



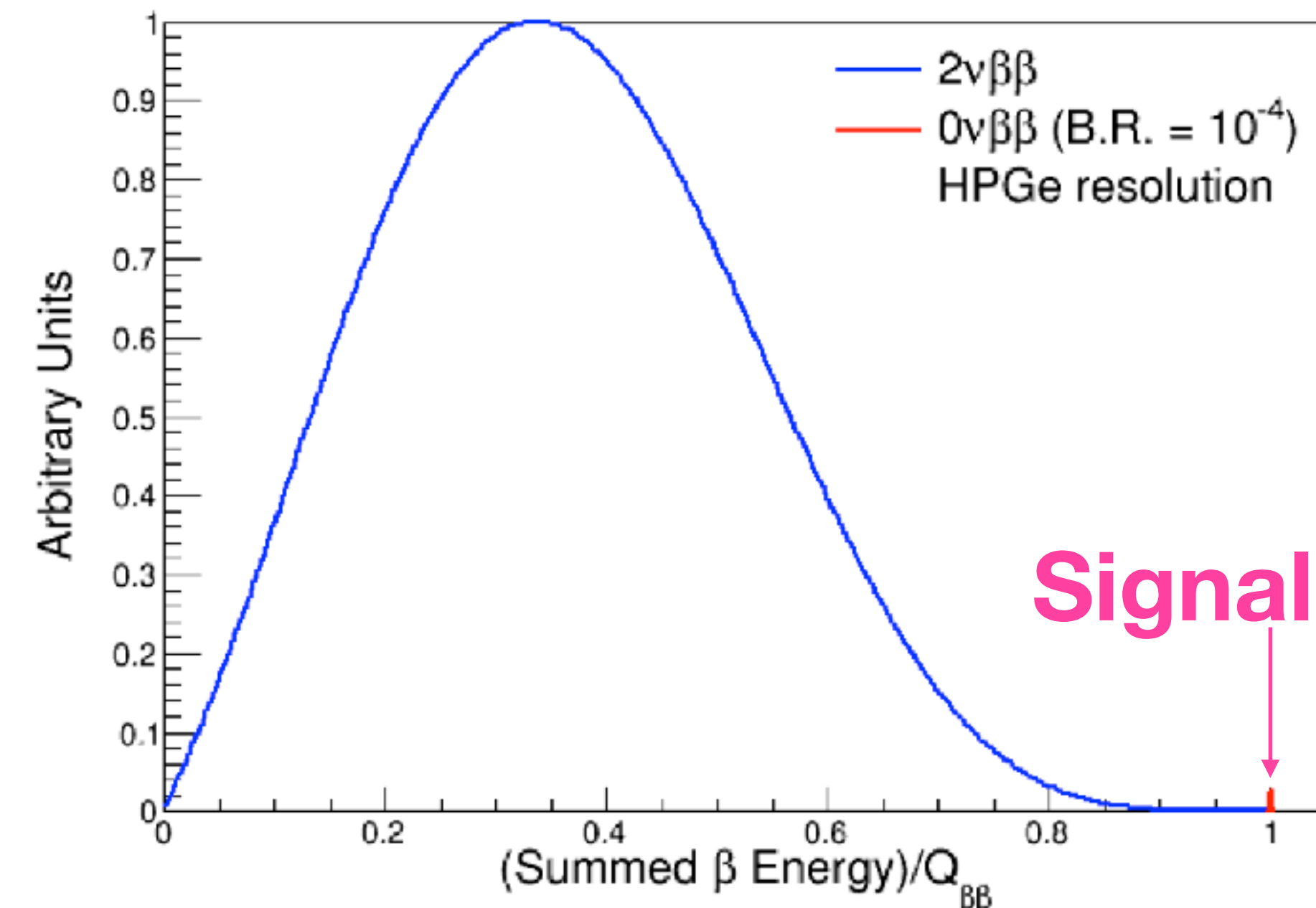
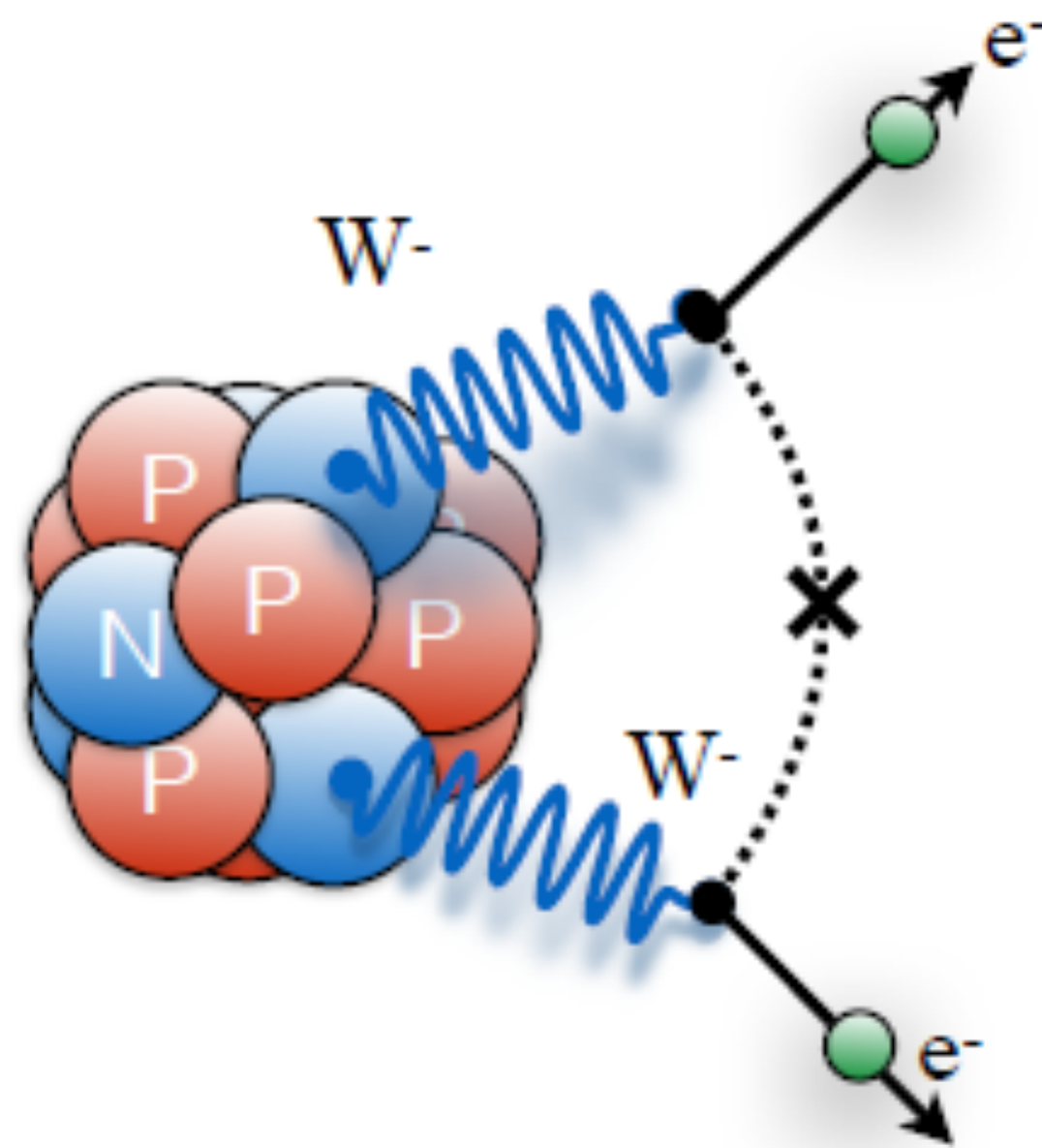
Neutrinoless Double-Beta Decay ($0\nu\beta\beta$)

$\Delta L = 2$ lepton number violation process

Explain the **matter-antimatter asymmetry** in our universe

Changes our fundamental understanding of particle physics

Has not been observed at $T_{\frac{1}{2}} > 10^{26} \text{ yrs}$

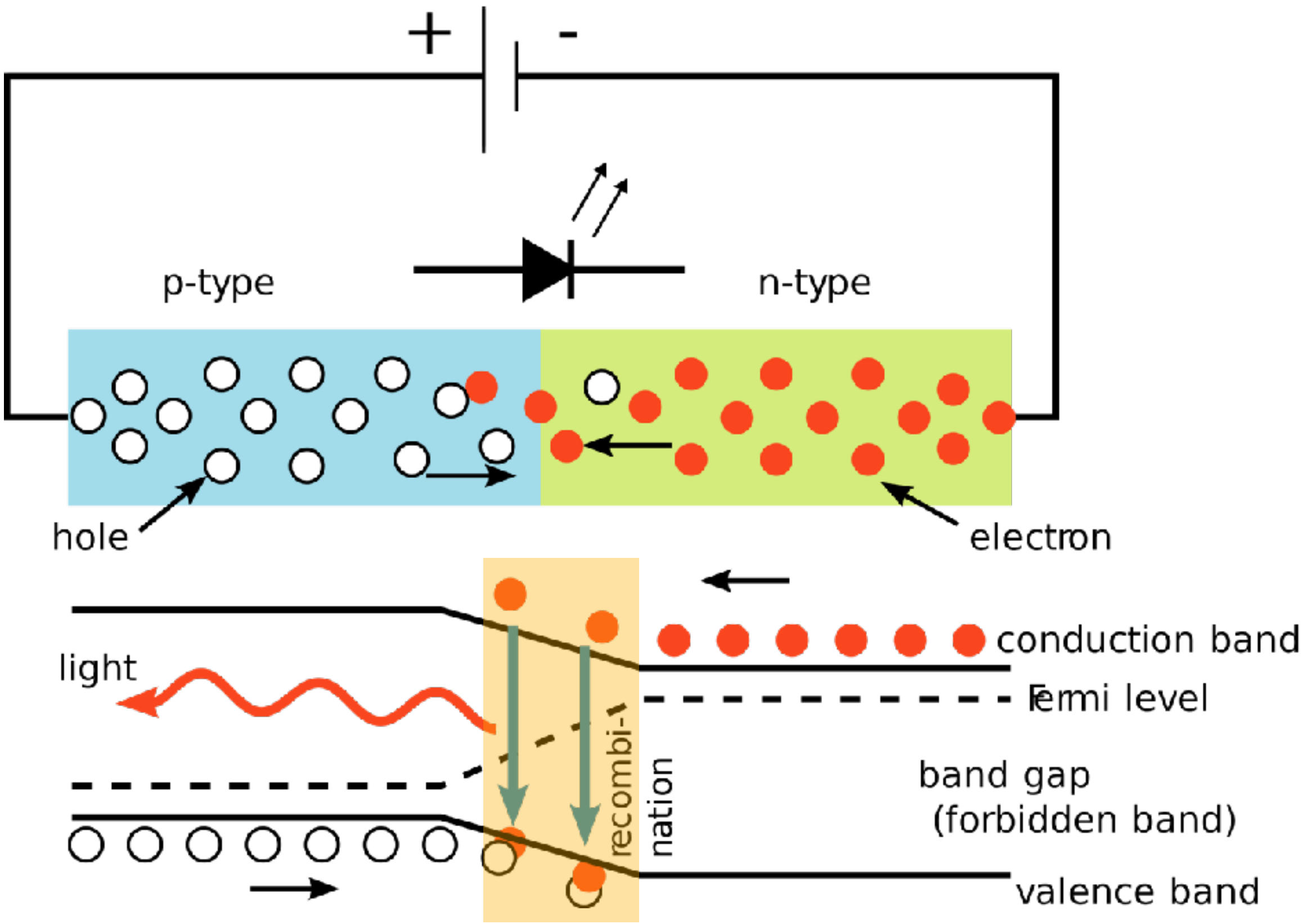


LED vs. HPGe Detector



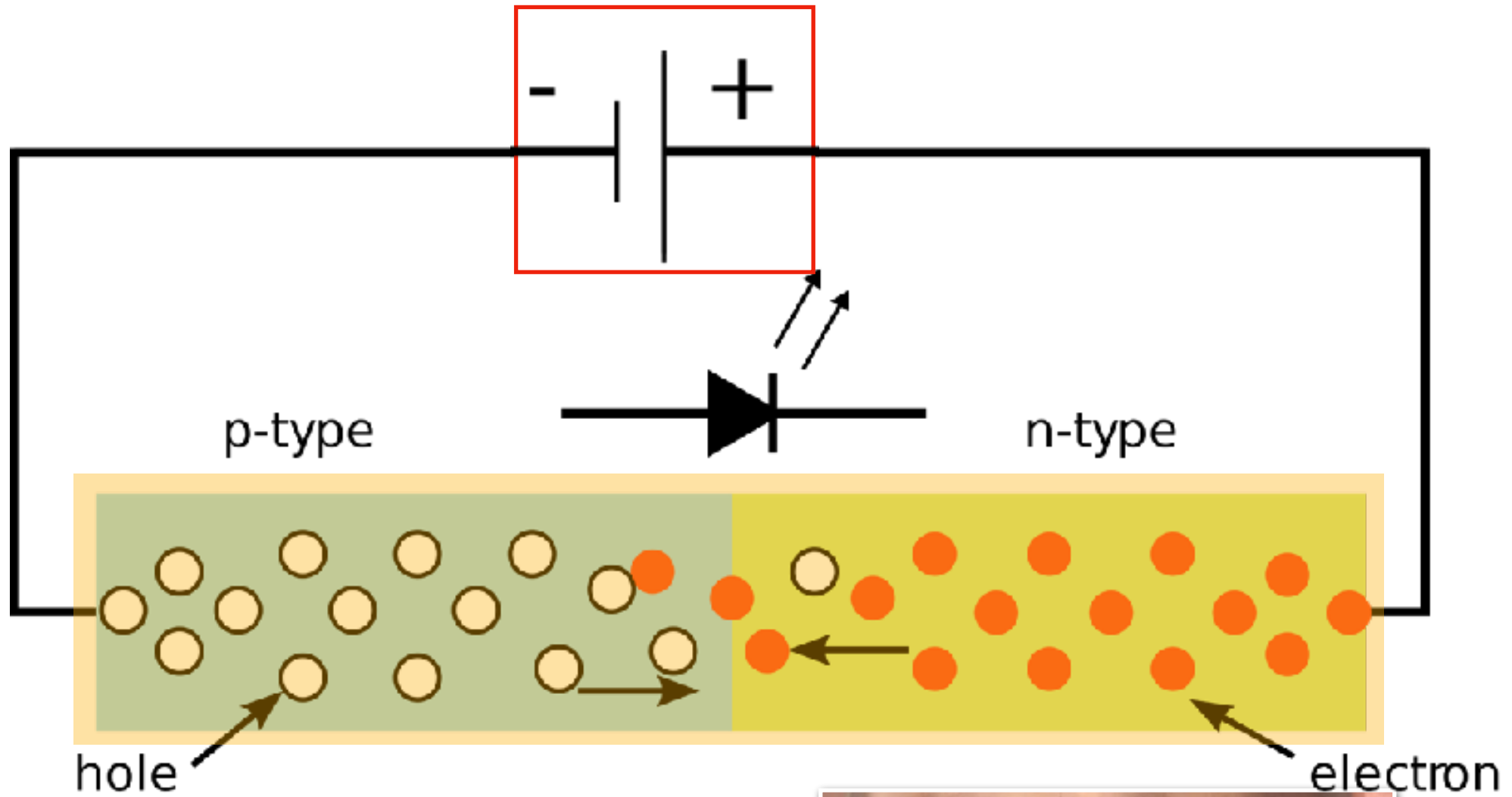
Light Emitting Diode (LED)

Semiconductor device with a **depletion region**

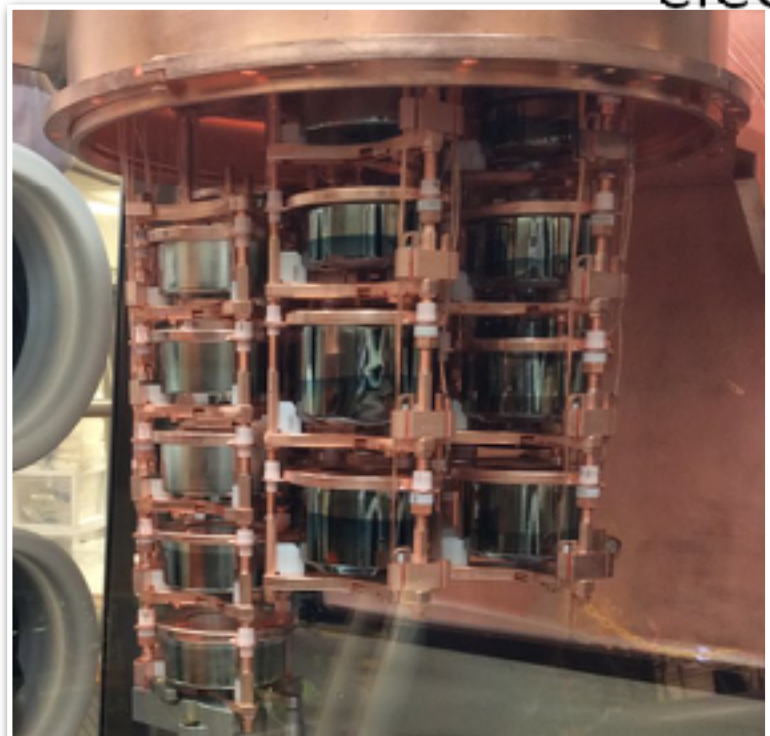


High Purity Ge Detector (HPGe)

Reverse Bias: increase the size of **depletion region**



MAJORANA DEMONSTRATOR Experiment



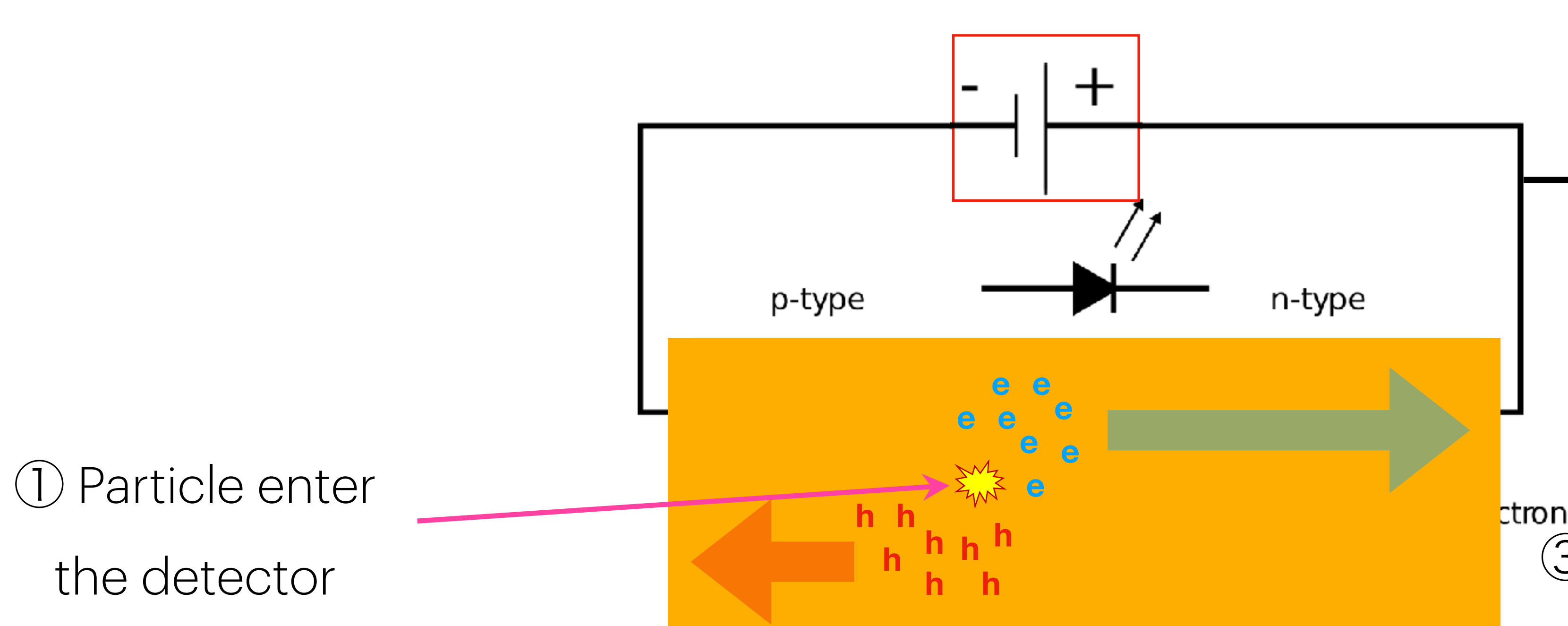
LED vs. HPGe Detector

High Purity Ge Detector (HPGe)

Reverse Bias: increase the size of **depletion region**

④ If we count the number of electrons+holes as n :

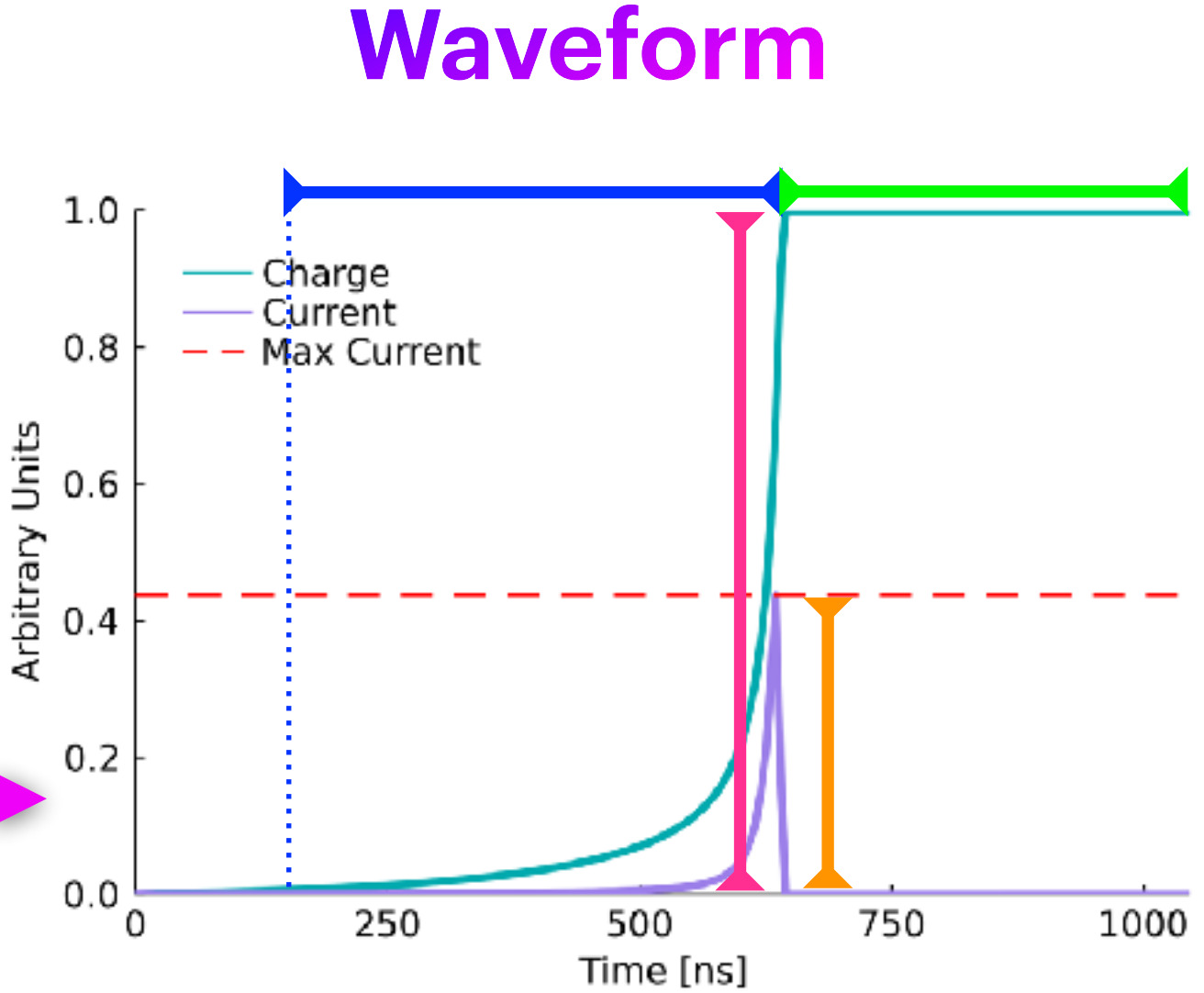
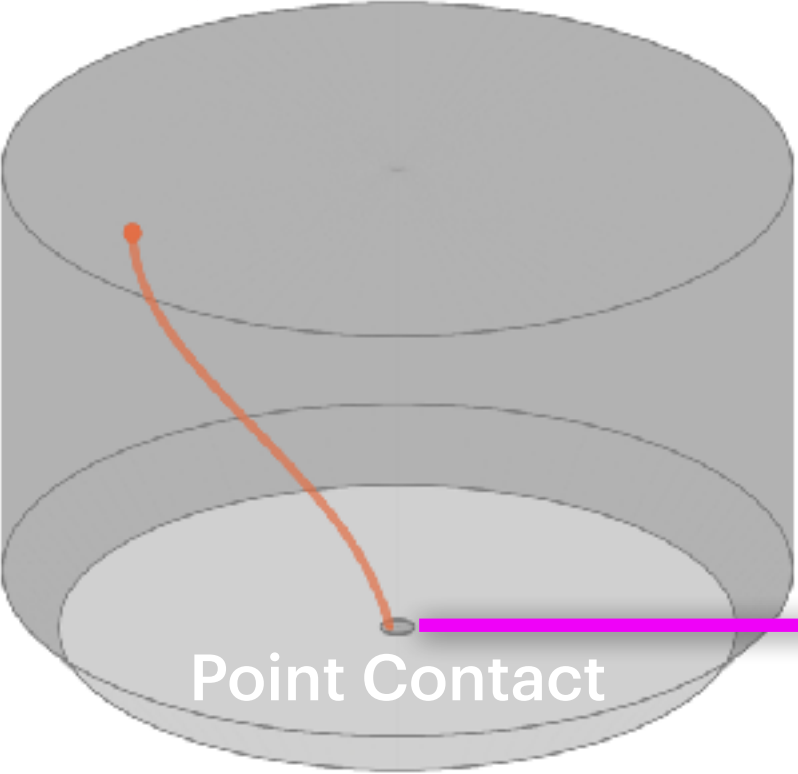
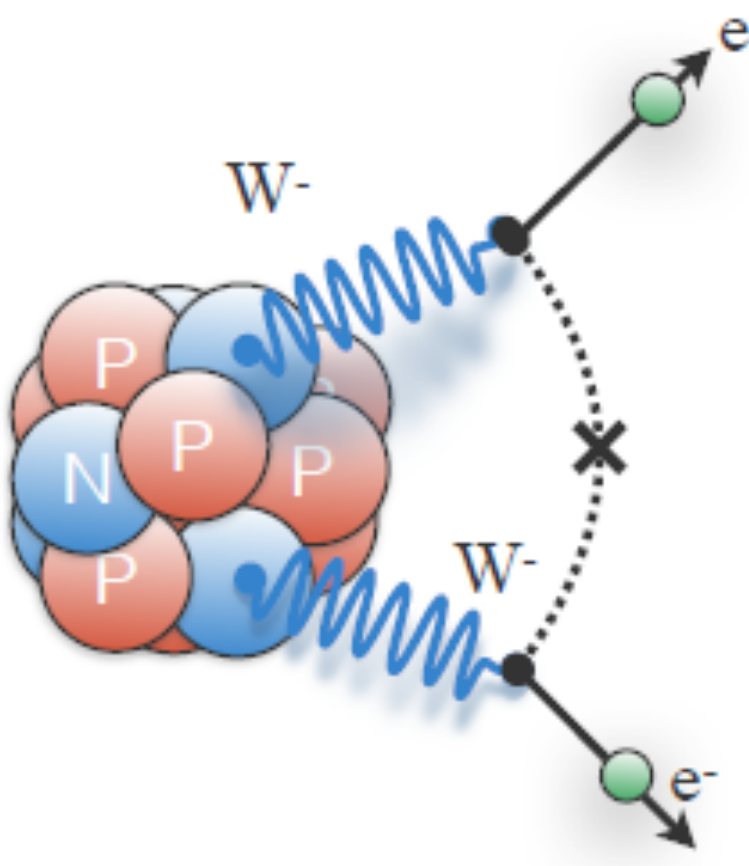
- Particle energy will be proportional to n



③ Since the battery produced a electric field, electron will drift toward the positive end/hole will drift toward the negative end of the battery

HPGe Detector Data (Read 2308.10856)

Single-Site Event ($0\nu\beta\beta$ -Like)

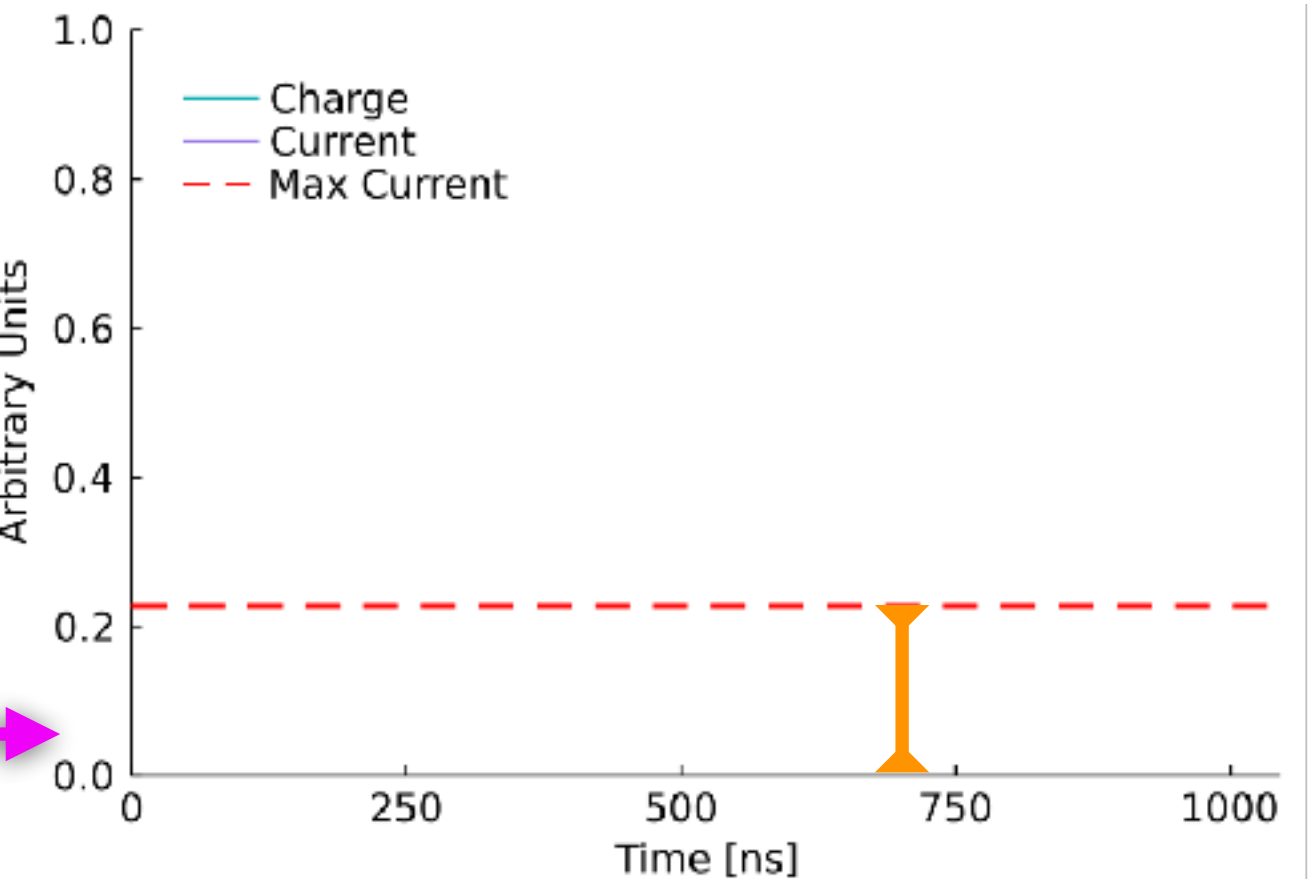
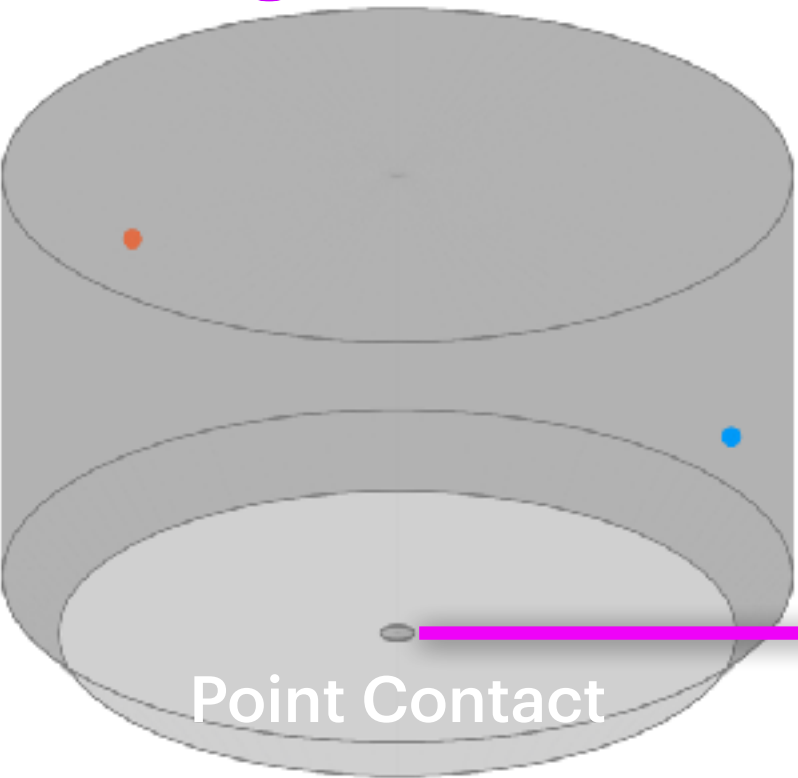
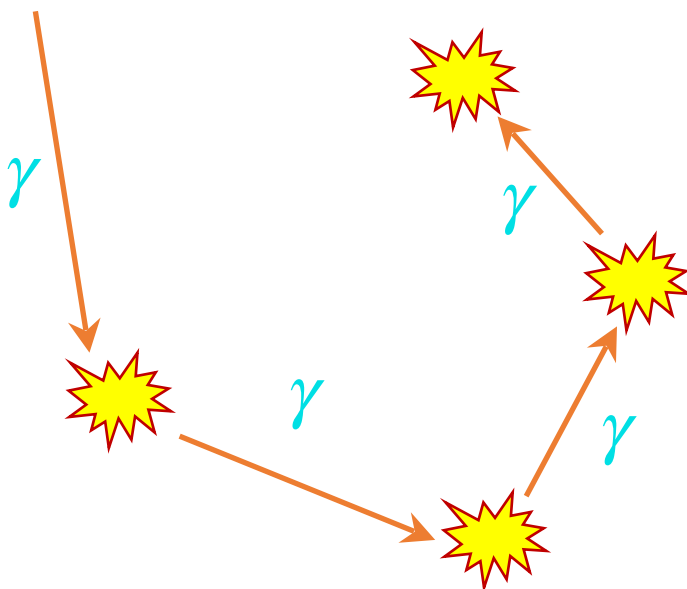


Pulse Shape Parameter

Tail Slope
For **surface event** rejection

Energy

Multi-Site Event (Background-like)



Maximal Current Amplitude
For **multi-site event** rejection

Drift Time

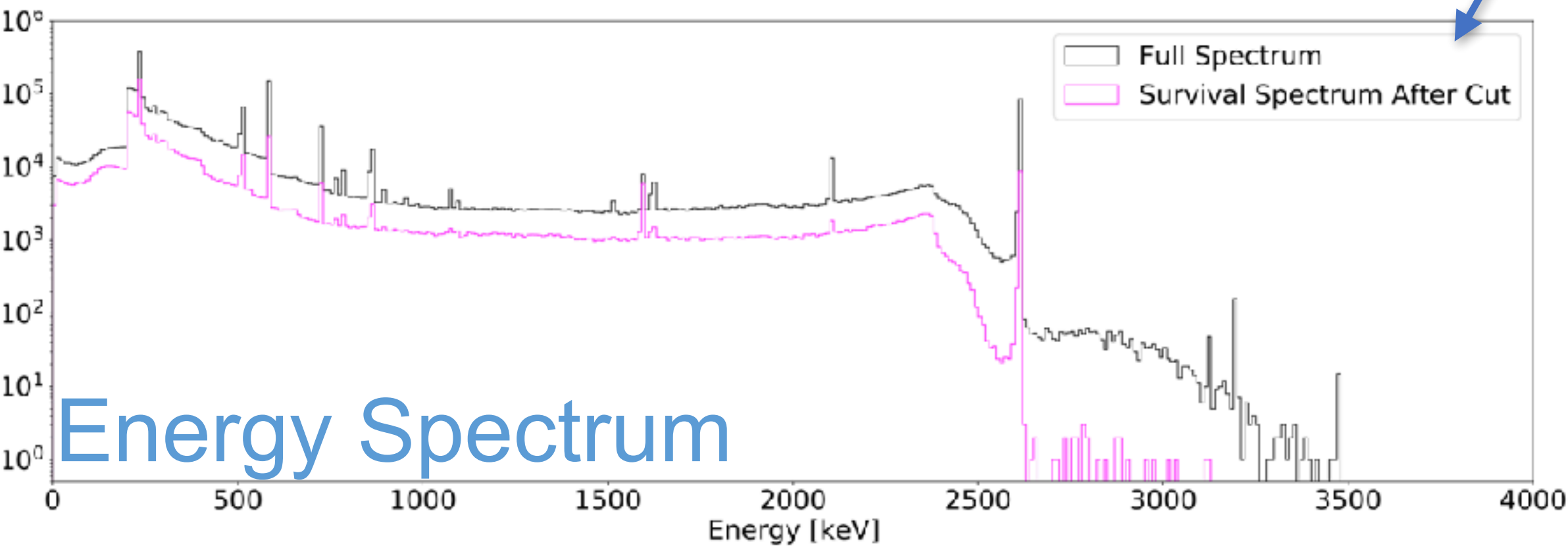
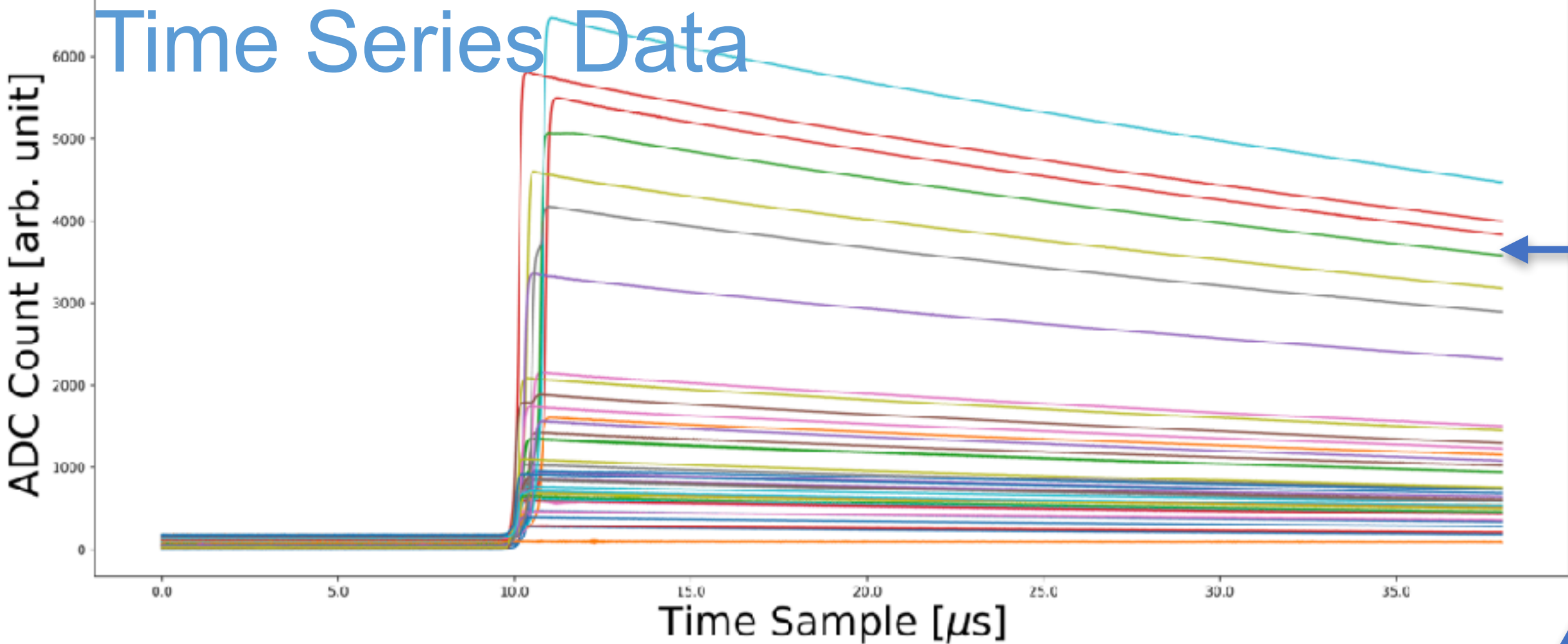
Reflect the location of incident particle

Majorana Dataset (ArXiv: 2308.10856)

Dataset: <https://zenodo.org/records/8257027>

Document: <https://arxiv.org/abs/2308.10856>

Majorana Final Result: Phys. Rev. Lett. 130, 062501



Field	Description	Data Type
raw_waveform	Detector Waveform	array(size=(3800,) dtype=float)
energy_label	Analysis Label	float
psd_label_low_avse	Analysis Label	binary
psd_label_high_avse	Analysis Label	binary
psd_label_dcr	Analysis Label	binary
psd_label_lq	Analysis Label	binary
tp0	Analysis Parameter	integer
detector	Metadata	integer
run_number	Metadata	integer
id	Metadata	integer

More on MJD: <https://www.energy.gov/science/np/articles/majorana-demonstrator-gives-its-final-answer-about-rare-nuclear-decay>

Majorana Dataset (ArXiv: 2308.10856)

All files are stored in .hdf5 format

More readings on HDF5: https://web.mit.edu/fwtools_v3.1.0/www/H5.intro.html

```
MJD_Test_2.hdf5
├── detector (65000,)
├── energy_label (65000,)
├── id (65000,)
├── psd_label_dcr (65000,)
├── psd_label_high_avse (65000,)
├── psd_label_low_avse (65000,)
├── psd_label_lq (65000,)
├── raw_waveform (65000, 3800)
├── run_number (65000,)
└── tp0 (65000,)
```

Read file with h5py.File command



Code

Read “energy” field with indexing

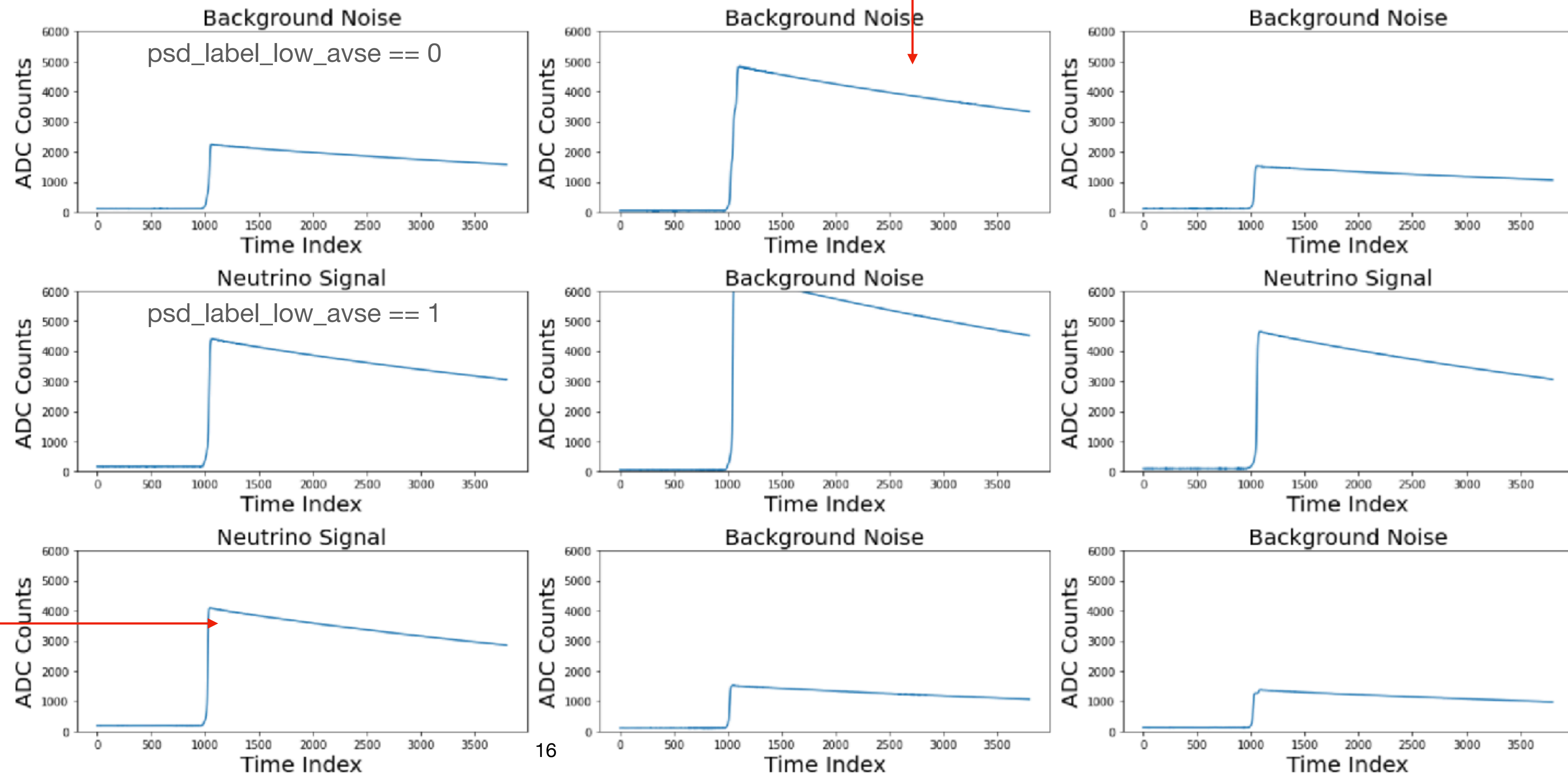
Read “waveform” field with indexing
.....

```
with h5py.File(file_path, 'r') as file:
    energy = np.array(file["energy_label"])
    #only selecting higher energy event to make the task easier
    selection_flgae = energy>self.energy_threshold
    randind = np.arange(selection_flgae.sum())
    np.random.shuffle(randind)
    data = np.array(file["raw_waveform"])[selection_flgae[randind][:,500:1500]]
    label = np.array(file["psd_label_low_avse"])[selection_flgae[randind]]
    energy = np.array(file["energy_label"])[selection_flgae[randind]]
```

Majorana Dataset (ArXiv: 2308.10856)

Take a look at raw waveforms:

Alarm: tail region is too long, but it does not contain physics



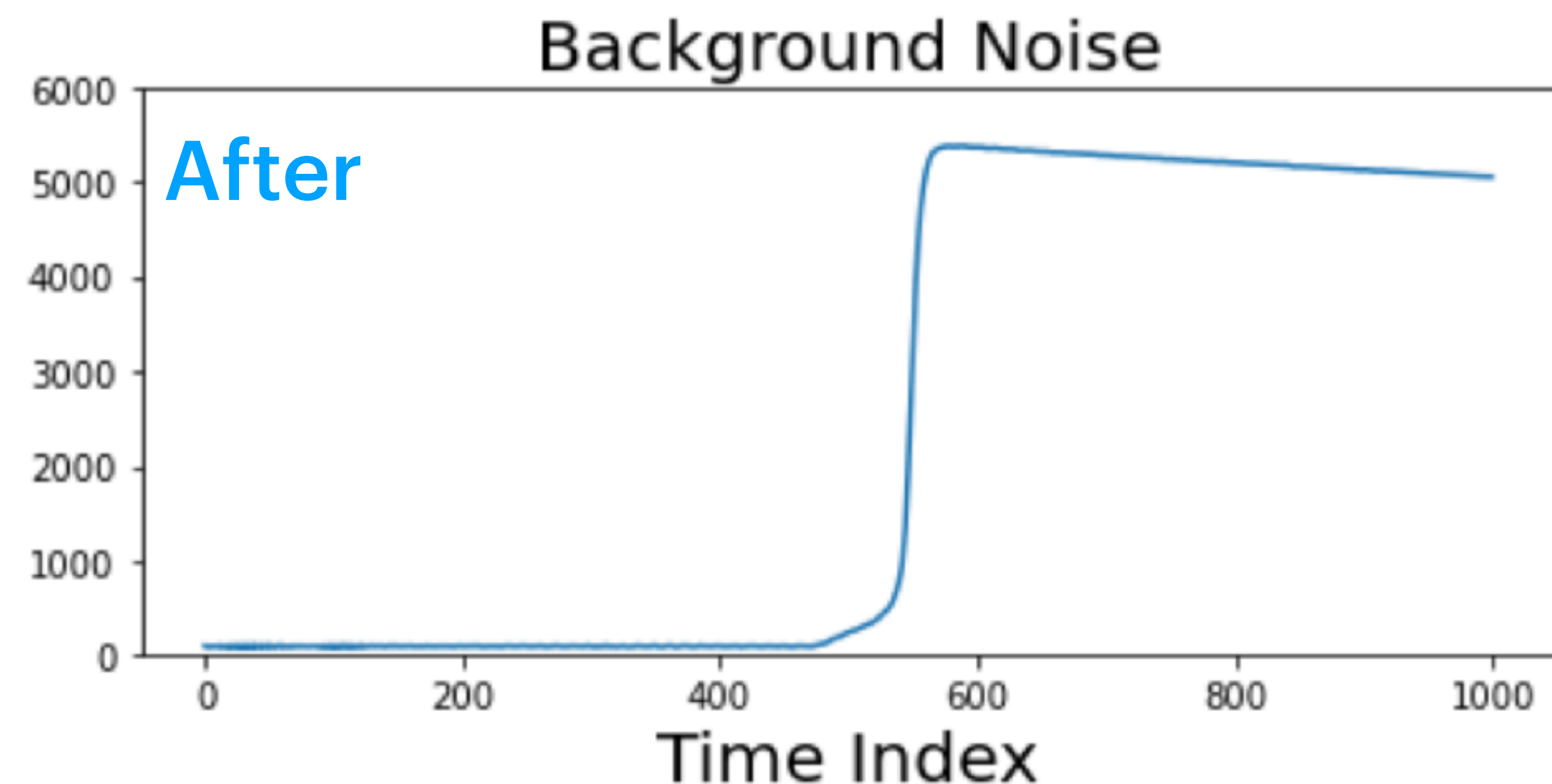
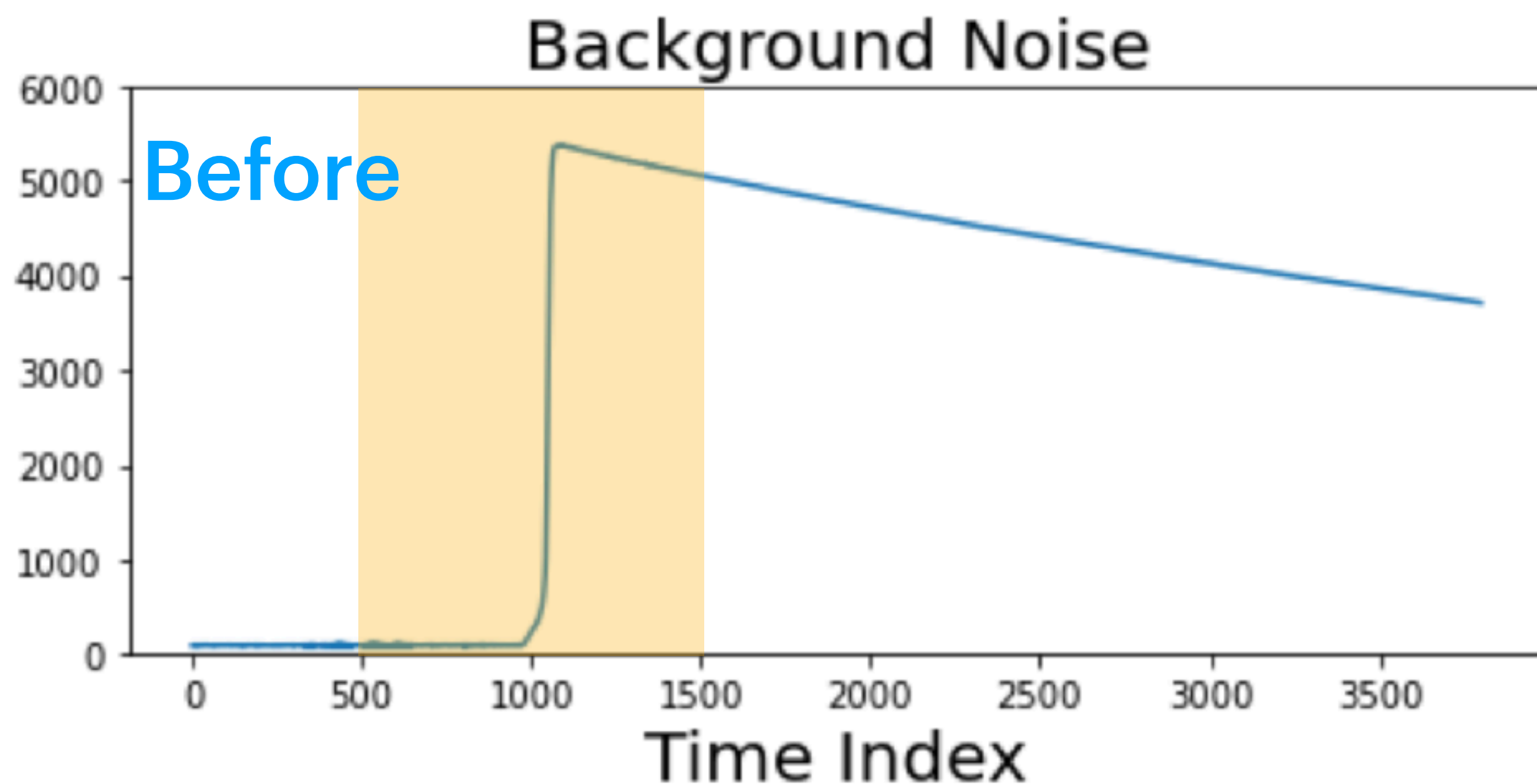
Alarm: Waveforms have different height due to energy difference.

Majorana Dataset (ArXiv: 2308.10856)

Concept

Data Preprocessing: Transform our data in certain way so that unwanted features does not affect model training

Windowing: zoom in to the rising edge of the waveform that contains most of the physics



Majorana Dataset (ArXiv: 2308.10856)

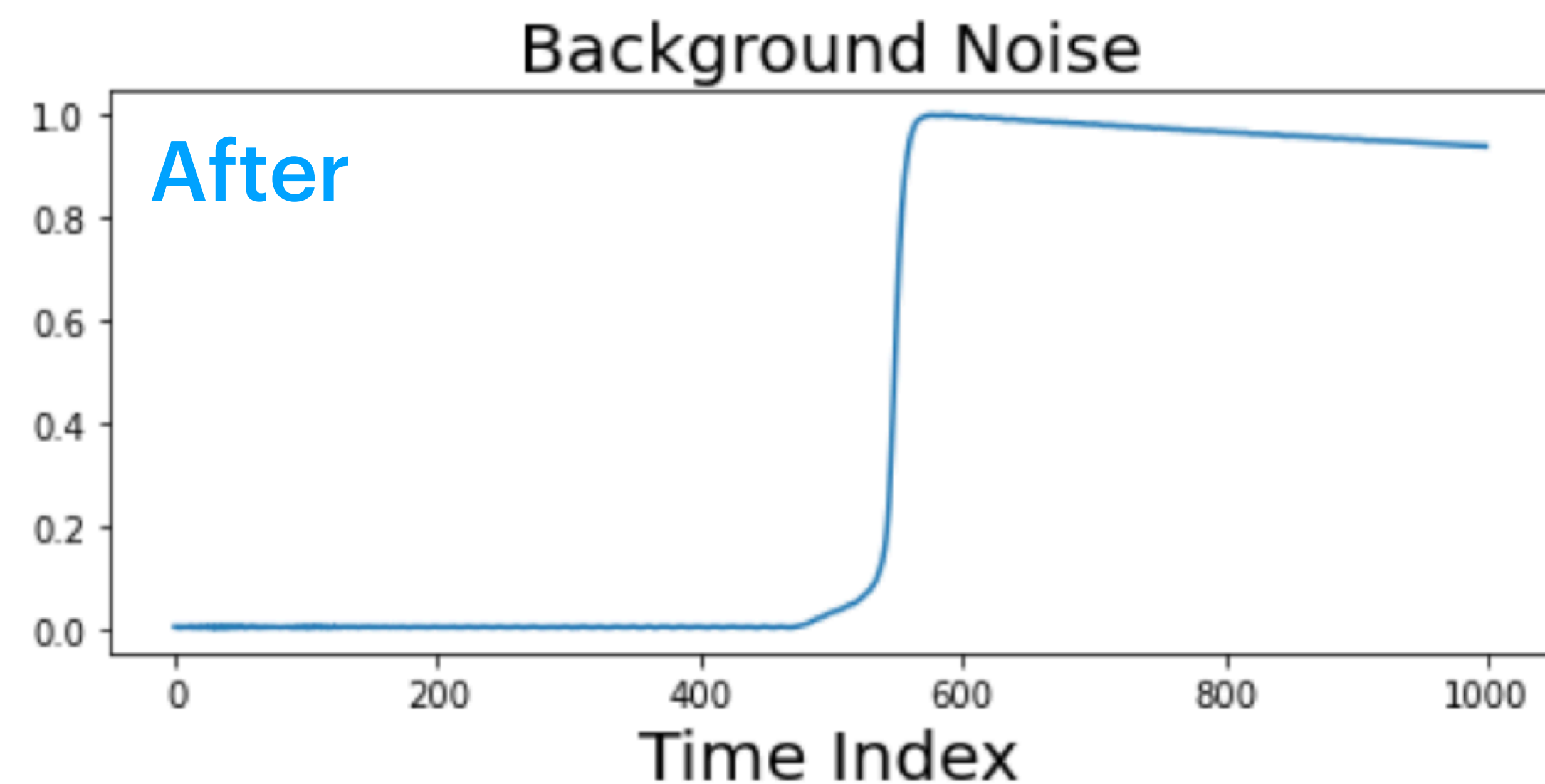
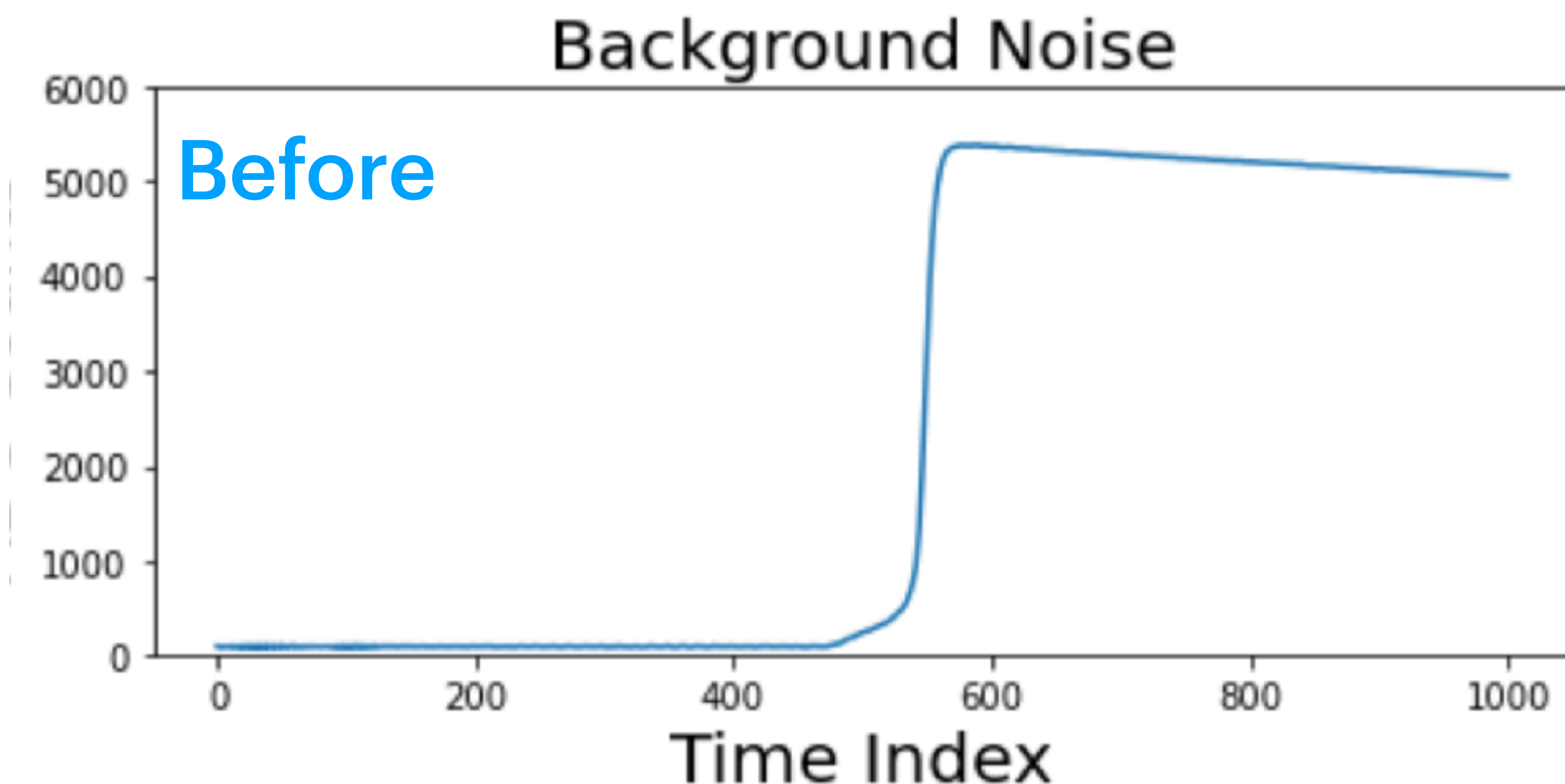
🧠 Concept

Data Preprocessing: Transform our data in certain way so that unwanted features does not affect model training

Normalizing: make sure all waveforms are of the same height (remove energy dependency)

💻 Code

```
wf_norm = (wf - wf.min()) / (wf.max() - wf.min()) # wf is a 1-d numpy array
```



Majorana Dataset (ArXiv: 2308.10856)

Concept

Data Preprocessing: Transform our data in certain way so that unwanted features does not affect model training

Concept

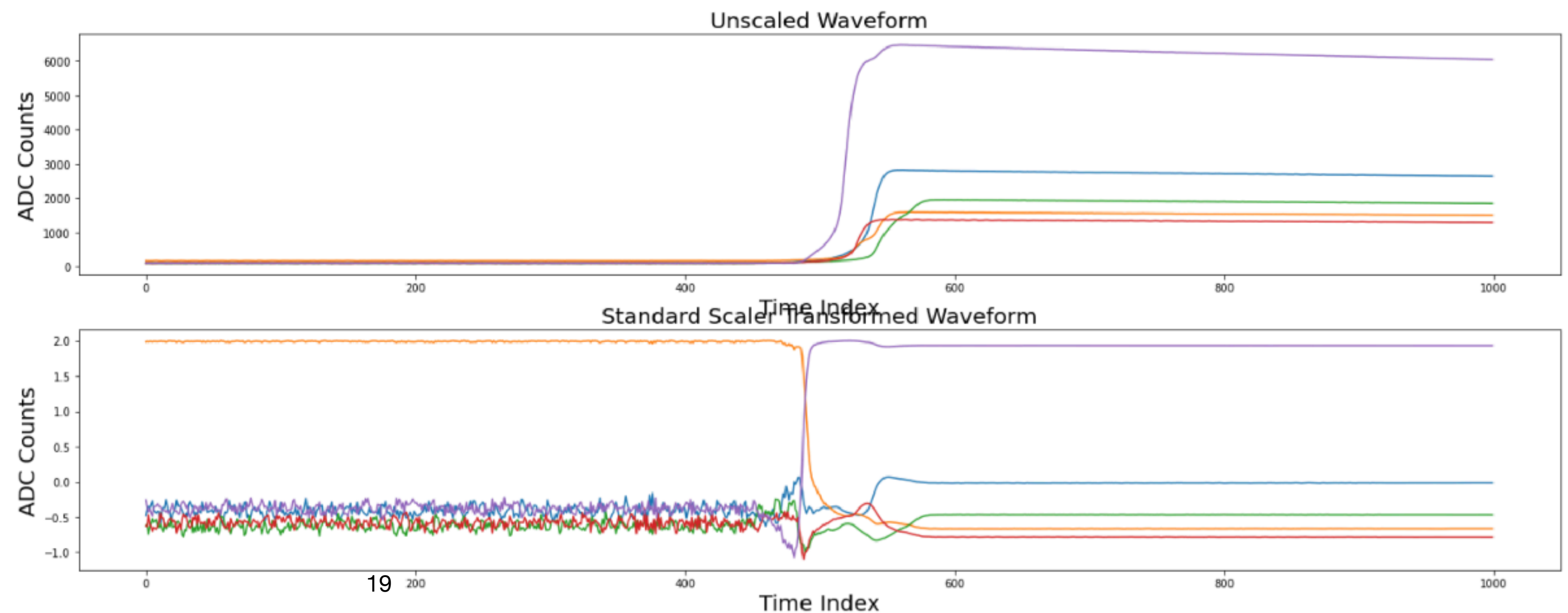
Standard Scaler: transform each dimension of data so that its mean = 0 and standard dev. = 1

Code

sklearn package contains Standard Scaler

```
from sklearn.preprocessing import  
StandardScaler  
scaler = StandardScaler()
```

Has to be applied over multiple wfs
`wf_norm = scaler.fit_transform(wf_arr)`



Majorana Dataset (ArXiv: 2308.10856)

Code

Putting everything into a PyTorch Dataset class

```
class NeutrinoDataset(torch.utils.data.Dataset):  
    def __init__(self, plot=True):  
        """  
        REQUIRED function  
        This function initializes the dataset object  
        """  
  
    def __len__(self):  
        """  
        REQUIRED function  
        This function returns the size of overall dataset  
        """  
        return 65000  
  
    def __getitem__(self, idx):  
        """  
        REQUIRED function  
        This function extract a single waveform from the dataset at the given index idx  
        """  
        return self.waveform[idx], self.psd_label_low_avse[idx], self.energy_label[idx]  
  
    def plot_data(self):  
        """  
        OPTIONAL functions  
        """
```

- **Only called once** when initializing the object
- Read information from the .hdf5 file into array(s)
 - apply dataset-level pre-processing like StandardScaler

- Return length of the dataset
- Called whenever we run `len(dataset_object)`

- Return **one data point** at input index **idx**
- Datapoint-level preprocessing can go here (i.e. Normalizing)
- Will be **called very frequently** during network training and validation
 - Don't put slow operations here!

- Define as many helper functions as you like

Majorana Dataset (ArXiv: 2308.10856)

Code

Used the pre-defined class on last slides to construct training and testing data loader

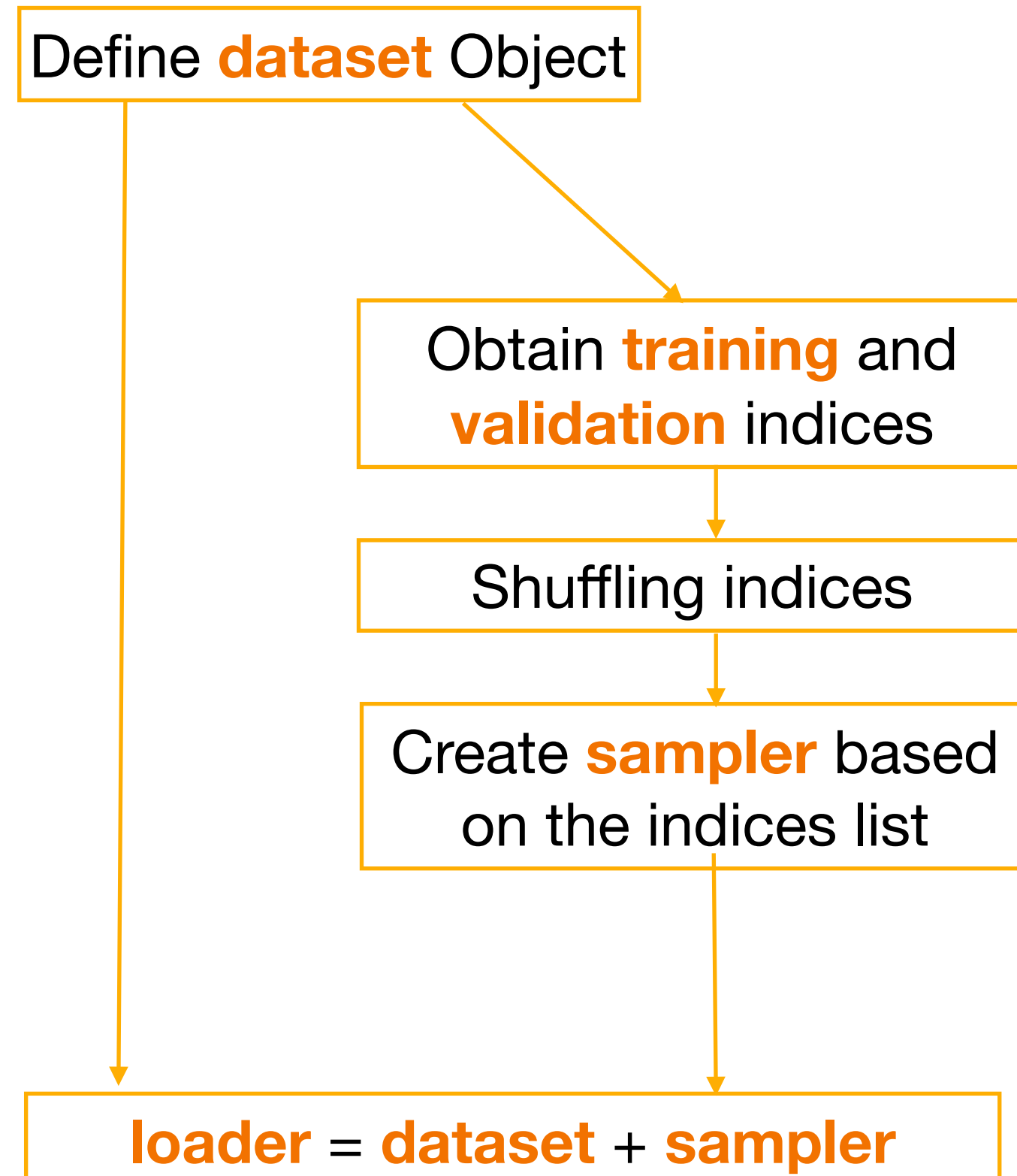
```
dataset = NeutrinoDataset(plot=False)
'''
Get the indices of train dataset and test dataset correspondingly,
indices [0:train_test_split] is the training dataset,
indices [train_test_split, len(dataset)] is the test dataset.
'''

train_test_split = int(0.7*len(dataset))
train_indices, val_indices = list(range(train_test_split)), list(range(train_test_split, len(dataset)))

#Shuffle the two indices list
np.random.shuffle(train_indices)
np.random.shuffle(val_indices)

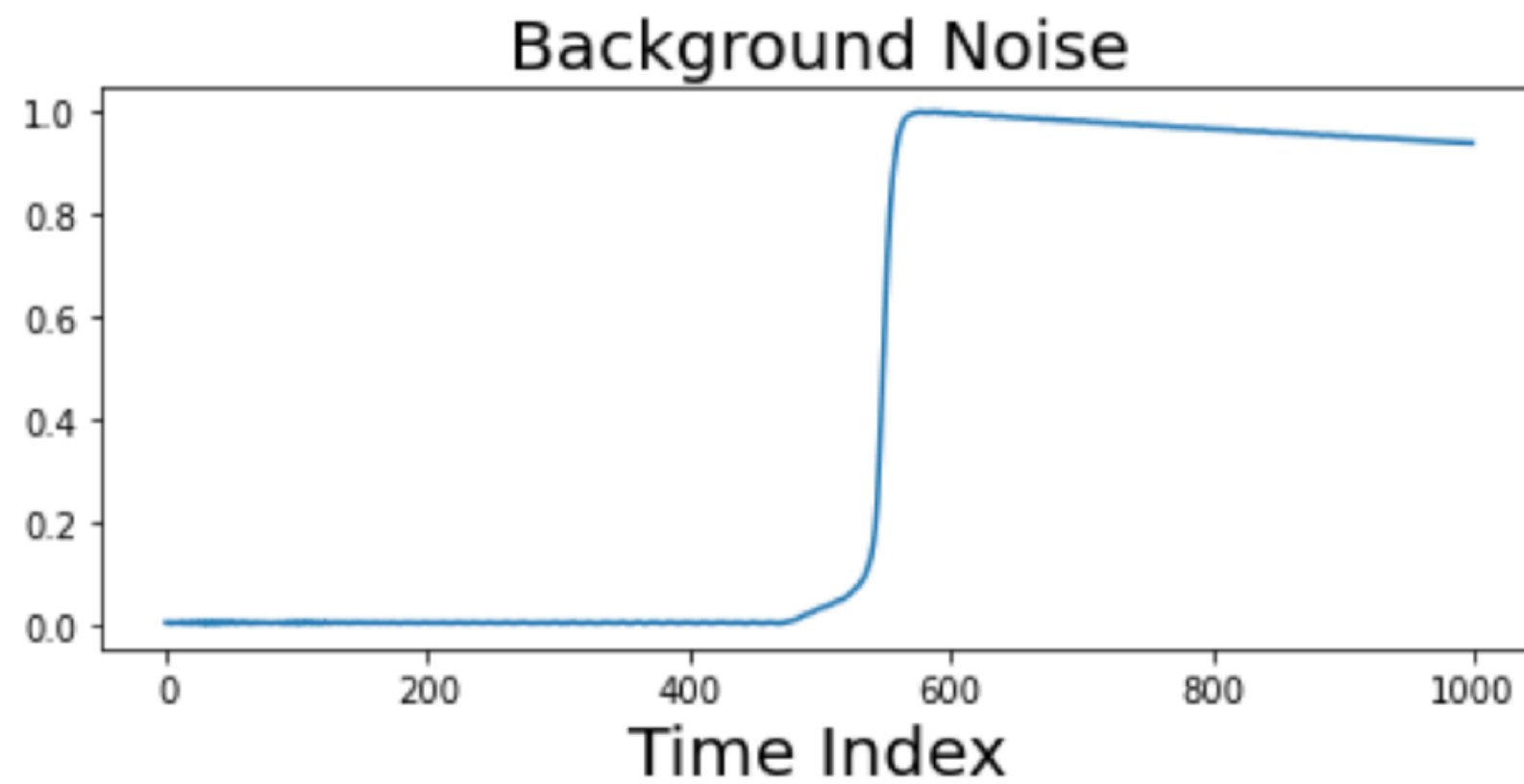
# Define two subset random sampler to sample events according to the training indices
train_sampler = SubsetRandomSampler(train_indices)
valid_sampler = SubsetRandomSampler(val_indices)
'''
Finally, define the loader by passing in the dataset, batch size and corresponding sampler
Note that the number of data in each sub-dataset might not be divisible by the batch size,
so drop_last=True drops the last batch with all the residual events.
'''

train_loader = data_utils.DataLoader(dataset, batch_size=BATCH_SIZE, sampler=train_sampler, drop_last=True)
test_loader = data_utils.DataLoader(dataset, batch_size=BATCH_SIZE, sampler=valid_sampler, drop_last=True)
```



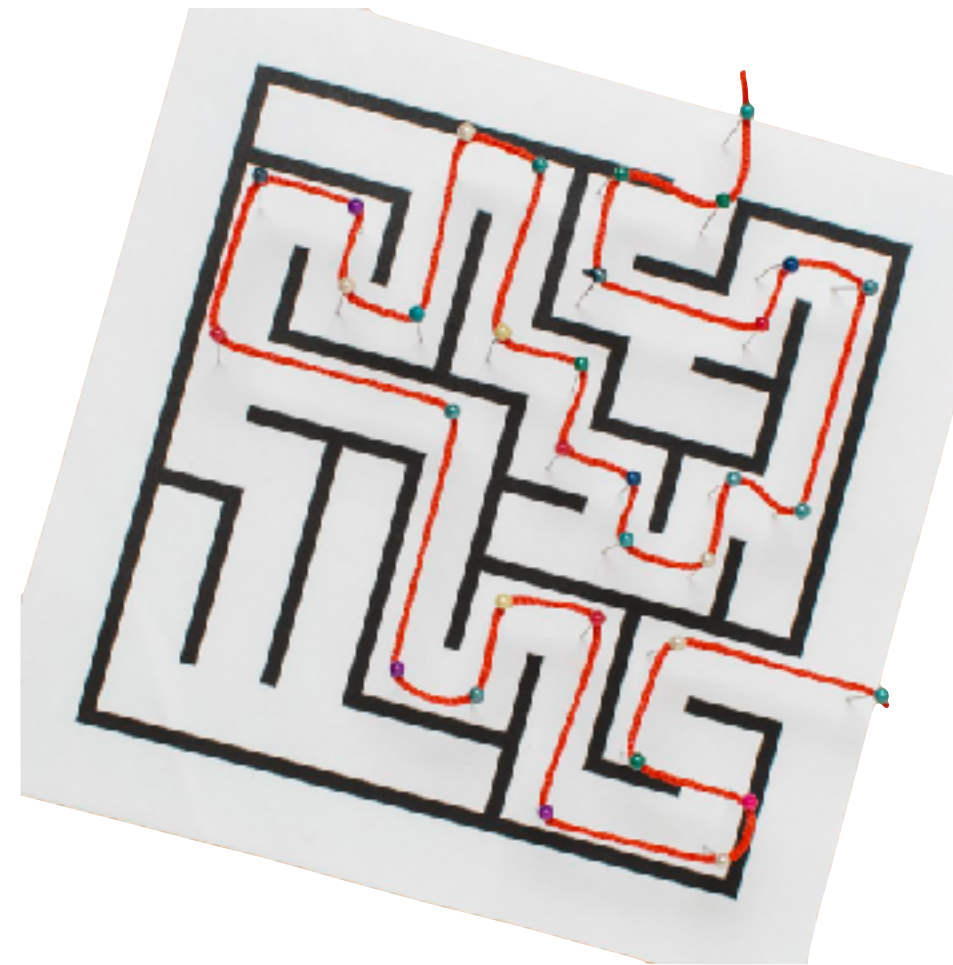
Majorana Dataset (ArXiv: 2308.10856)

Time Series Data (BATCH_SIZE, 1000)



Model

Map input to output



→ **Signal or
Background**
(batch_size, 1)

→ **Event Energy**
(batch_size, 1)



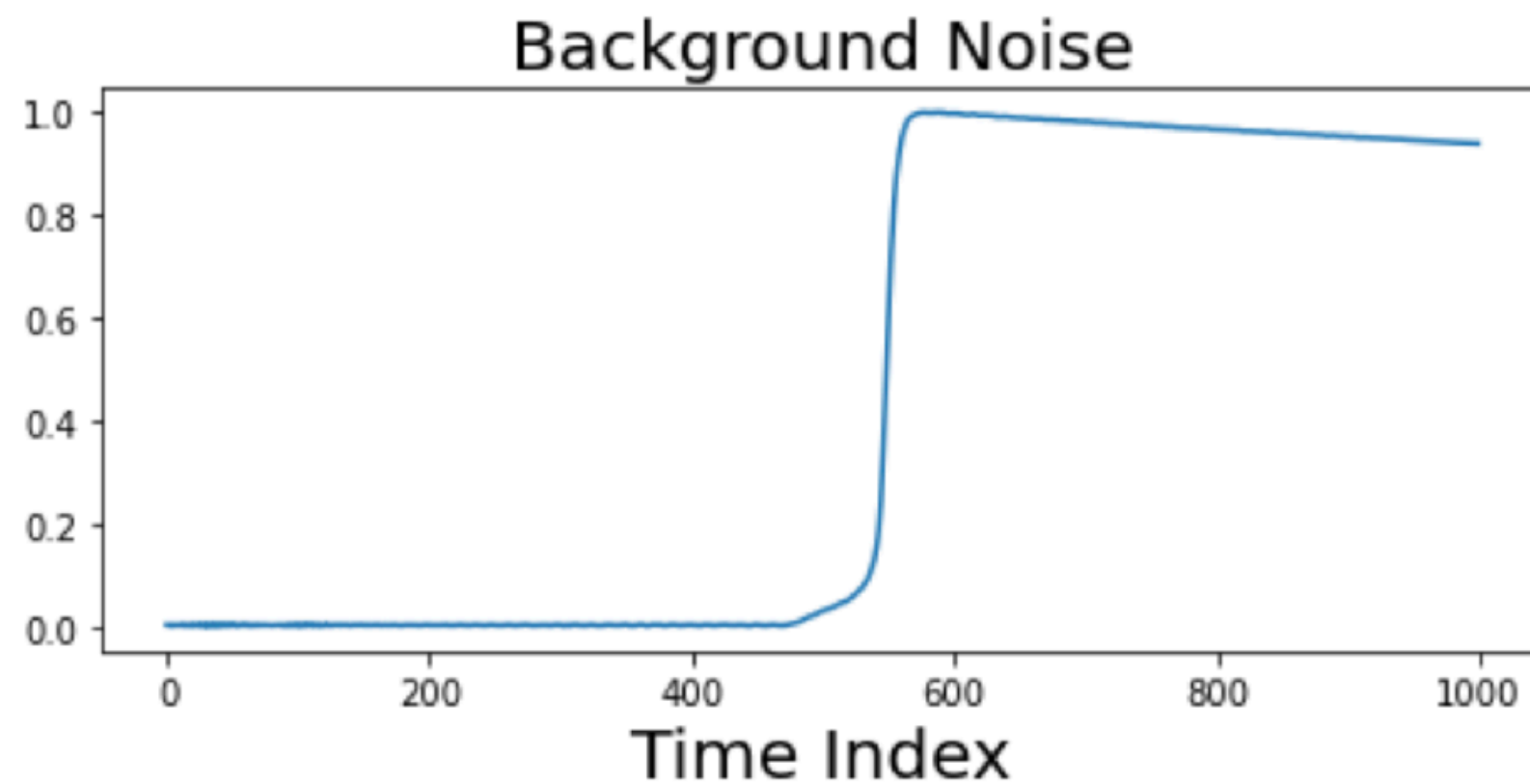
psd_label_low_avse
(batch_size, 1)



energy_label
(batch_size, 1)

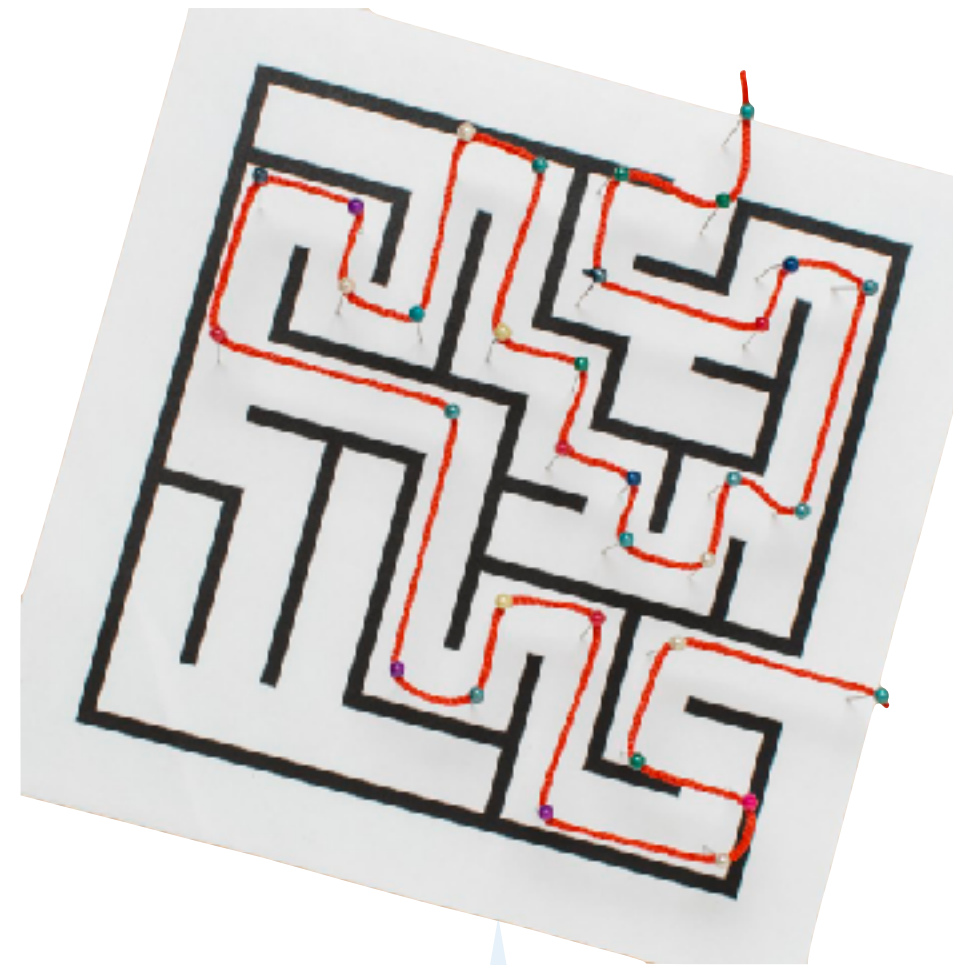
Majorana Dataset (ArXiv: 2308.10856)

Time Series Data (BATCH_SIZE, 1000)



Model

Map input to output



→ **Signal or
Background**
(batch_size, 1)



psd_label_low_avse
(batch_size, 1)

→ **Event Energy**
(batch_size, 1)



energy_label
(batch_size, 1)

Concept

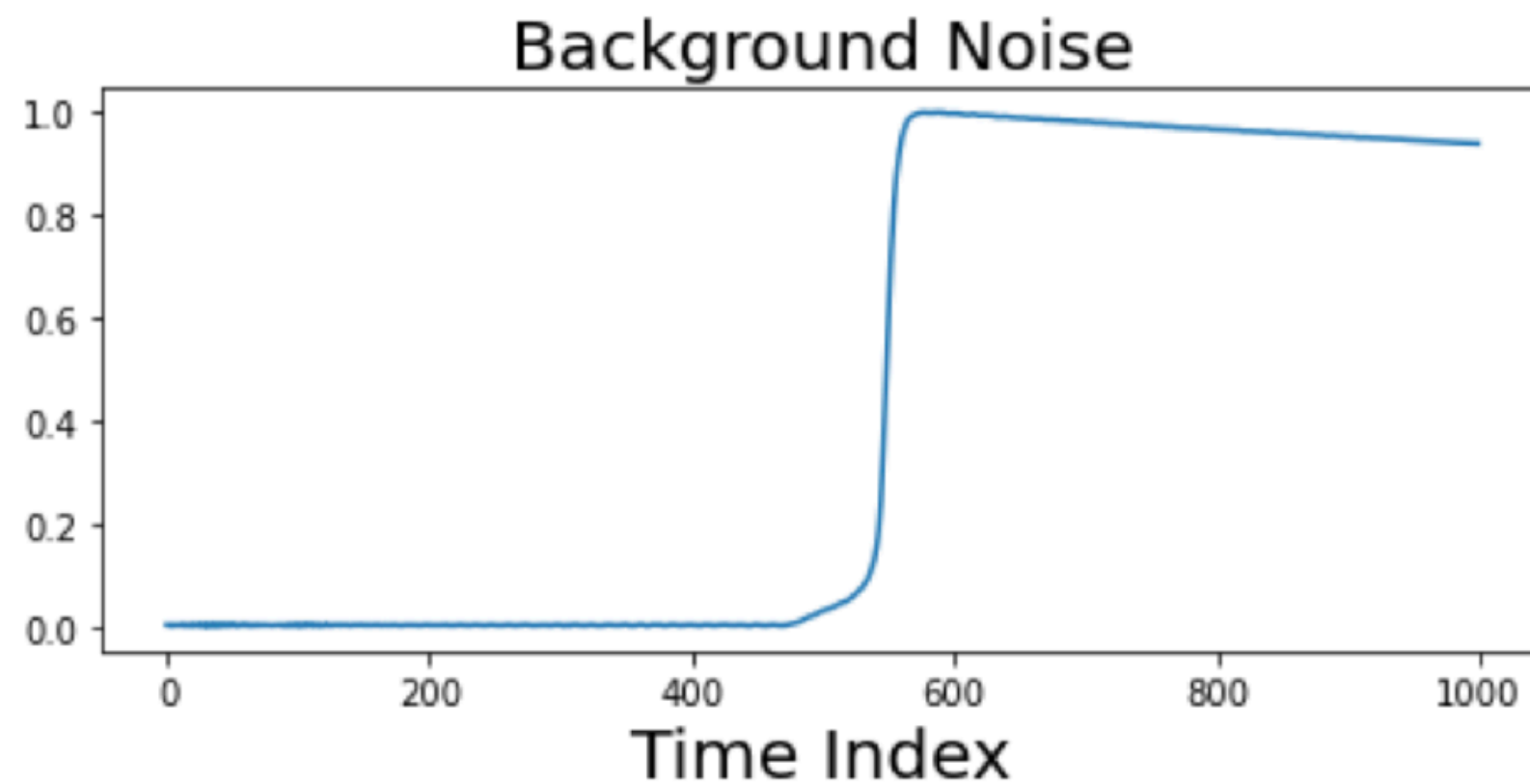
Feature Extractor Network

Take raw data as the input and
output a low-dimensional vector

$NNLayer(1000, 512) \rightarrow \sigma$
 $\rightarrow NNLayer(512, 256) \rightarrow \sigma$
 $\rightarrow NNLayer(256, 32) \rightarrow \sigma$
 $\rightarrow NNLayer(32, 1) \rightarrow \sigma$

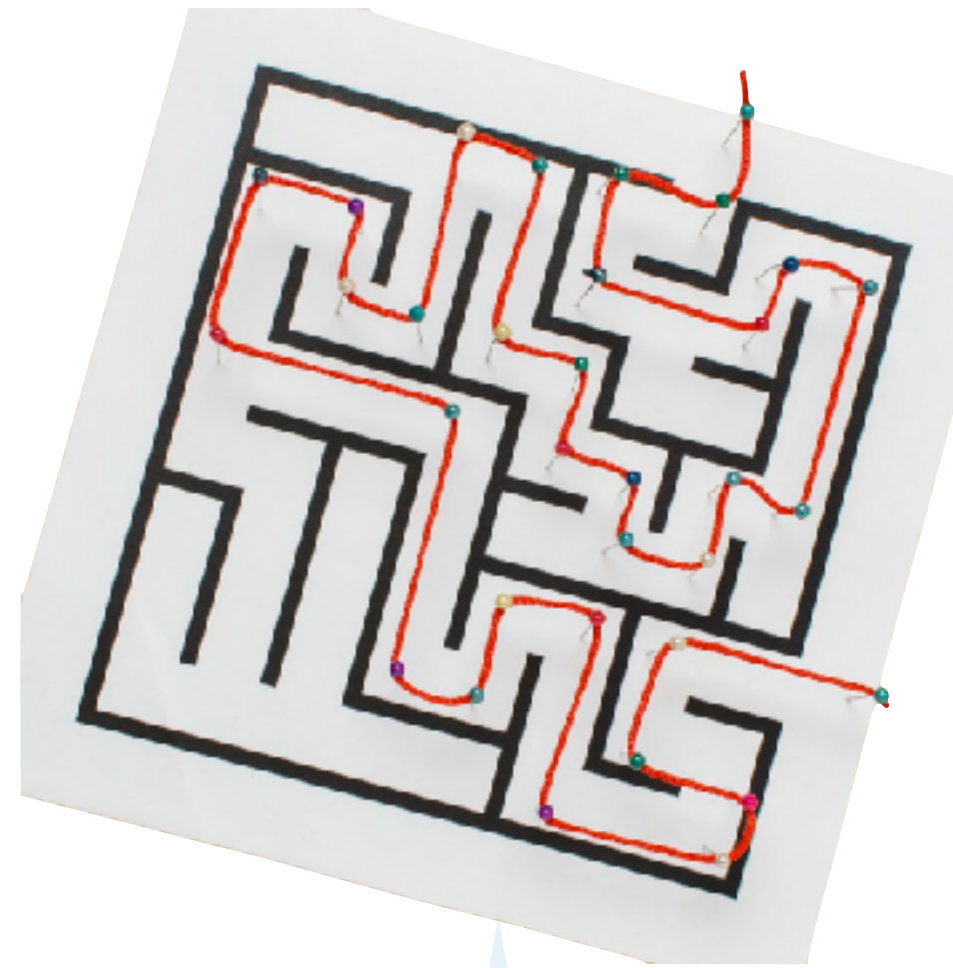
Majorana Dataset (ArXiv: 2308.10856)

Time Series Data (BATCH_SIZE, 1000)



Model

Map input to output



→ **Signal or Background**
(batch_size, 1)



psd_label_low_avse
(batch_size, 1)

→ **Event Energy**
(batch_size, 1)



energy_label
(batch_size, 1)

🧠 Concept

Feature Extractor Network

Take raw data as the input and
output a low-dimensional vector

$NNLayer(1000, 512) \rightarrow \sigma$
 $\rightarrow NNLayer(512, 256) \rightarrow \sigma$
 $\rightarrow NNLayer(256, 32) \rightarrow \sigma$
 $\rightarrow NNLayer(32, 1) \rightarrow \sigma$

🧠 Concept

Task Module: Task Layer + Loss Function

- **Task Layer:** Take low-dimensional vector as input and produce a value/vector as **output**
- **Loss Function:** produce a quantitative measure evaluating how well your algorithm models your dataset
- Minimizing the loss function means that the **model output** reproduces the **label** better

Majorana Dataset (ArXiv: 2308.10856)

NNLayer

σ

L

Concept

Label Classification 1

Task Layer: $\text{NNLayer}(32, 1) \rightarrow$ One float point No. between $[-inf, inf]$

- After training, it can be considered as a “**classification score**”:
 - Higher score means the answer is more likely “yes”
 - Lower score means the answer is more likely “no”
 - A **threshold** is need to distinguish “yes” from “no”

σ : $\text{torch.sigmoid}(x)$

- Sigmoid function $\sigma(x) = 1/(1 + e^x)$ maps input from $[-inf, inf]$ to $[0,1]$

Loss Function: $\text{torch.nn.BCELoss}()$

- Input: has to be a number between $[0,1]$
- Target: has to be either 0 or 1, cannot be other number

Concept

Label Classification 2

Task Layer: $\text{NNLayer}(32, 1) \rightarrow$ One float point No. between $[-inf, inf]$

After training, it can be considered as a “**classification score**”:

- Higher score means the answer is more likely “yes”
- Lower score means the answer is more likely “no”
- A **threshold** is need to distinguish “yes” from “no”

σ : None

Loss Function: $\text{torch.nn.BCEWithLogitsLoss}()$

- Input: any range between $[-inf, inf]$
- Target: has to be either 0 or 1, cannot be other number

Majorana Dataset (ArXiv: 2308.10856)

NNLayer

σ

L

Concept Energy Regression

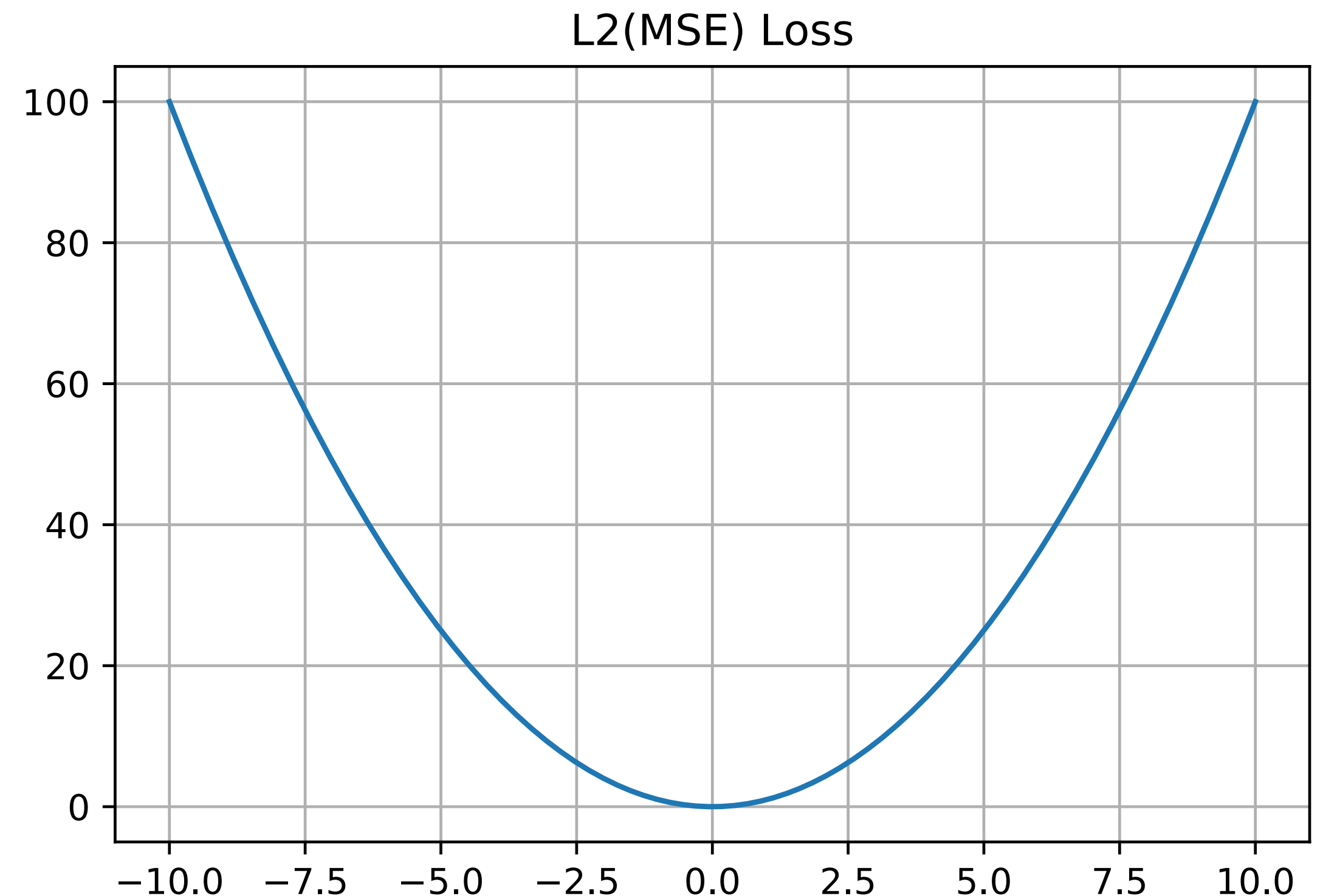
Task Layer: `NNLayer(32, 1)` $\rightarrow$ One float point No. between $[-inf, inf]$

σ :

- None if you want to fit a physics quantity like energy
- `torch.sigmoid(x)` if you want to model a percentage like efficiency

Loss Function: `torch.nn.MSELoss()`

- $L = (\text{TL_Output} - \text{Energy})^2$ for energy reconstruction
- Most commonly used, everywhere differentiable
- Gradient explosion with extreme values



TIDMAD Dataset

NeurIPS 2025 D&B Track Spotlight Paper

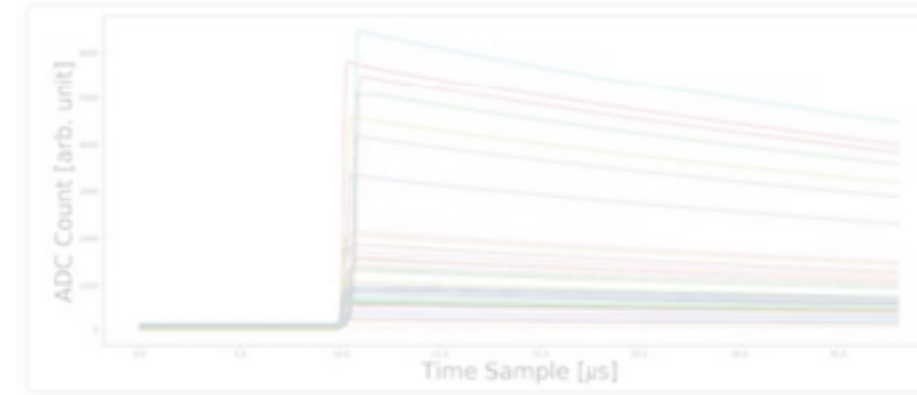


Fig 1. Sample Waveforms

MAJORANA DEMONSTRATOR DATA RELEASE

BASIC DATASET HIGH PURITY GERMANIUM DETECTOR SIGNALS

This dataset contains over **3 million data points** derived from the **Majorana Demonstrator experiment**, which utilizes High Purity Germanium Detectors to search for Neutrinoless Double-Beta Decay. Each data point represents a real time-series waveform generated by the detector.

Input Data: 1D Numpy vector with 4000 samples (Time Series).

🔑 **Labels:** 4 Binary Classification labels + 1 Energy Regression label.

🎯 **Goal:** Train ML models to use the waveform as input to predict the analysis labels.

Format: HDF5 (See Tutorial 1 for reading instructions).

📄 Download Dataset (Zenodo)

📖 Read Description (arXiv)

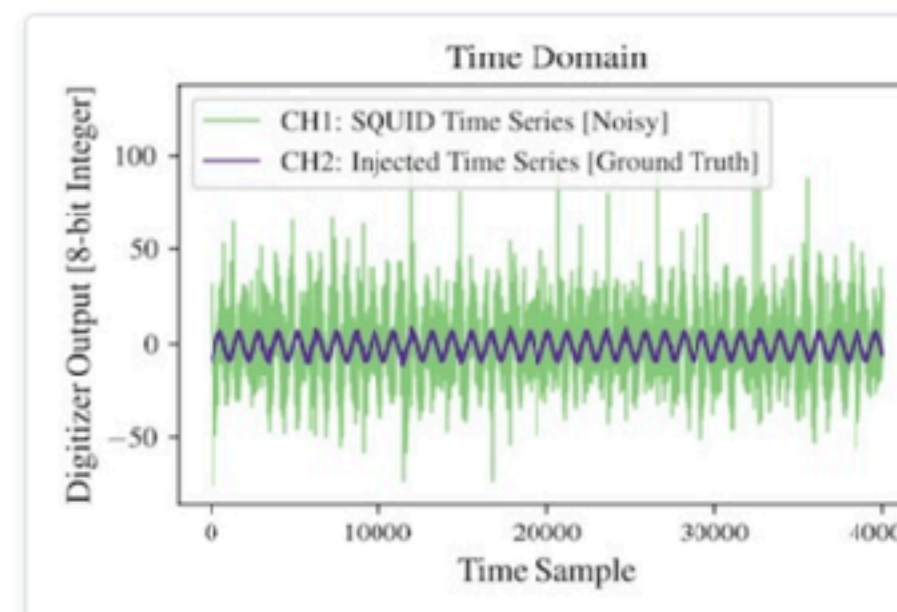


Fig 2. Noisy Input vs Ground Truth

TIDMAD: DARK MATTER DISCOVERY WITH AI DENOISING

ADVANCED DATASET QUANTUM-ENABLED AXION DETECTOR DATA

Generated by a cutting-edge **quantum-enabled lumped-element axion detector**, this dataset targets the search for dark matter (85% of the universe's matter). It is an ultra-long time series dataset (>1 TB) containing 8-bit integer samples.

The Logic: We inject a clean dark-matter-like signal (Ground Truth) and read it out via a quantum sensor (Noisy Input). **The Task:** Build a Neural Network to denoise the readout and recover the injected signal.

🗂️ **Structure:** HDF5 format. Split into Training (20 files), Validation (20 files), and Science (208 files).

Benchmark 1: Denoising Score (calculated on Validation Set).

🌐 **Benchmark 2:** Dark Matter Limit (Physics result calculated on Science Set).

⚠️ **High Difficulty:** This dataset is extremely large (>1TB). Beginners are strongly suggested to complete other tutorials first. If starting here, please begin with a single training file.

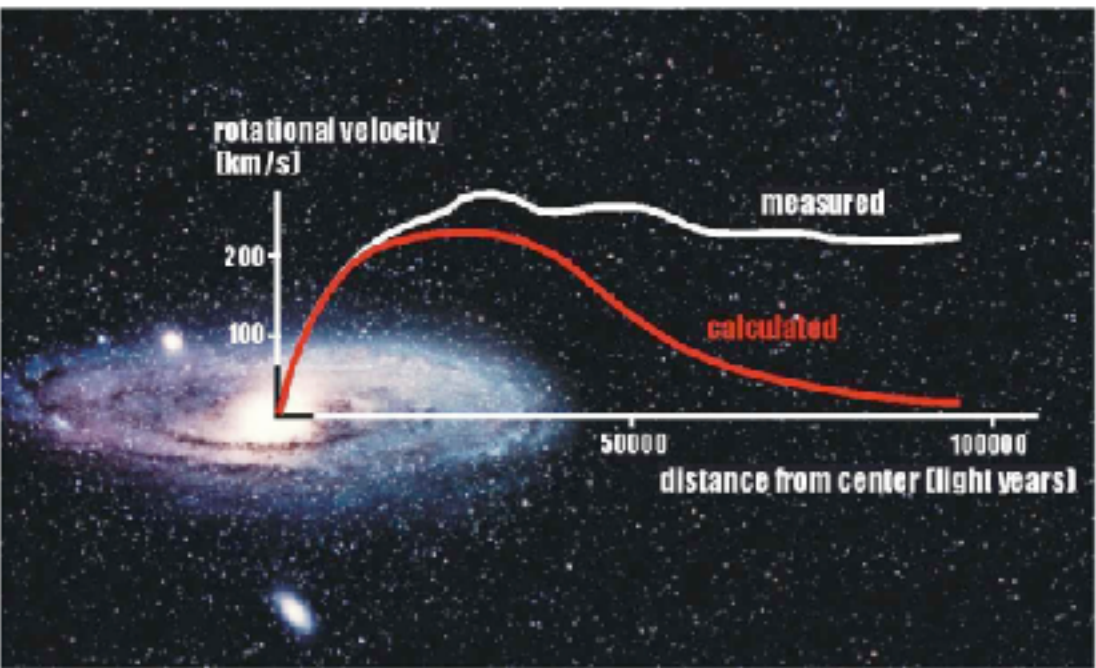
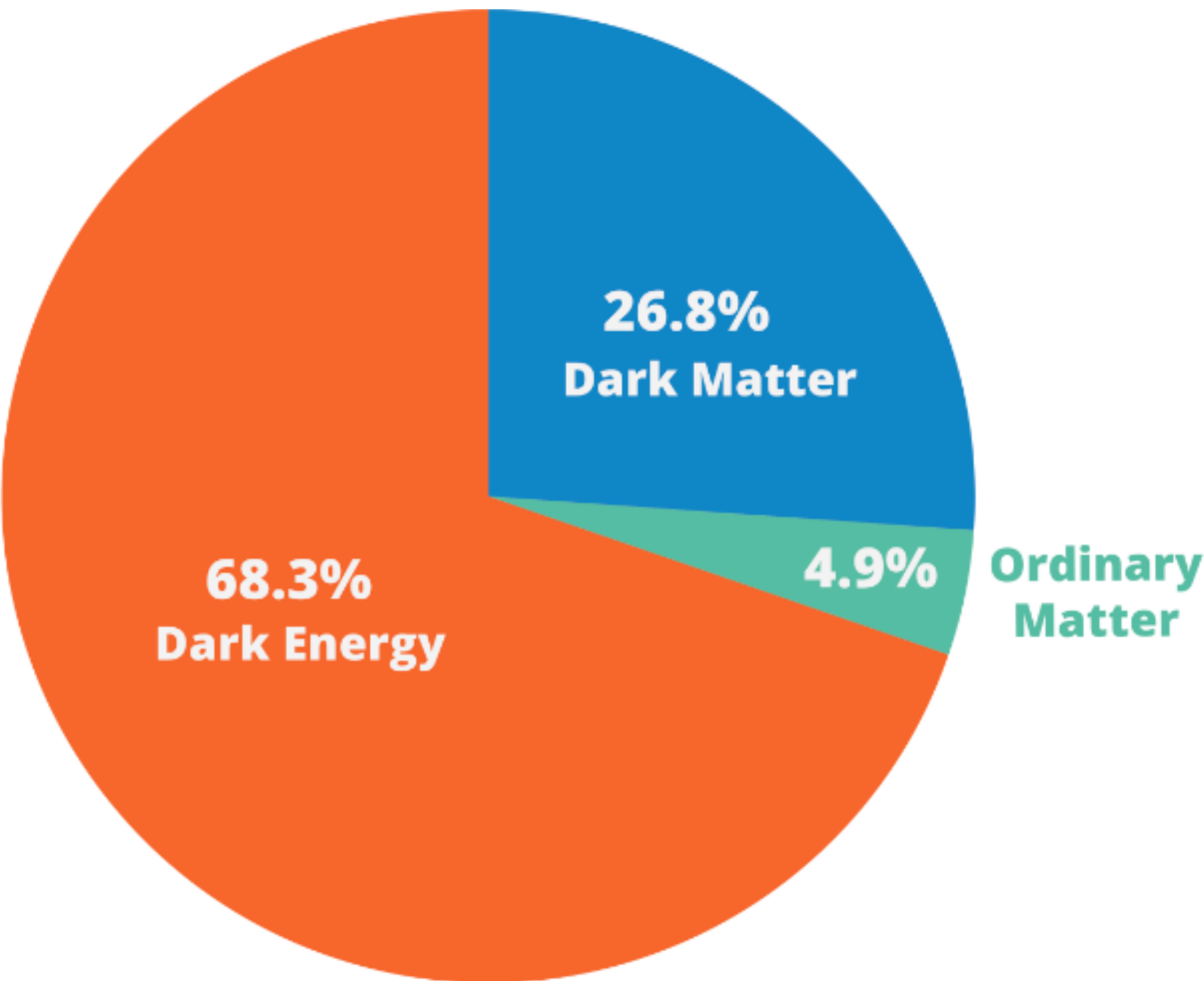
📄 Download & Scripts

📖 Benchmark Paper

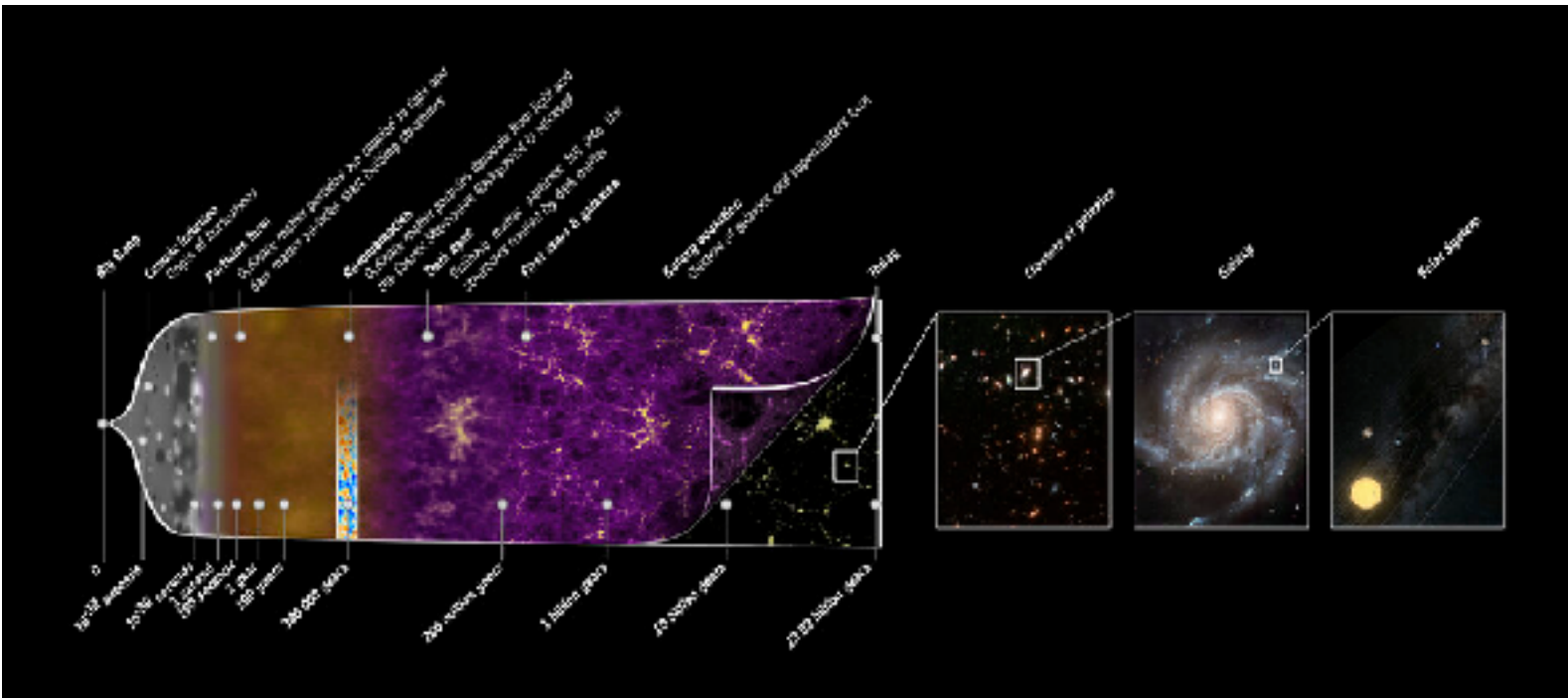
The Physics of TIDMAD Dataset

Dark Matter

The evidence for the existence of dark matter has been plenty



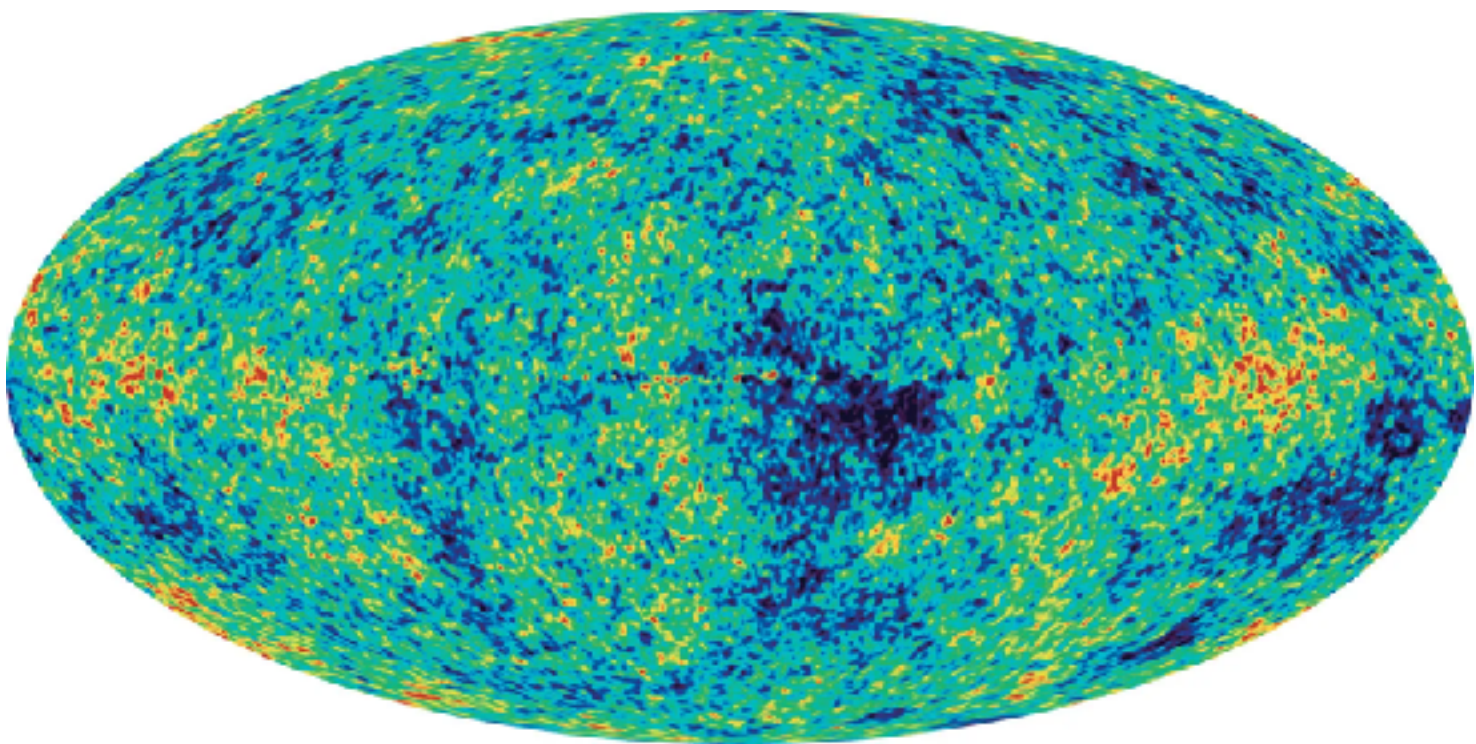
Galaxy Rotation Curve



Large Scale Structure Formation



Gravitational Lens



Cosmic Microwave Background

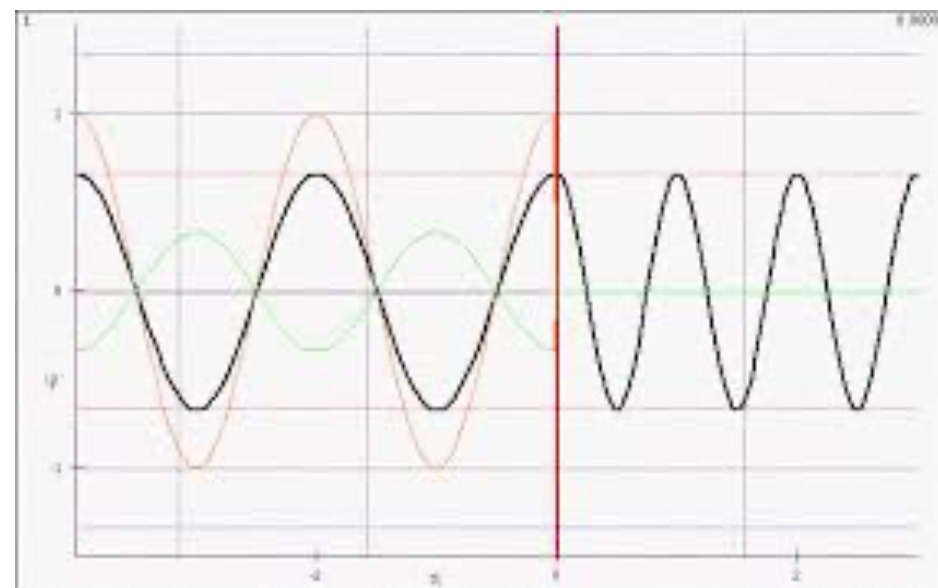
The Physics of TIDMAD Dataset

Dark Matter

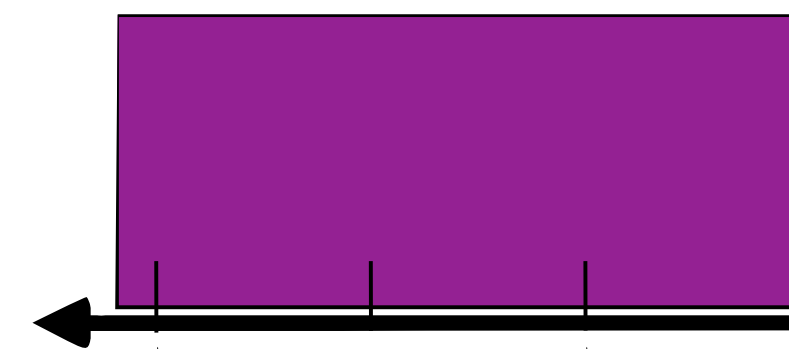
The evidence for the existence of dark matter has been plenty
Many DM candidates have been proposed (WIMP, Axion, etc.)
None has been observed.

Dark Matter can feel like a zoo.

—Prof. Lindley Winslow



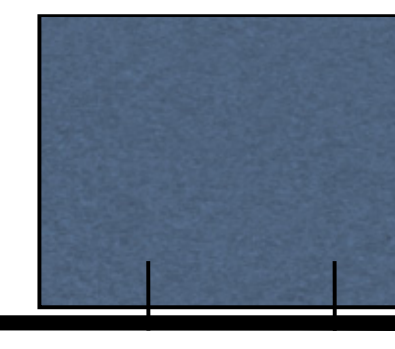
Axion Dark Matter



10^{-12}

10^{-6}

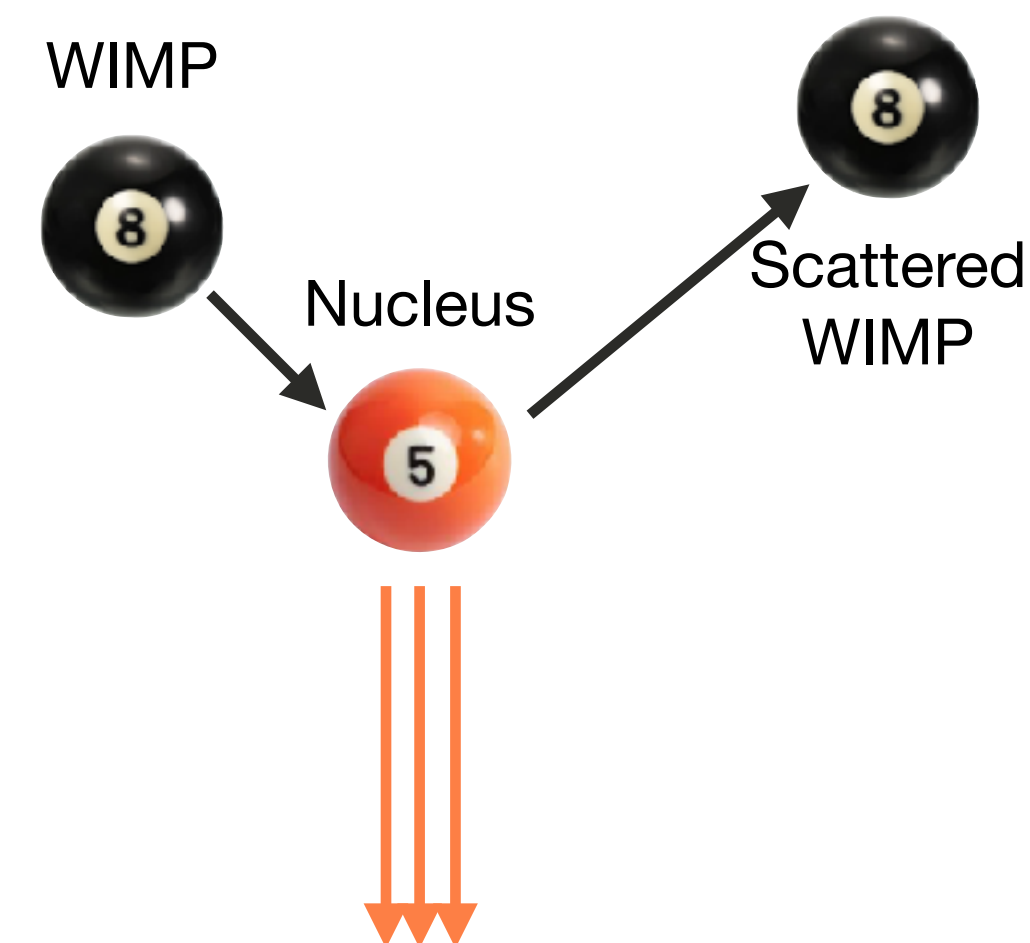
WIMP Dark Matter



10^6

10^{12}

mass [eV]



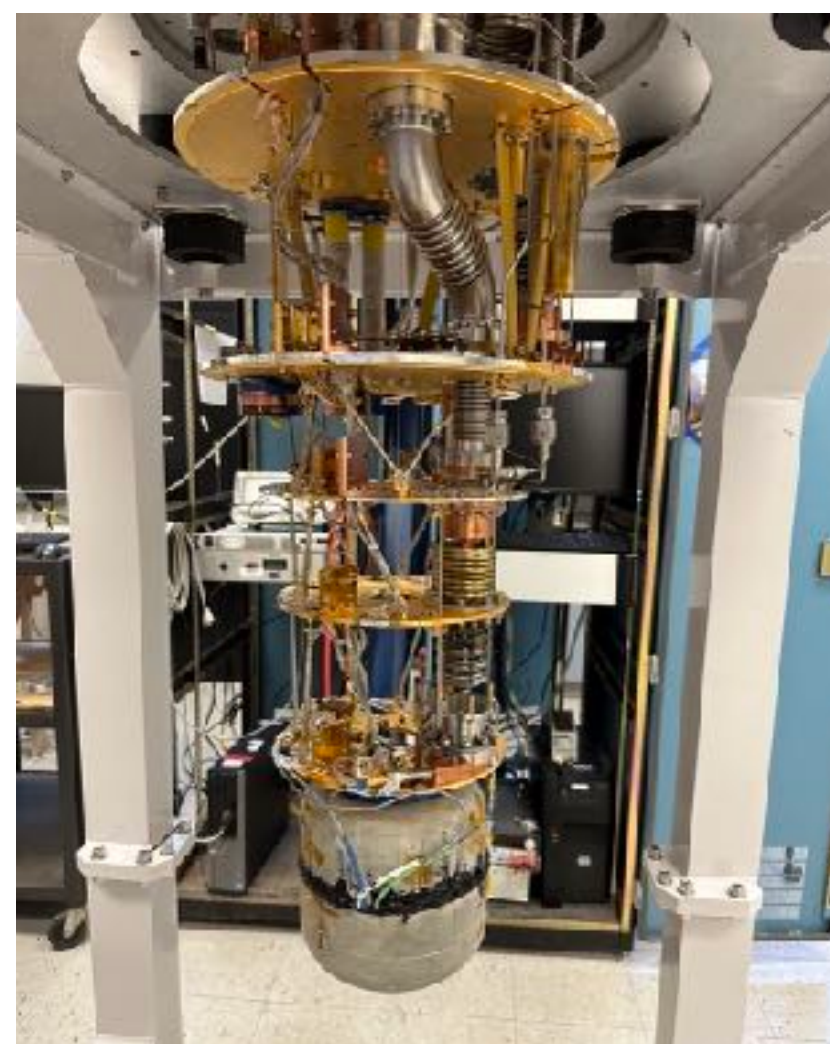


Y. Kahn, B. R. Safdi, and J. Thaler,
Phys. Rev. Lett. 117, 141801

J. L. Ouellet et al.
Phys. Rev. Lett. 122, 121802 (2019)

C. P. Salemi et al.
Phys. Rev. Lett. 127, 081801 (2021)

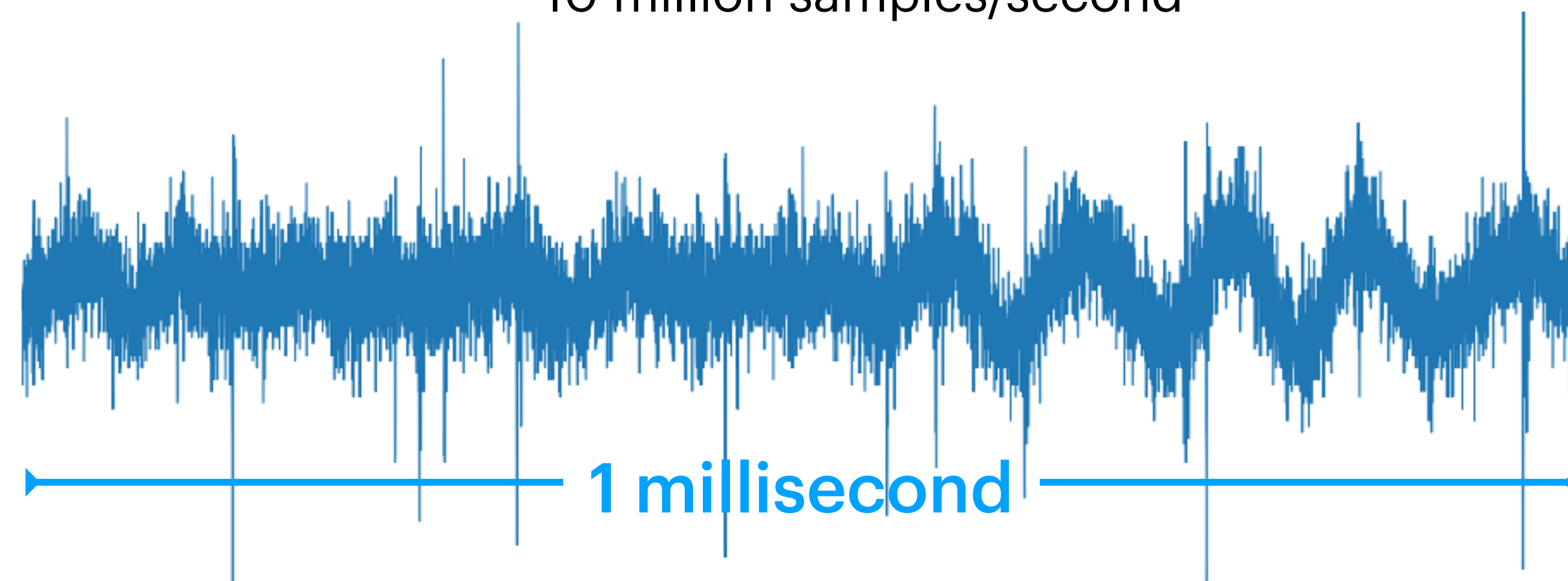
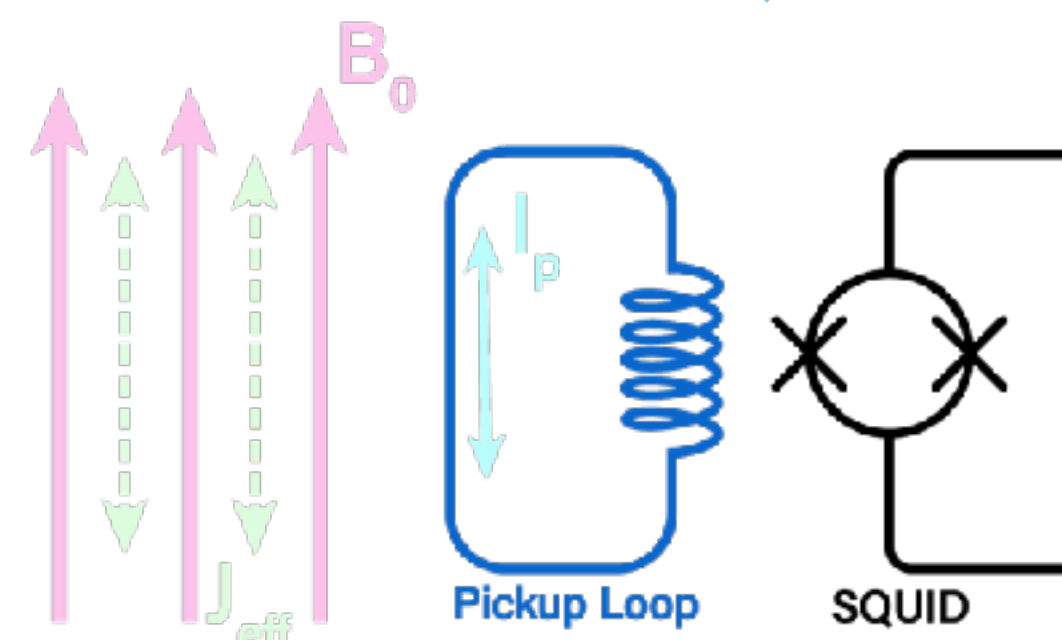
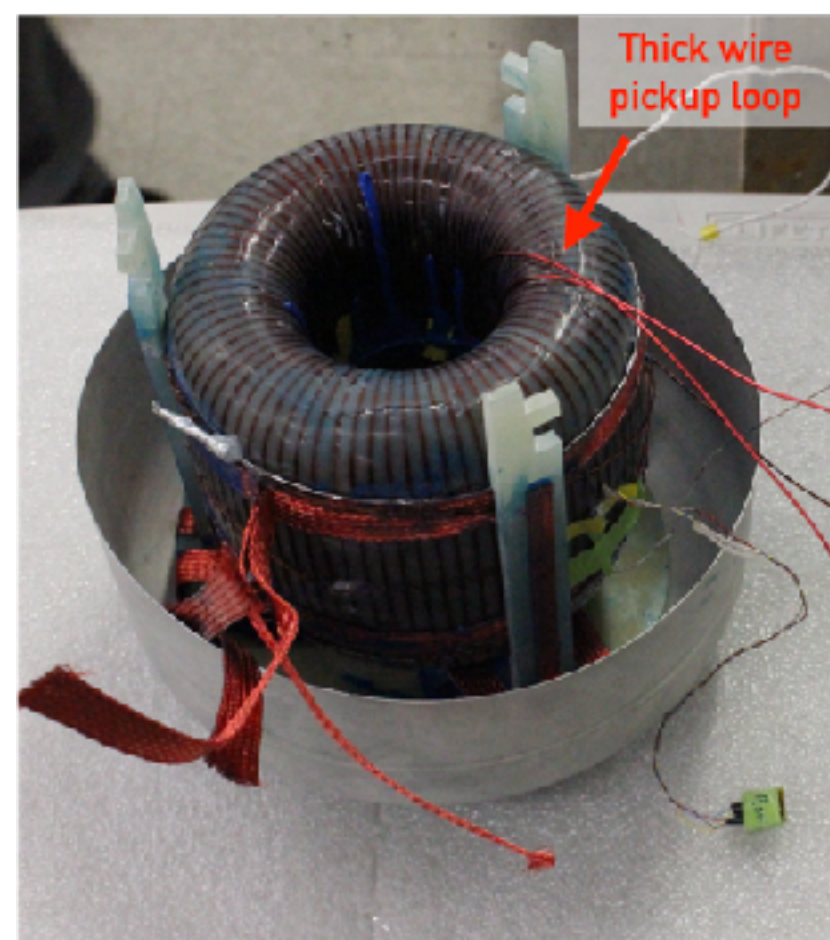
Broadband Axion Dark Matter Search with Toroidal Magnet



Axion-Modified Maxwell's Equation:

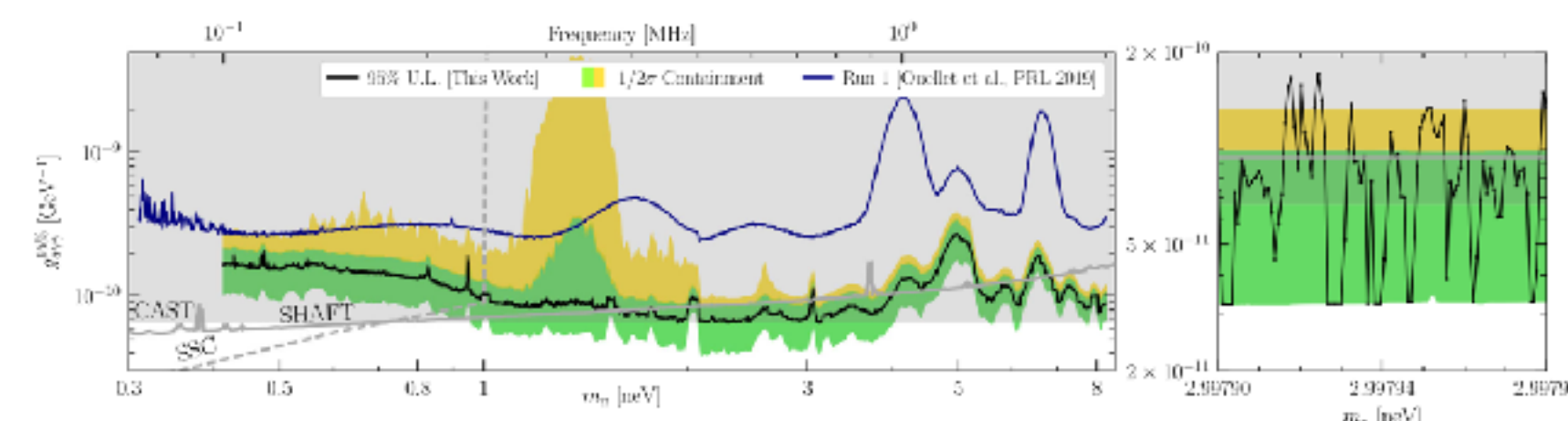
$$\nabla \times B = \frac{\partial E}{\partial t} - g_{a\gamma\gamma}(E \times \nabla a - \frac{\partial a}{\partial t} B)$$

$$J_{eff} = g_{a\gamma\gamma} \sqrt{2\rho_{DM}} \cos(m_a t) B$$



Frequency Spectrum

Broadband search for axion DM



Ultra-long Time Series

10 million samples/second

Experimental Apparatus Constructed by Winslow Lab at MIT



J. T. Fry* , X. H. Fu, Z. Fu, K. M. W. Pappas, L. Winslow, A. Li*

arXiv:2406.04378

NeurIPS 2025 D&B Track Spotlight

TIDMAD: Time Series Dataset for Discovering Dark Matter with AI Denoising

TIDMAD: Time Series Dataset for Discovering Dark Matter with AI Denoising

J. T. Fry^{1*}
jtfry@mit.edu

Xinyi Hope Fu¹
hopefu@mit.edu

Zhenghao Fu¹
fuzh@mit.edu

Kaliroe M. W. Pappas¹
kaliroe@mit.edu

Lindley Winslow¹
lwinslow@mit.edu

Aobo Li^{2*}
aol002@ucsd.edu

¹Department of Physics, Massachusetts Institute of Technology, Cambridge, MA 02139, USA

²Halicioğlu Data Science Institute, Department of Physics, UC San Diego, La Jolla, CA 92093, USA

* Corresponding Authors



Open Data

Release dark matter detector data for AI/ML algorithms



Easy Benchmarking

Design a quantitative benchmarking score to quantify the performance of different algorithms

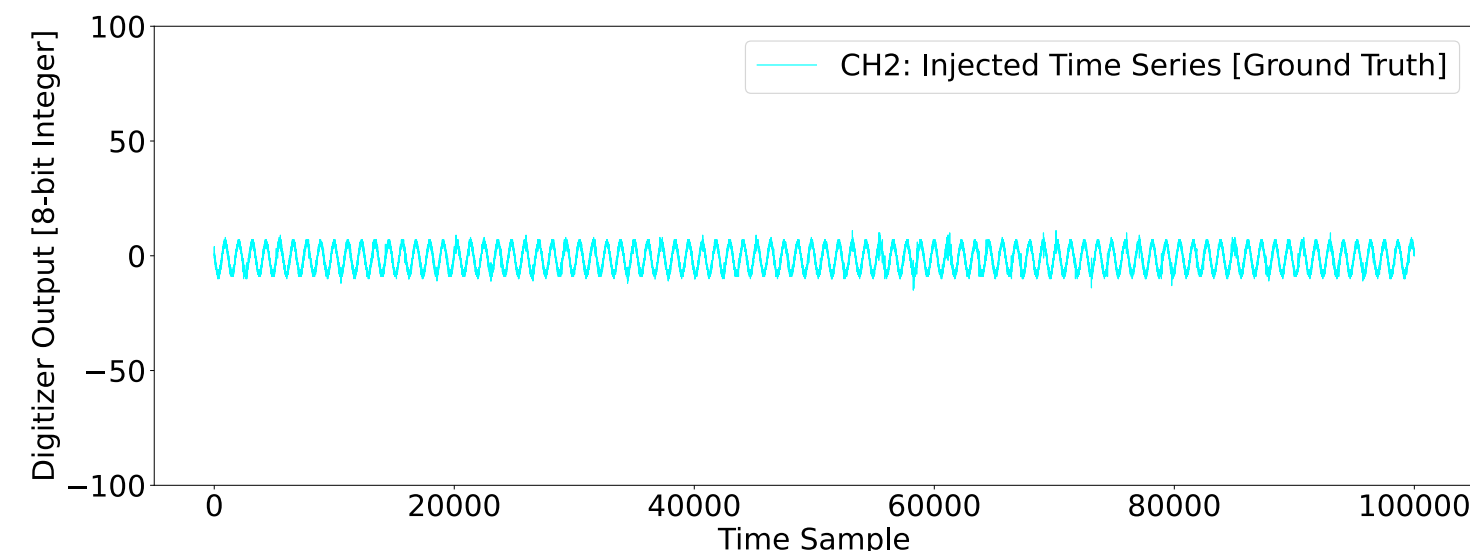


AI for Science

Enables core AI algorithms to extract the signal and produce real physics results thereby advancing fundamental science

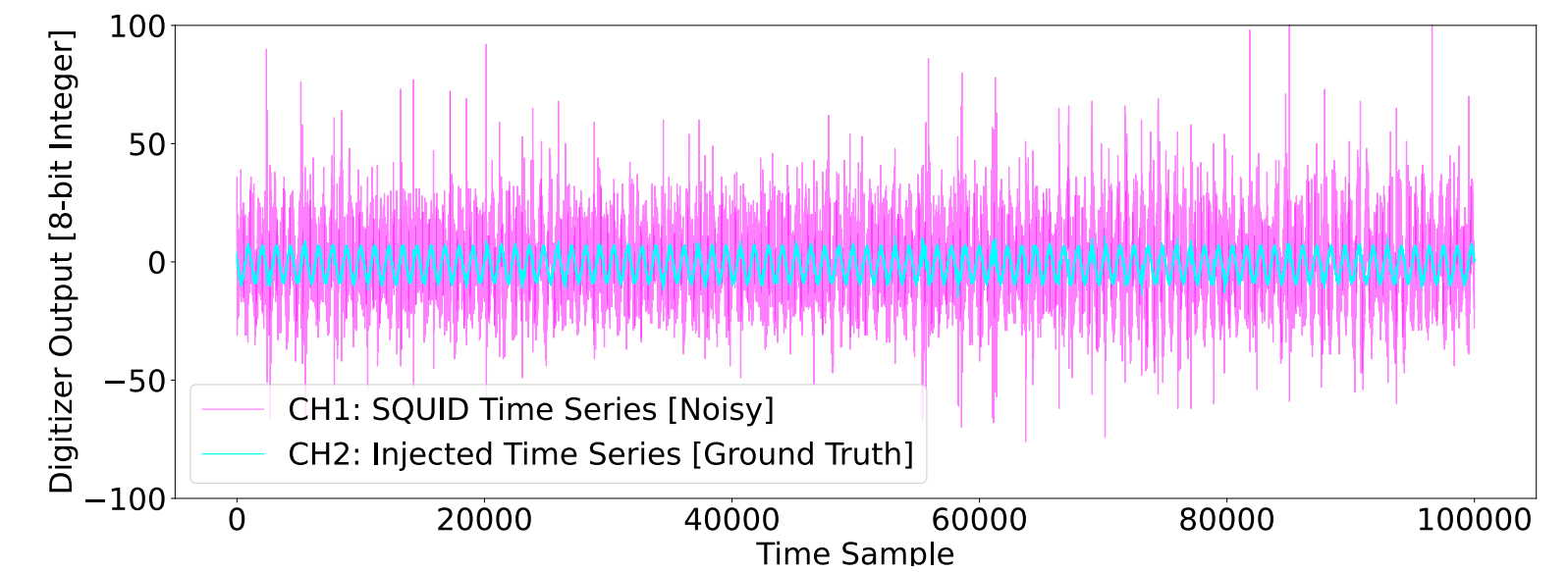
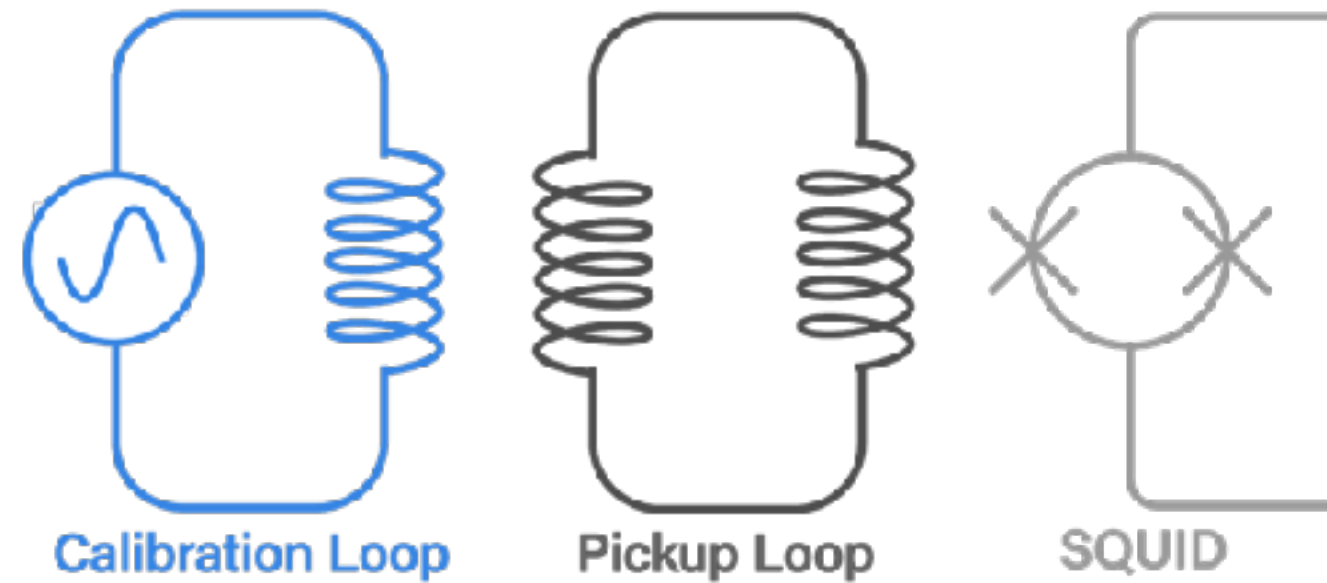


TIDMAD: Time Series Dataset for Discovering Dark Matter with AI Denoising



CH2: Injected Time Series [Ground Truth]

Injected Dark Matter Signal Excitation

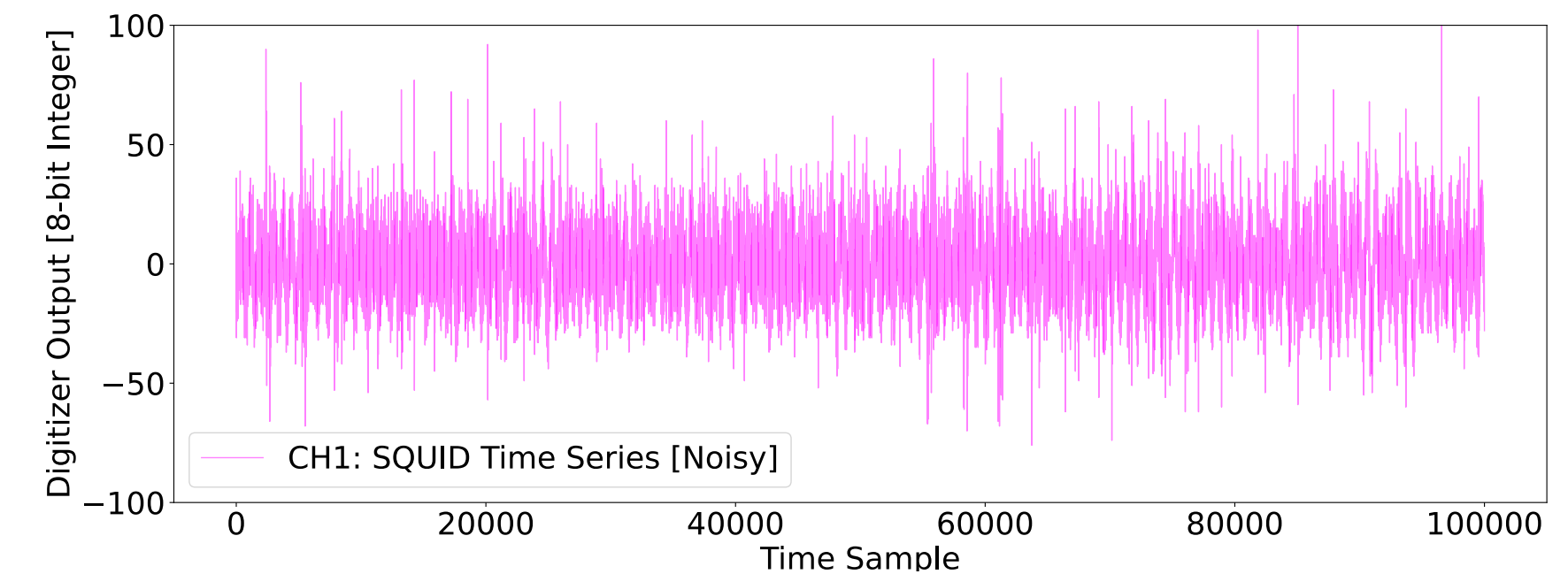
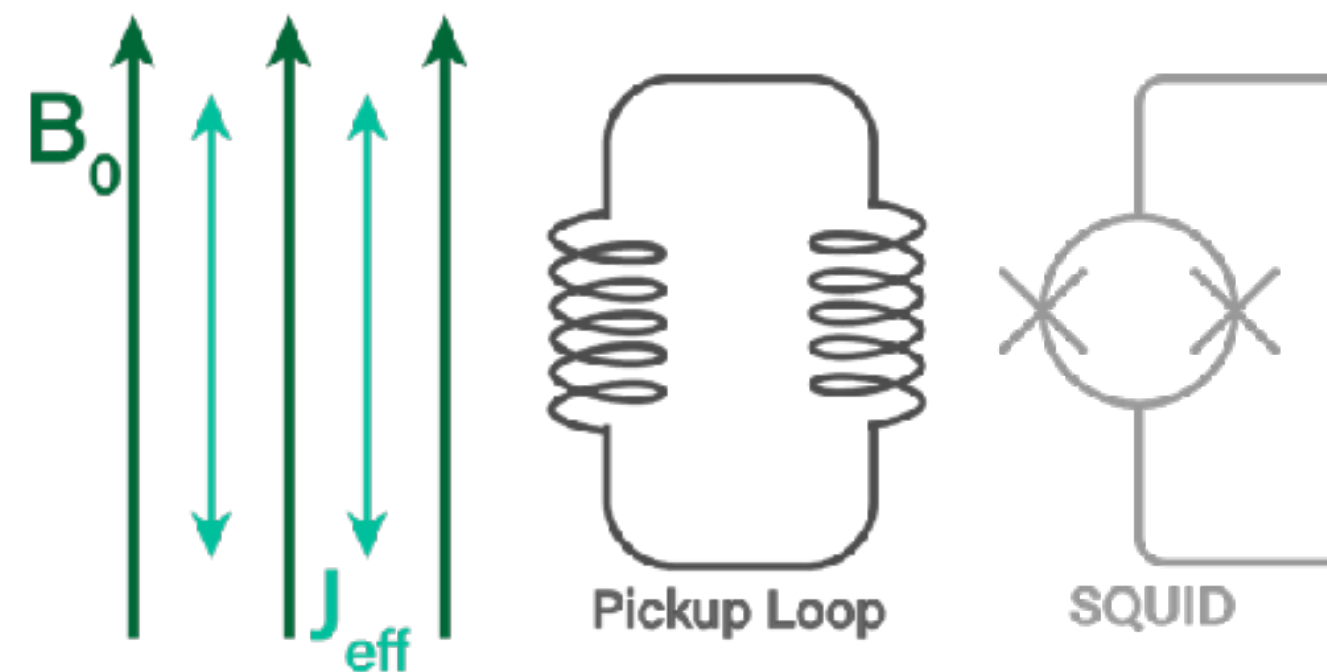


CH1: SQUID Time Series [Noisy]

Train AI denoising model to recover...

No Signal Injected

Real Dark Matter Signal Excitation

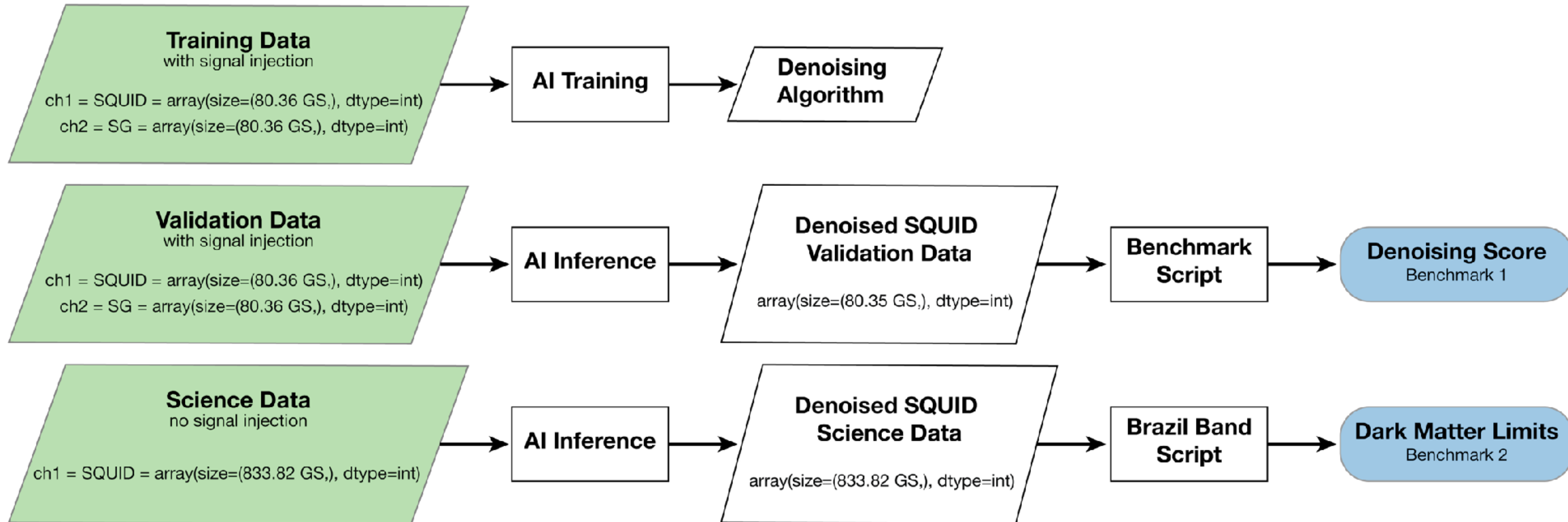


CH1: SQUID Time Series [Noisy]

Use trained AI model to denoise...



TIDMAD: Time Series Dataset for Discovering Dark Matter with AI Denoising



TIDMAD Dataset (ArXiv: 2406.04378)

TIDMAD Data: <https://github.com/jessicafry/TIDMAD>

Code

`python download_data.py`

- `--output_dir, -o`
- `--cache, -c`
- `--train_files, -t #`
- `--validation_files, -v #`
- `--science_files, -s #`
- `-f, --force`
- `-sk, --skip_downloaded`
- `-w, --weak`

- Data downloading files within the GitHub repo
- Requires no dependencies

Cache Selection:

- Data stored at Open Science Data Federation (OSDF) by San Diego Supercomputer Center
- NY (New York); NorCal (Sunnyvale); SoCal (San Diego); **Director** (Default)
- **Director** cache automatically finds the fastest route but can be unstable. If your download fails, manually switch to a regional cache.

Number of Files to Download:

- —train files: 0 to 20
- —validation_files: 0-20
- —science_files: 0-208
- Each file is roughly 2-4Gb, total dataset size over 1Tb

- Download a special “weak signal” version where the signal is only 1/5 of the normal version (more challenging denoising task!)

TIDMAD Dataset (ArXiv: 2406.04378)

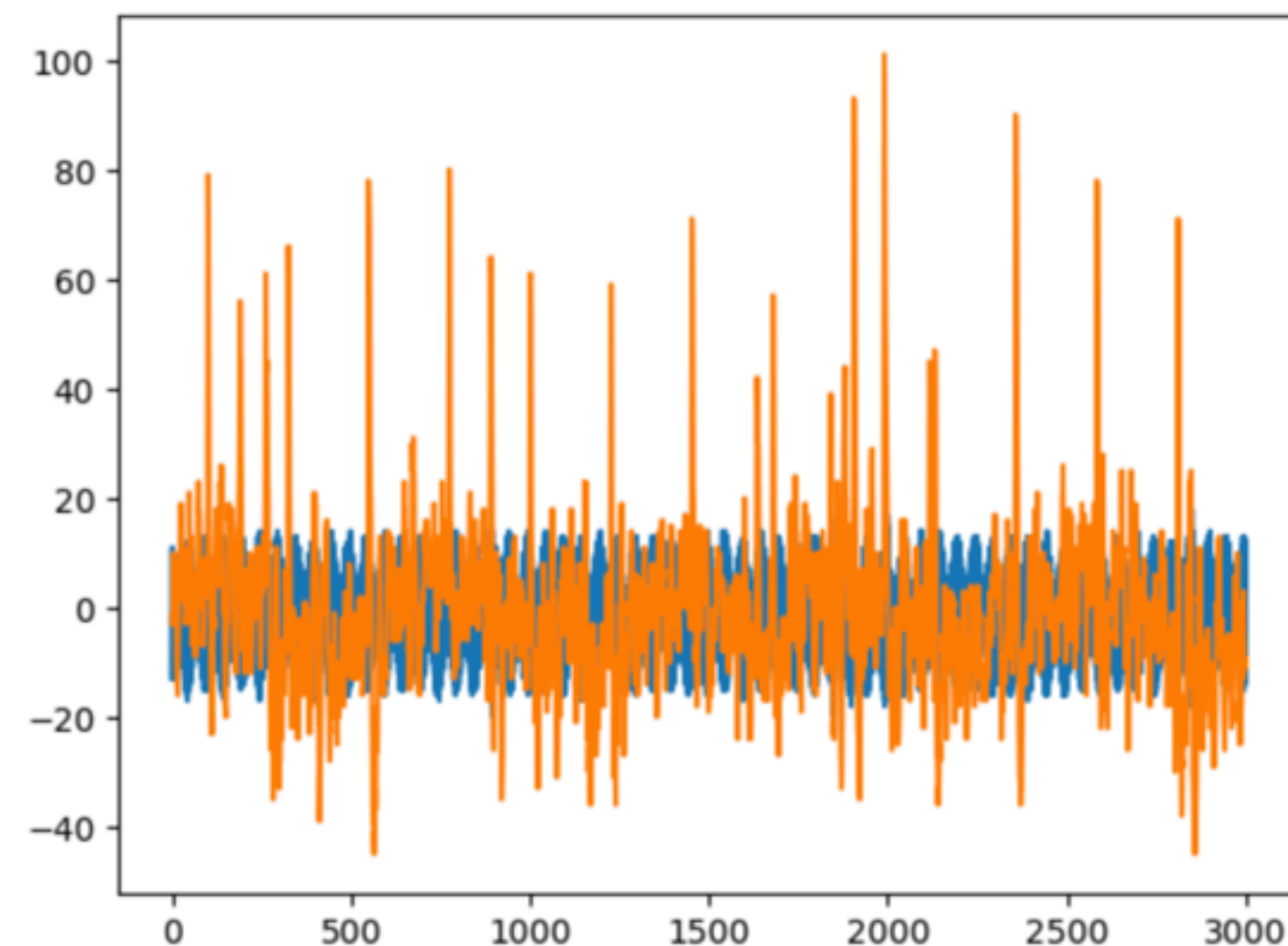
TIDMAD Data: <https://github.com/jessicafry/TIDMAD>

Code

```
with h5py.File("abra_training_0000.h5") as file:
    file.visititems(print_hdf5_structure)
    injected = np.array(file["timeseries"]["channel0002"]["timeseries"][:3000])
    noisy = np.array(file["timeseries"]["channel0001"]["timeseries"][:3000])

    plt.plot(injected)
    plt.plot(noisy)
```

```
Group: timeseries
Group: timeseries/channel0001
Dataset: timeseries/channel0001/timeseries | Shape: (2010000000,) | Type: int8
Group: timeseries/channel0002
Dataset: timeseries/channel0002/timeseries | Shape: (2010000000,) | Type: int8
```



- Ch1 and Ch2 are one-to-one correspondent
- 2 billion samples
- Each sample is a int8 integer (0-255)

1. Training Dataset: abra_training_00{##}.h5 where ## varies from 00 to 19

- Each training .h5 file has the following format:

```
abra_training_0000.h5
├── timeseries
│   ├── channel0001
│   │   └── timeseries (2010000000,)
│   └── channel0002
│       └── timeseries (2010000000,)
```

2. Training Dataset: abra_validation_00{##}.h5 where ## varies from 00 to 19

- Each validation .h5 file has the following format:

```
abra_validation_0000.h5
├── timeseries
│   ├── channel0001
│   │   └── timeseries (2010000000,)
│   └── channel0002
│       └── timeseries (2010000000,)
```

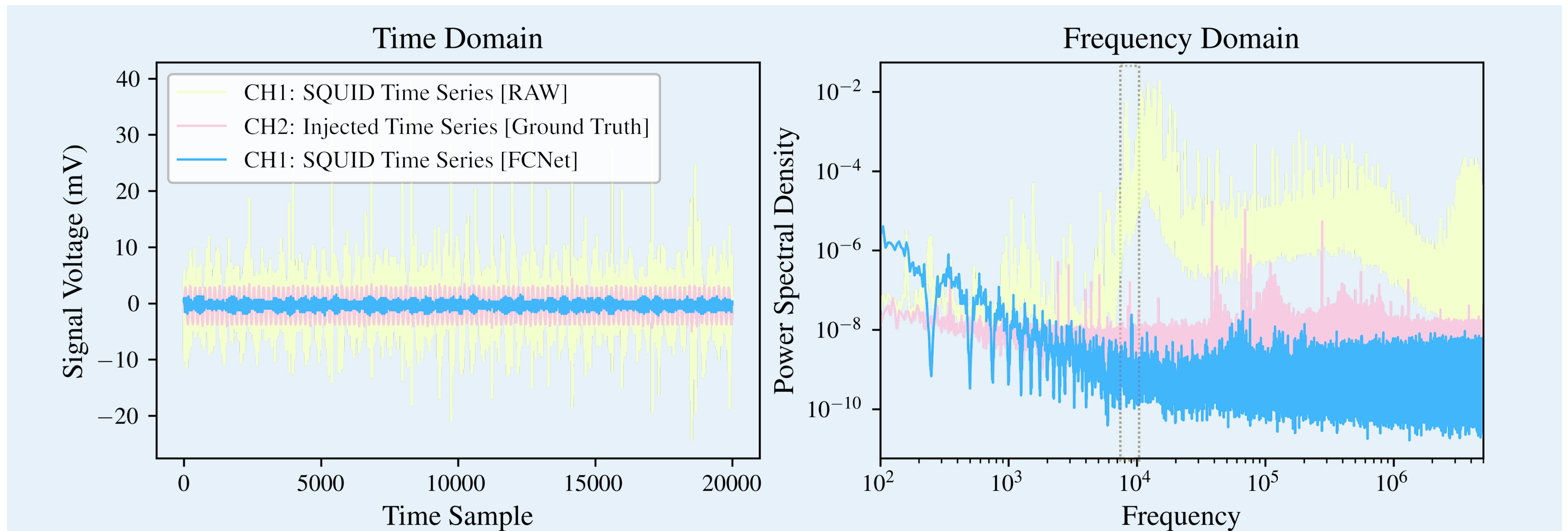
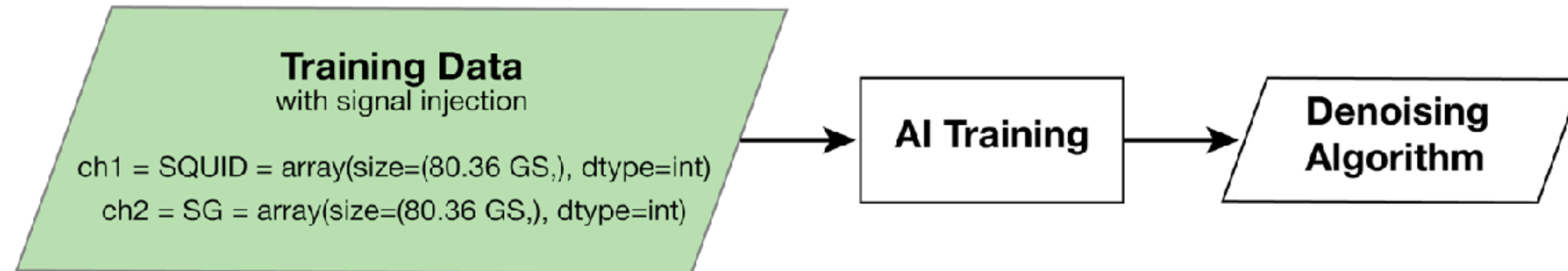
3. Science Dataset: abra_science_0{###}.h5 where ### varies from 000 to 207

- Each science .h5 file has the following format:

```
abra_science_0000.h5
├── timeseries
│   └── channel0001
│       └── timeseries (4010000000,)
```

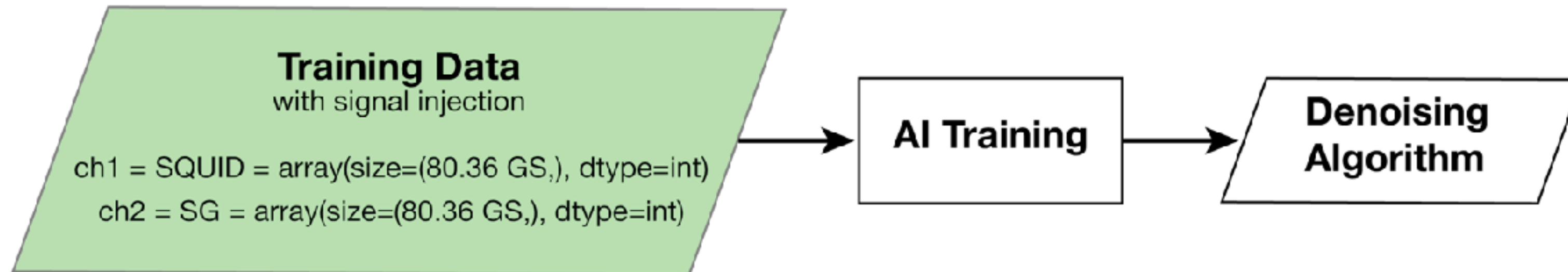
TIDMAD Dataset (ArXiv: 2406.04378)

TIDMAD Data: <https://github.com/jessicafry/TIDMAD>



TIDMAD Dataset (ArXiv: 2406.04378)

TIDMAD Data: <https://github.com/jessicafry/TIDMAD>



Code

python train.py

- *-d [directory]*
- *-m [model]*

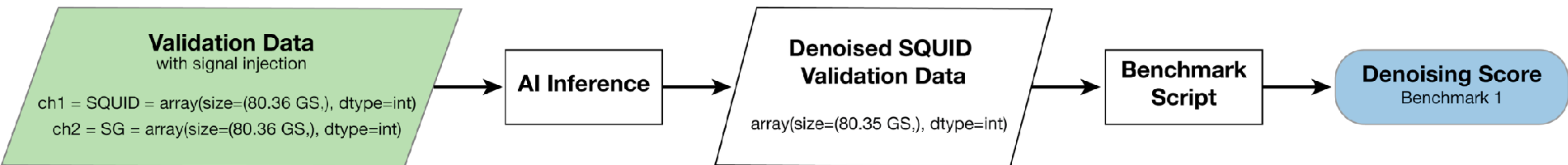
Directories of downloaded TIDMAD Dataset

Select which ML model to train, options includes:

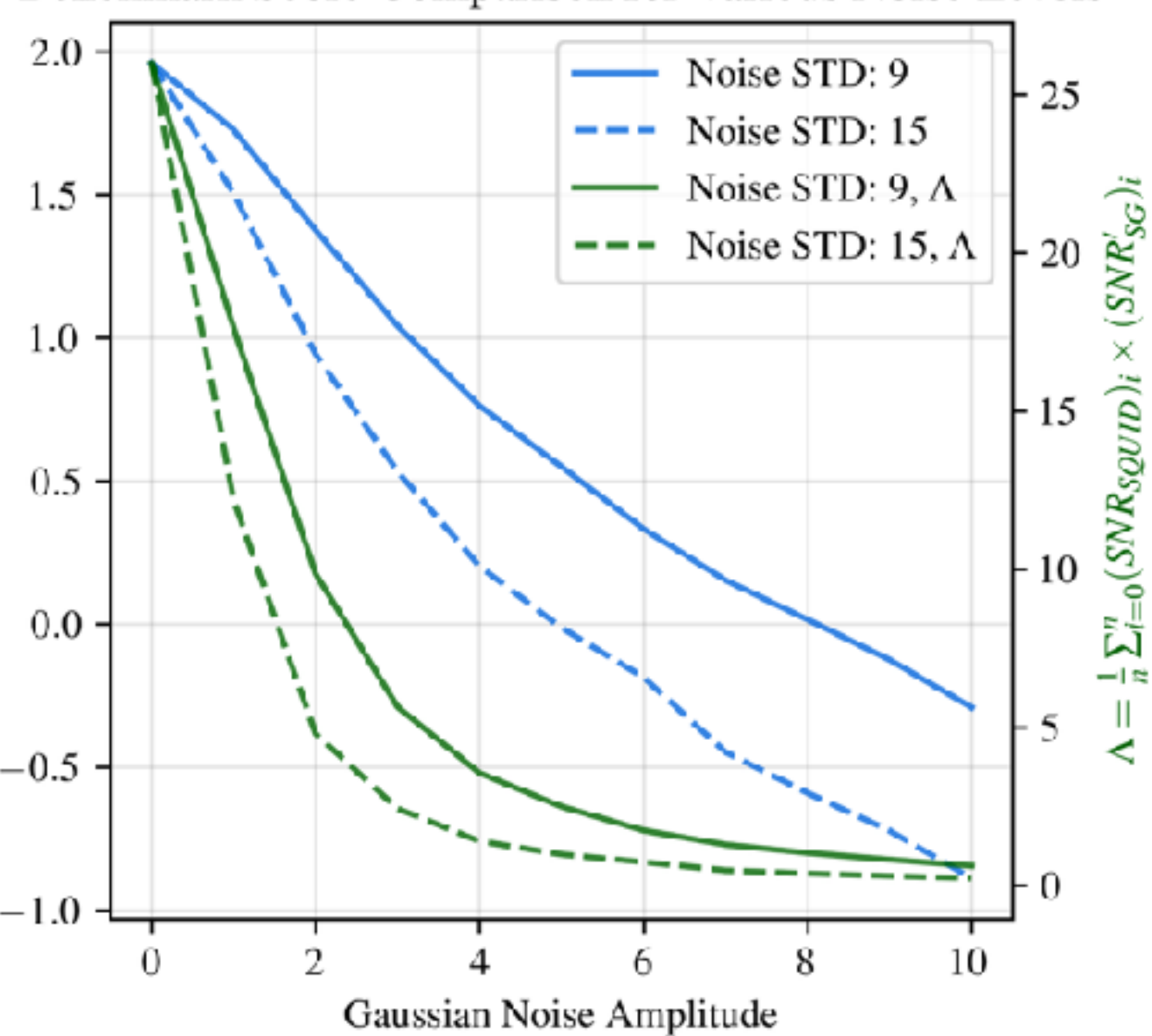
- **fcnet**: Fully Connected Neural Networks
- **punet**: Positional U-Net
- **transformer**: transformer encoder layers
- **wavenet**: WaveNet with Causal Convolution Blocks
- **rnn**: Recurrent Neural Network Seq-to-Seq model

TIDMAD Dataset (ArXiv: 2406.04378)

TIDMAD Data: <https://github.com/jessicafry/TIDMAD>



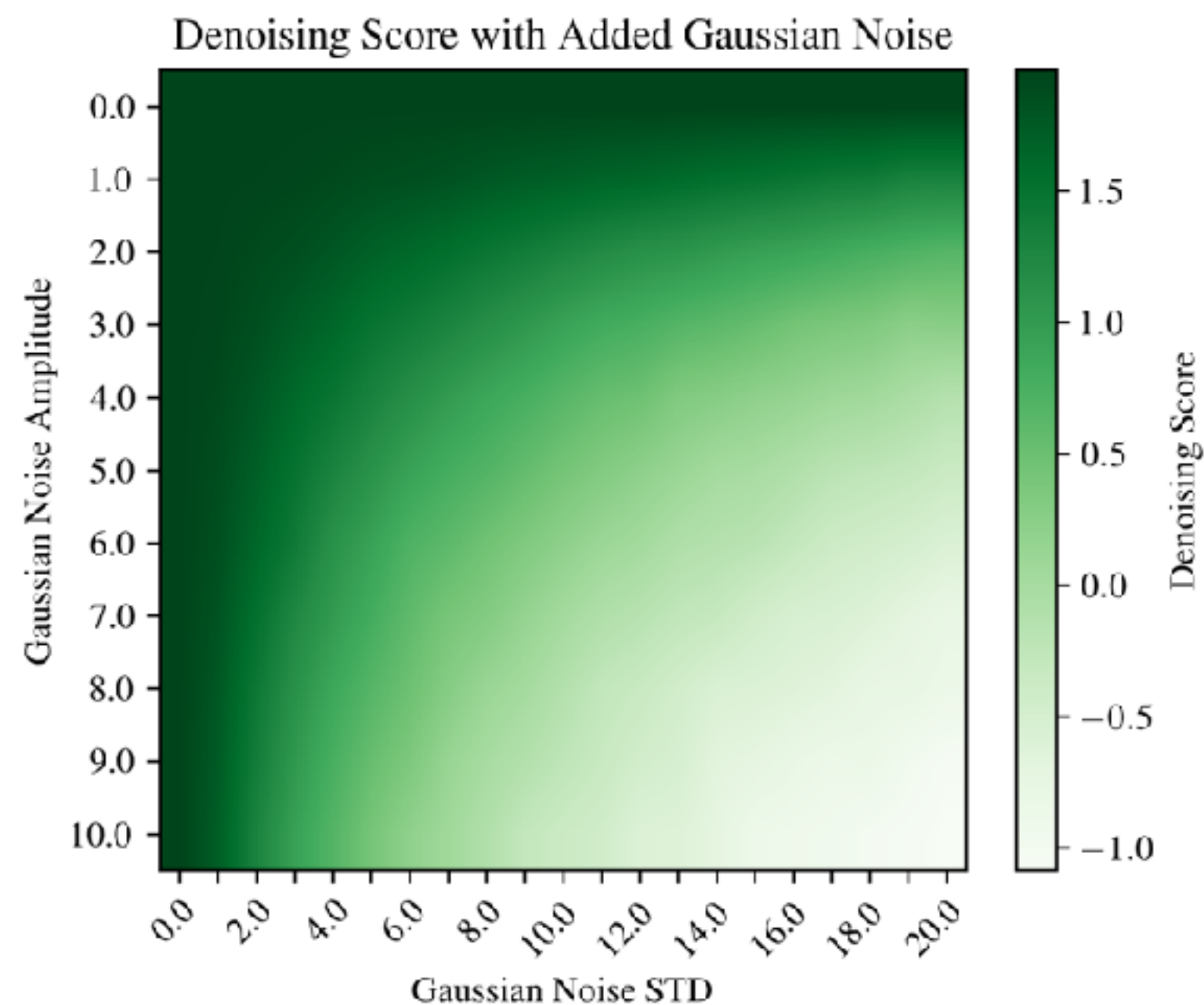
Benchmark Score Comparison for Various Noise Levels



$$\Lambda = \left(\frac{1}{n} \sum_{i=0}^n (SNR_{SQUID})_i \times (SNR'_{Injected})_i \right)$$

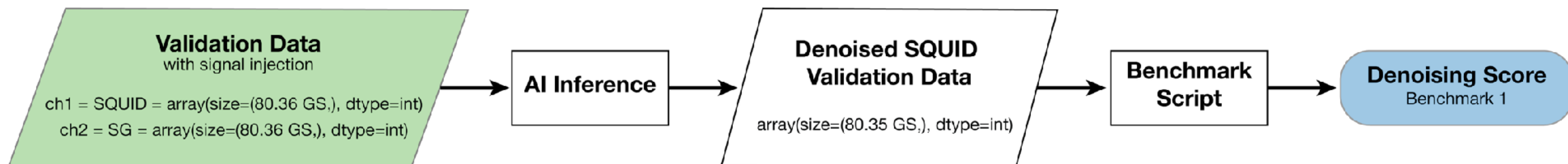
$$\text{Denoising Score} = \log_{5.27} \Lambda$$

Test the denoising score by doping gaussian noise into Time Series



TIDMAD Dataset (ArXiv: 2406.04378)

TIDMAD Data: <https://github.com/jessicafry/TIDMAD>



Code

python inference.py ←

- *-d [directory]*
- *-m [model]* ←

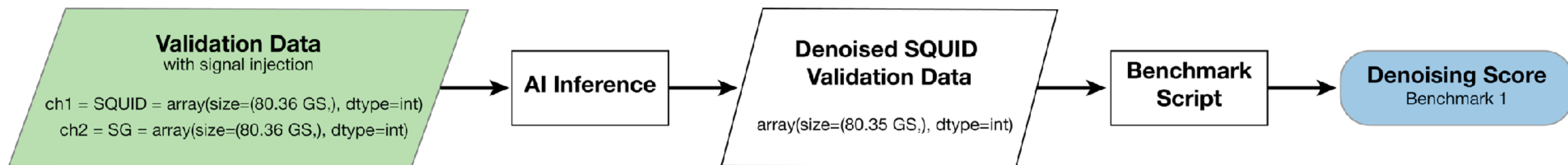
For each validation file *abra_validation_00{##}.h5* in *[directory]* a denoised validation file *abra_validation_denoised_[model]_00{##}.h5* will be generated.

Select which model to apply denoising, options includes:

- **mavg**: Moving Average filter [non-ML]
- **Savgol**: Savitzky-Golay filter [non-ML]
- **fcnet**: Fully Connected Neural Networks
- **punet**: Positional U-Net
- **transformer**: transformer encoder layers
- **wavenet**: WaveNet with Causal Convolution Blocks
- **rnn**: Recurrent Neural Network Seq-to-Seq model

TIDMAD Dataset (ArXiv: 2406.04378)

TIDMAD Data: <https://github.com/jessicafry/TIDMAD>



Code

`python inference.py`

- `-d [directory]`
- `-m [model]`

Code

`python benchmark.py -d -m`

- `-c --coarse`
- `-p --parallel`

Calculate coarse denoising score instead of fine denoising score

Parallelize the running of the score calculation script

- Usually takes 4 hours to calculate fine denoising score, 8 core parallelization reduce this to 30min

TIDMAD Dataset (ArXiv: 2406.04378)

TIDMAD Data: <https://github.com/jessicafry/TIDMAD>

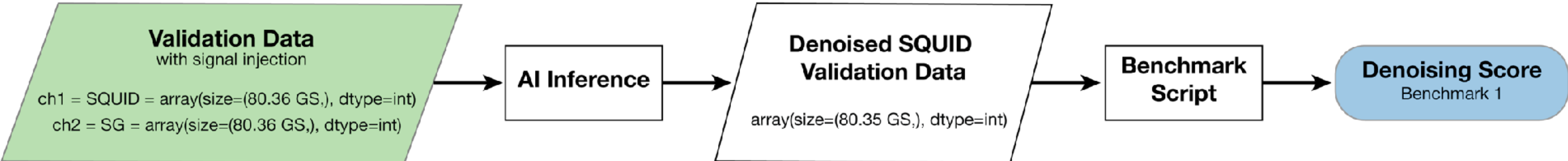
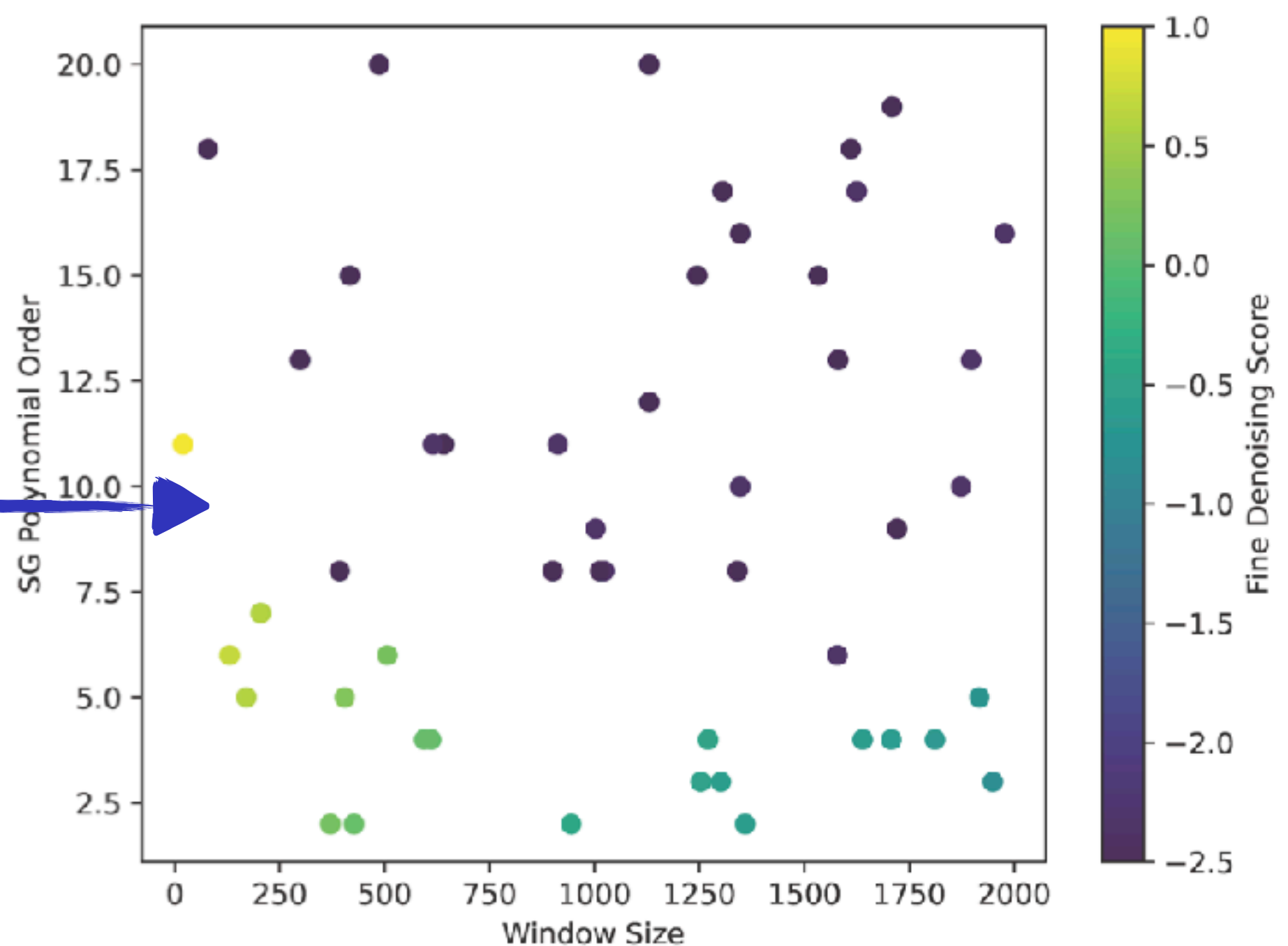


Table 1: Fine and coarse denoising score for raw data, traditional algorithms, and trained ML models. FS means frequency splitting, or training multiple versions of the same model to handle different frequency ranges. The detail of segment size and FS is discussed in Section B.

Algorithms	Segment Size	FS	Parameters	Fine Score	Coarse Score
None		Traditional Denoising		1.00	1.10
Fourier Averaging	1×10^8	–	10-fold Average	0.24	0.26
Moving Average	1×10^6	–	window = 100	0.86	0.95
SG Filter	1×10^6	–	window = 19, order = 11	0.95	1.04
FC Net	4×10^4	Yes	See Appendix B	6.43	6.55
PU Net	4×10^4	Yes	See Appendix B	3.69	3.84
Transformer	2×10^4	Yes	See Appendix B	3.95	4.18
WaveNet	4×10^4	No	See Appendix B	4.99	5.16
RNN Seq2Seq	4×10^4	Yes	See Appendix B	3.38	3.79



AI Denoising

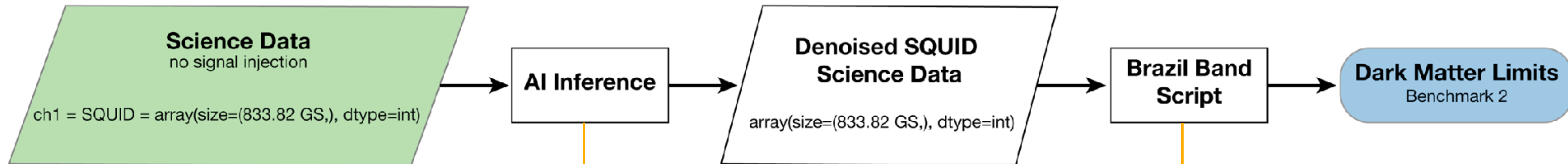
TIDMAD Dataset

J. T. Fry* , X. H. Fu, Z. Fu, K. M. W. Pappas, L. Winslow, A. Li*

arXiv:2406.04378

NeurIPS 2025 D&B Track Spotlight

TIDMAD Data: <https://github.com/jessicafry/TIDMAD>



Code

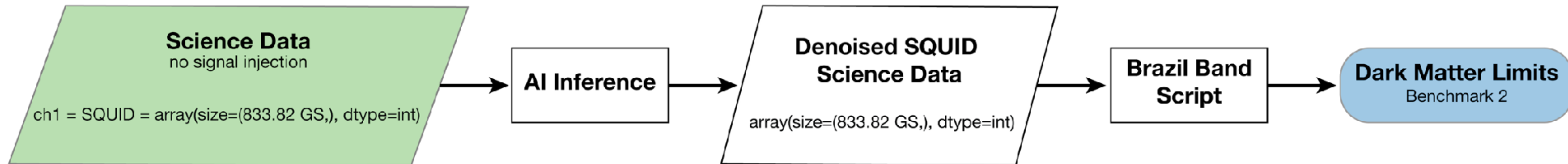
Run `python process_science_data.py -d [directory] -m [model]` to generate the denoised time series over the 208 science files provided.

Code

- `python brazilband.py [path] [files] [output file name (no extention)] --level coarse --v` to generate dark matter limit in [outpt].csv and brazil band plot in [output].png
- `AxionPhoton_TIDMAD.ipynb` to produce the global Dark Matter Limit plot.

TIDMAD Dataset

TIDMAD Data: <https://github.com/jessicafry/TIDMAD>



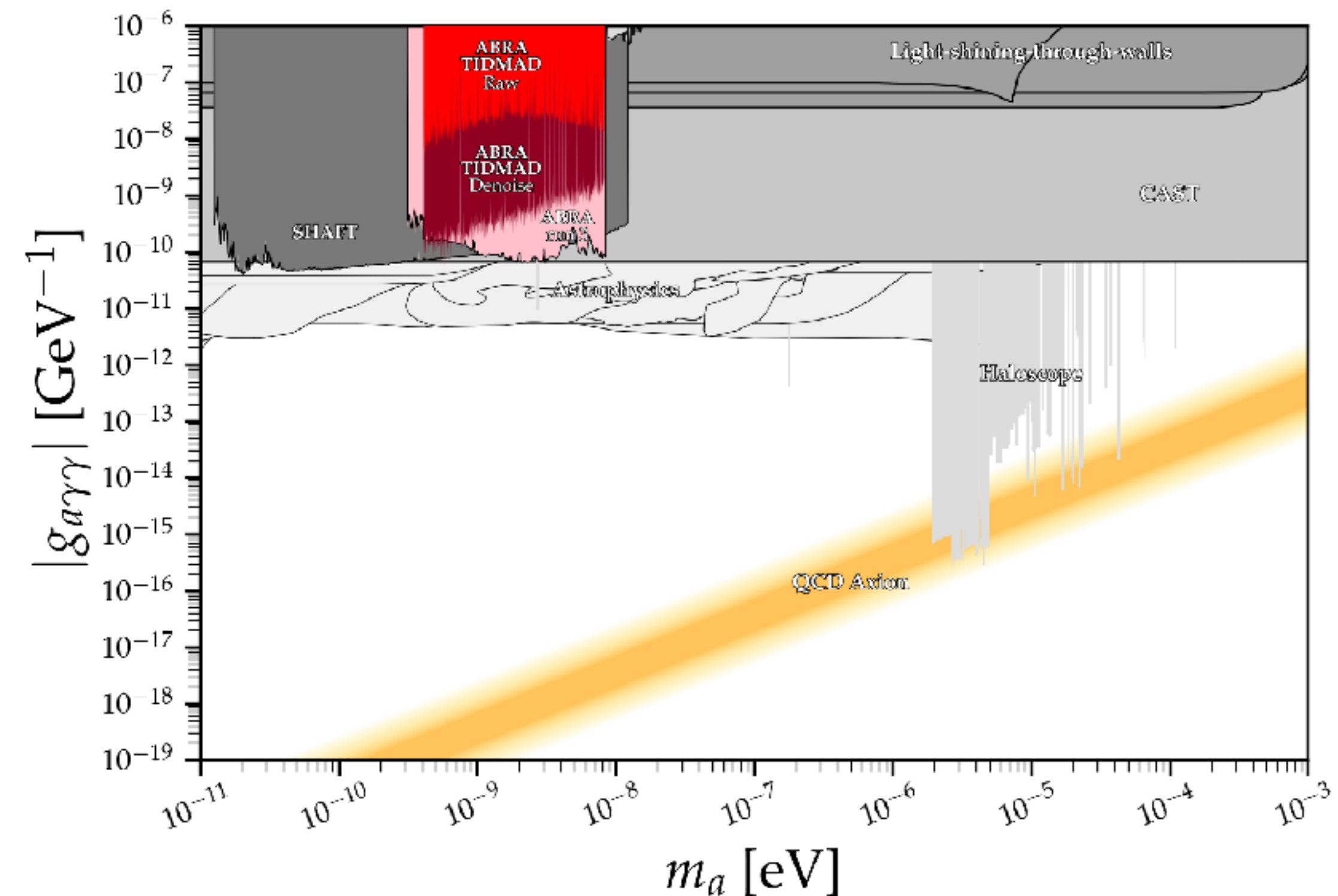
⚡ Axion Limit Boost

ABRA TIDMAD Raw: 24 hr data, no denoising

ABRA TIDMAD Denoised: 24 hr data with FCNet denoising

ABRA Run 3: 2,400 hr data, no denoising

Efficient denoising algorithms increased Axion search limit by **1-2 orders of magnitude**, approaching the previous world-leading ABRA run 3 results with only **1%** of statistics



$0\nu\beta\beta$ AI Summer School

- A satellite event of Neutrino 2026
- Releasing additional **AI-Ready Datasets:**
 - CUORE **Time Series**
 - KamLAND-Zen **Point Cloud**
 - EXO-200 **2D Image**
 - NEXT **3D Image**
 - SuperNEMO **3D Track**
- Will be made available on the event website:
 - <https://0vbb-ai.physics.ucsd.edu>



20-21 June 2026
UC Irvine Colloquium 1200
<https://0vbb-ai.physics.ucsd.edu>

Some Useful Links



AI Tutorial: <https://aobol.github.io/AoboLi/#tutorials>

0vββ AI Summer School: <https://0vbb-ai.physics.ucsd.edu>



The Practical Machine Learning

<https://pire.gemadarc.org/education/school21/#ai>

2024 Summer Bootcamp on Deep Learning and Applications

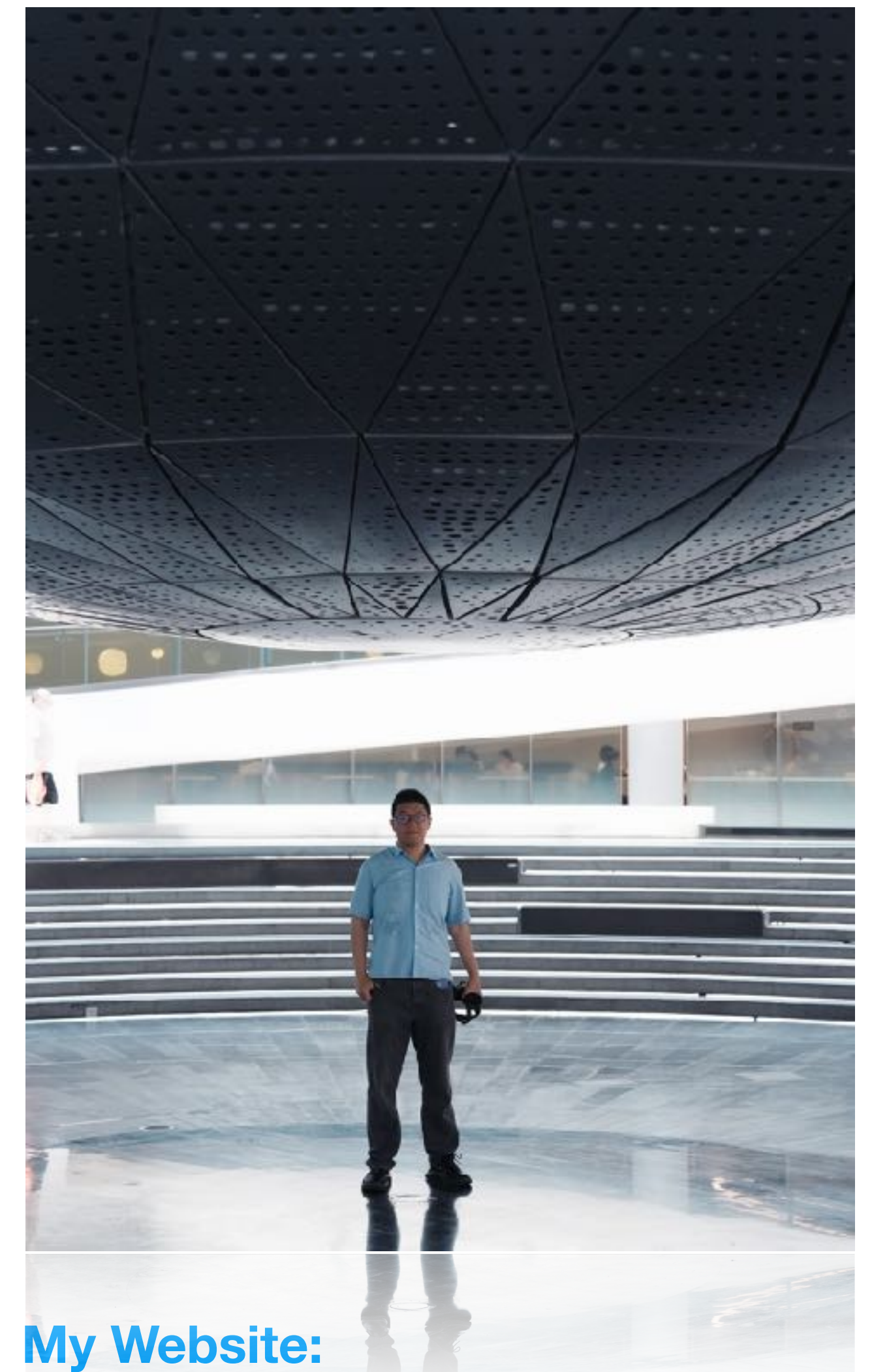
<https://ai-bootcamp2024.github.io/>

MIT 6.S191 Introduction to Deep Learning

<http://introtodeeplearning.com/>

Andrew Ng: Deep Learning Specialization

Link



My Website:

<https://aobol.github.io/AoboLi/>

My Email Addresses:

aol002@ucsd.edu